

Entity Framework



Note Source

Ahmed Mamdoh + Entity Framework Core in Action



Created By
Abdelrhman Elsaid

EF Learning Path

00-Introduction

01-Database First setup

02-EF behavior with IEnumerable & IQueryable

03-Relationships & Navigation Properties

04-CRUD & Change Tracking

05-Code First

06-Migrations

07-Configuration & Data Annotations

08-Advanced Relationships & Teamwork

09-EF Core 5 (Migrations)

10-EF Core 5 Model Configuration (Fluent API)

11-EF Core 5 Data Loading (Lazy, Explicit, Eager)

12-EF Core 5 (Evaluation, Tracking, Global Query...

Handling Stored Procedures

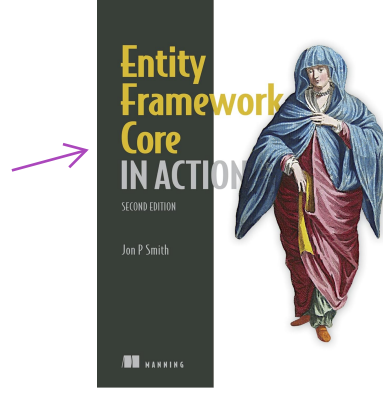
How to Start your Projects with Code First

Inheritance Mapping (TPH & TPT)

Owned Types

Reading in hierarchical data efficiently

Relational Fixup Stage



Introduction

1. مقدمة وتاريخ الـ Data Access في .NET

إحنا اتكلمنا قبل كده عن الـ LINQ وقلنا إنه اتعمل عشان يوحد طريقة كتابة الكود اللي بـ Access بيه الداتا، سواء الداتا دي كانت في Memory، أو XML، أو Database.

تاريخ ظهور التكنولوجيا (2008)

في سنة 2008 ظهرت الـ LINQ ومعها 3 حاجات:

1. الـ LINQ to Objects: عشان نتعامل مع داتا في الميموري.
 2. الـ LINQ to SQL: عشان نتعامل مع داتا في الـ SQL Database.
 3. الـ LINQ to XML: عشان نتعامل مع ملفات XML.
- في نفس السنة (نص 2008)، تيم تاني جوه مايكروسوفت طلع تكنولوجيا تانية اسمها **Entity Framework (EF)**.
- المشكلة: بقي عندنا طريقتين عشان نكلم الداتابيز: (LINQ to SQL) و (Entity Framework).
 - القرار: مايكروسوفت قررت بعد فترة توقف الـ LINQ to SQL Support وتركز كل استثمارها في الـ **Entity Framework**، وبقت هي الطريقة الـ Standard لينا كـ .NET Developers.

نسخ الـ Entity Framework

- الـ EF 6.x: دي آخر نسخة ظهرت من الـ EF الكلاسيك، ودي اللي هنتشغل بيها النهاردة.
- الـ EF Core: دي ظهرت بالتوازي مع الـ .NET Core، وهنتكلم عنها وعن الفروقات بينهم قدام.

2. يعني إيه ORM؟ ولية بنستخدمه؟

الـ Entity Framework بتتدرج تحت مسمى **ORM** اختصار لـ **Object Relational Mapper**.

مشاكل الـ ADO.NET (ليه فكرنا في بديل؟)

قبل الـ EF، كنا بنستخدم ADO.NET وكانت بتقابلنا مشاكل كتير خلطنا ندور على حل جديد:

1. الـ **Impedance Mismatch** (اختلاف الطبيعة):
 - إحنا في الكود شغالين **Object Oriented (Classes & Objects)**.
 - الداتابيز شغالة **Relational (Tables & Rows)**.
 - كان لازم نعمل عملية **Mapping** بايدينا عشان نحول الـ Rows لـ Objects والعكس.
2. تكرار الكود **(Repetitive Code)**:
 - عشان تعمل **Select**، بتكتب دالة وتفتح **Connection** وتعمل **Command**.
 - عشان تعمل **Update**، بتعيد نفس الخطوات.

- أسئلة الطلبة كانت بتدور حول إننا بنكتب كود كثير عشان نعمل حاجات بسيطة، ولو غيرنا حاجة في الداتابيز بنضطر نغير في أماكن كثير في الكود.

3. الـ **Magic Strings & No Validation**:

- الكويريز في الـ ADO عبارة عن **Strings**.
- لو كتبت اسم عمود غلط، الكومبايلر مش هيعترض، والمشكلة هتظهر بس وقت التشغيل (Runtime Exception).

4. الـ **Data Type Conversion**:

- الداتا بتجيلك في شكل **DataRow** ، وكل الخلايا عبارة عن **Object** .
- بتضطر تعمل Casting و Parsing بايديك (لو **int** تعمل **int.Parse** ، ولو **null** تهندله).

دور الـ **EF** كـ **ORM**

الـ Entity Framework بيحل المشاكل دي عن طريق:

1. الـ **Automatic Mapping**: هو اللي بيبيّن الـ **Classes** بناءً على شكل الداتابيز أوتوماتيك.
2. الـ **Querying**: بيساعدنا نكتب كود **C# (LINQ)** وهو يحوله لـ **SQL**.
3. الـ **CRUD Operations**: بيسهل عمليات الـ **Insert, Update, Delete**.

Database First setup

3. طرق الربط مع الداتابيز (Approaches)

عندنا 3 طرق عشان نربط الـ Application بالـ Database:

1. الـ **Database First**: (عندي داتابيز جاهزة وهخلي الـ EF يعمل الكود).
2. الـ **Model First (Design First)**: (برسم الموديل وهو يعمل الداتابيز والكود).
3. الـ **Code First**: (بكتب كود #C وهو يعمل الداتابيز). *ودي الطريقة الأهم والمستقبلية.*

ملحوظة: إحنا هنشرح النهاردة **Database First** عشان نفهم الفكرة، لكن الشغل الحقيقي والمستقبل كله للـ **Code First**، خصوصاً إن الـ Design First اتلغى في الـ EF Core.

4. تطبيق عملي: Database First (Design First)

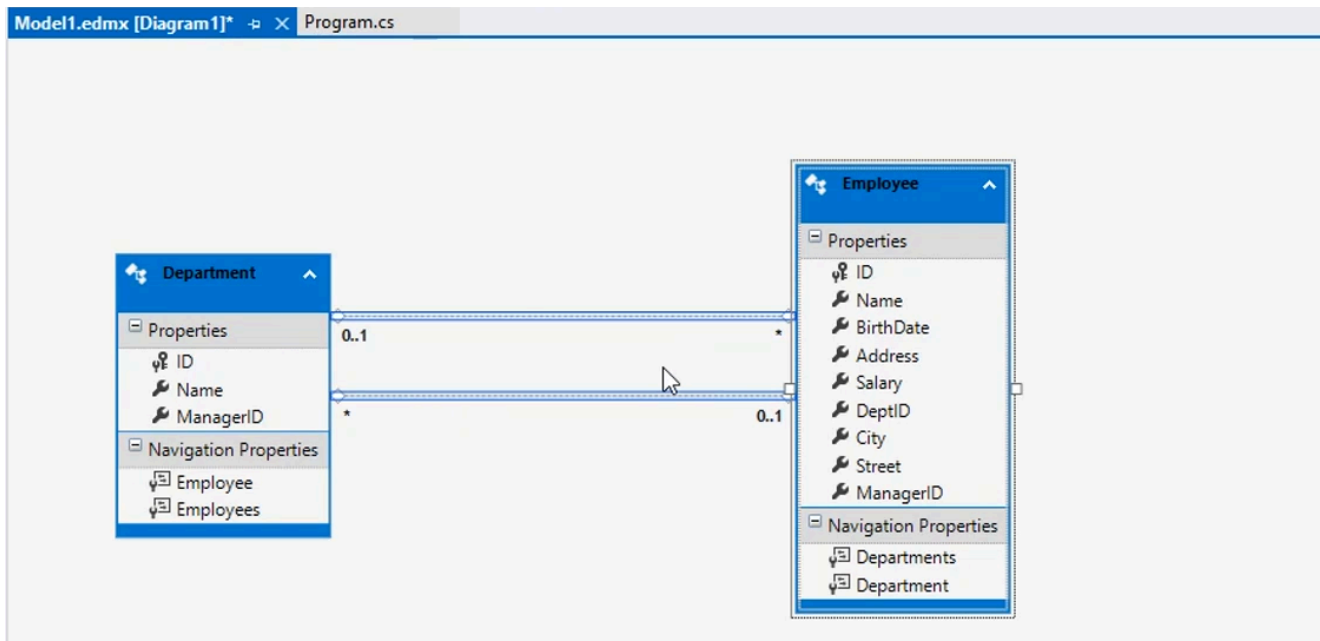
الخطوات اللي عملناها في المحاضرة عشان نربط بـ Database موجودة (Company DB):

1. على الـ Visual Studio، نعمل `Add New Item`.
2. نختار `Data -> ADO.NET Entity Data Model`.
3. نختار `EF Designer from database`.
4. نعمل `New Connection` ونربط على الـ SQL Server (داتابيز Company).
5. الـ Wizard هيسألك تحفظ الـ Connection String في ملف الـ `App.config` باسم إيه؟ (سييناه `CompanyEntities`).
6. نختار الجداول اللي محتاجينها (اختارنا `Employee` و `Department`).

الناتج بعد الـ Wizard

بعد ما الـ Wizard خلص، الـ EF عمل `Generate` لمجموعة فايلات مهمة جداً:

أ. ملف الـ `edmx`.



- ده ملف XML بيوصف الداتابيز والـ Mapping.
- الـ Visual Studio بيعرضهولنا في شكل **Designer** (رسمه للجدول والعلاقات بينهم).

ب. الـ Generated Classes (Models)

generated from a template.

to this file may cause unexpected behavior in

to this file will be overwritten if the code i

الـ EF بص على الداتابيز وراح عمل **Classes** بتمثل الجداول دي. مثلاً جدول **Department** في الداتابيز، اتحول لـ **Class** اسمه **Department** في الكود:

```

1 public partial class Department
2 {
3     public int ID { get; set; }
4     public string Name { get; set; }
5     public Nullable<int> ManagerID { get; set; } // عشان في الداتابيز بيقبل NULL
6     // ...
7 }

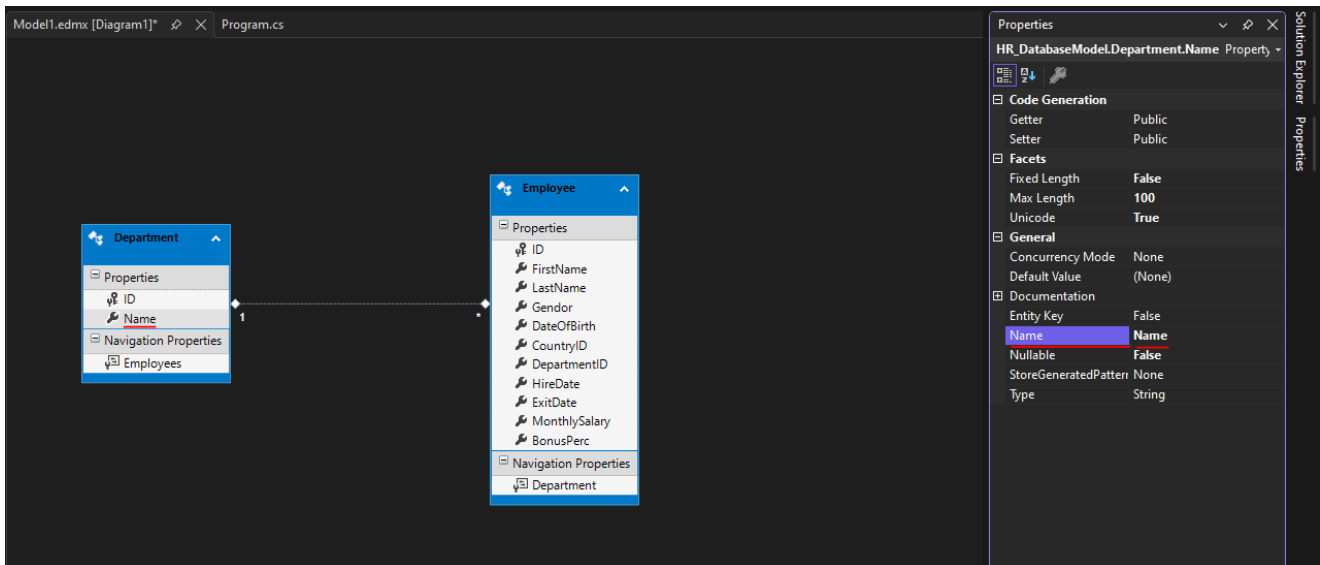
```

ملاحظات على الـ Mapping الأوتوماتيك:

- الـ **Nullable Handling**: لو العمود في الداتابيز بيقبل NULL ، الـ EF بيخليه **Nullable<T>** في السي شارب (زي **ManagerID**).
- الـ **Data Types**: لو العمود **nvarchar** بيتحول لـ **string** ، ولو **int** بيتحول لـ **int**.

- الـ **Separation of Concerns**: تقدر تغيير اسم الـ Property في الكلاس (مثلاً تسميها DeptName بدل Name) من غير ما تأثر على الداتابيز، لأن فيه طبقة Mapping في النص.

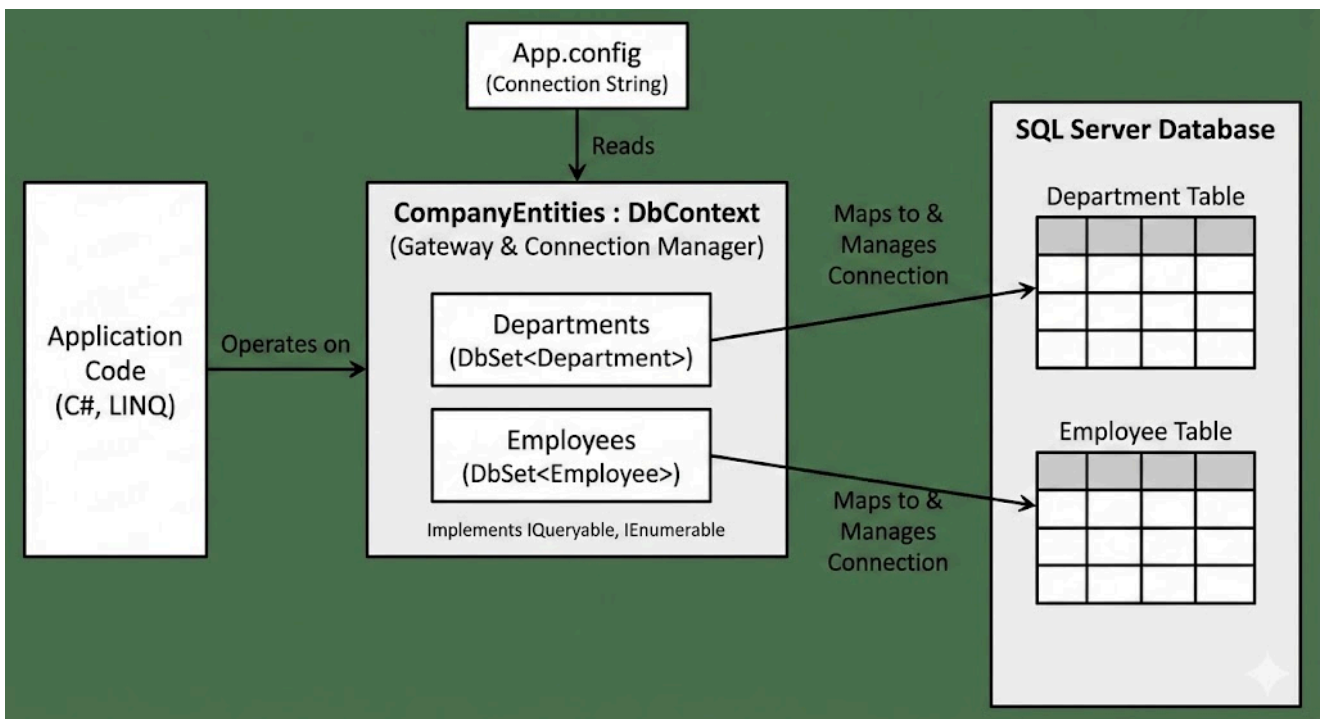
مثال لتغيير اسم Property هنا مثلاً من قائمة Properties تقدر تغيير أى حاجه بس خلى بالك ده تغيير فى الـ class structure ملوش علاقة بجداول الـ SQL الاصلية انت هنا بس بتشكل الحاجات فى الـ class زى ما انت عاوز



5. الـ DbContext (حلقة الوصل)

أهم كلاس الـ EF عمله هو الكلاس اللي بيورث من الـ DbContext . في حالتنا اسمه CompanyEntities .

وظيفة الـ DbContext:

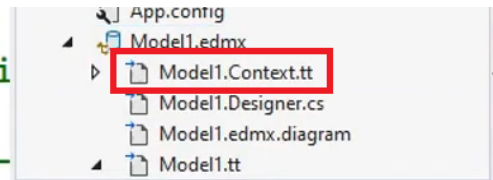


1. حلقة الوصل (Gateway): هو اللي بيدبر التعامل بين الكود بتاعك والداتابيز. أي عملية (Select, Insert, etc). لازم تمر من خلاله.

2. إدارة الاتصال (Connection): يحدد مكان الداتا بيز من الـ Connection String الموجودة في ملف App.config

شكل الكلاس Generated:

this file may cause unexpected behavior in this file will be overwritten if the code i



```
1 public partial class CompanyEntities : DbContext
2 {
3     // DbContext ده كونسرتكتور هيمرر اسم الكونيكشن استرينج لى كلاس
4     // App.config علشان يقرأ الكونيكشن استرينج من
5     // ويتصل مع الداتا بيز
6     public CompanyEntities():base("name=CompanyEntities")
7     {
8
9     }
10
11     // اللي بتمثل الجداول Properties هنا الـ
12     public virtual DbSet<Department> Departments { get; set; }
13     public virtual DbSet<Employee> Employees { get; set; }
14 }
```

شكل الـ Connection string داخل الـ App Config

```
<connectionStrings>
  <add name="CompanyEntities" connectionString="metadata=res://*/Model1.cs
</connectionStrings>
```

يعني إيه DbSet ؟

```
// that there is always a one-to-one correlation between a type and a set.
public class DbSet<TEntity> : DbQuery<TEntity>, IDbSet<TEntity>, IQueryable<TEntity>, IEnumerable<TEntity>, IQueryable<TEntity>
```

- الـ DbSet<T> هو كلاس بيمثل الجدول جوه الكود.
- هو Generic Collection تقدر تعمل عليه Loop.
- بيورث من DbQuery وبيعمل Implement لـ IEnumerable و IQueryable (ودي نقطة جوهرية هنشرحها في الجزء الثاني).

6. أول تعامل مع الكود (Retrieving Data)

عشان نجيب داتا، لازم ناخد Object من الـ Context ونستخدم الـ DbSet .

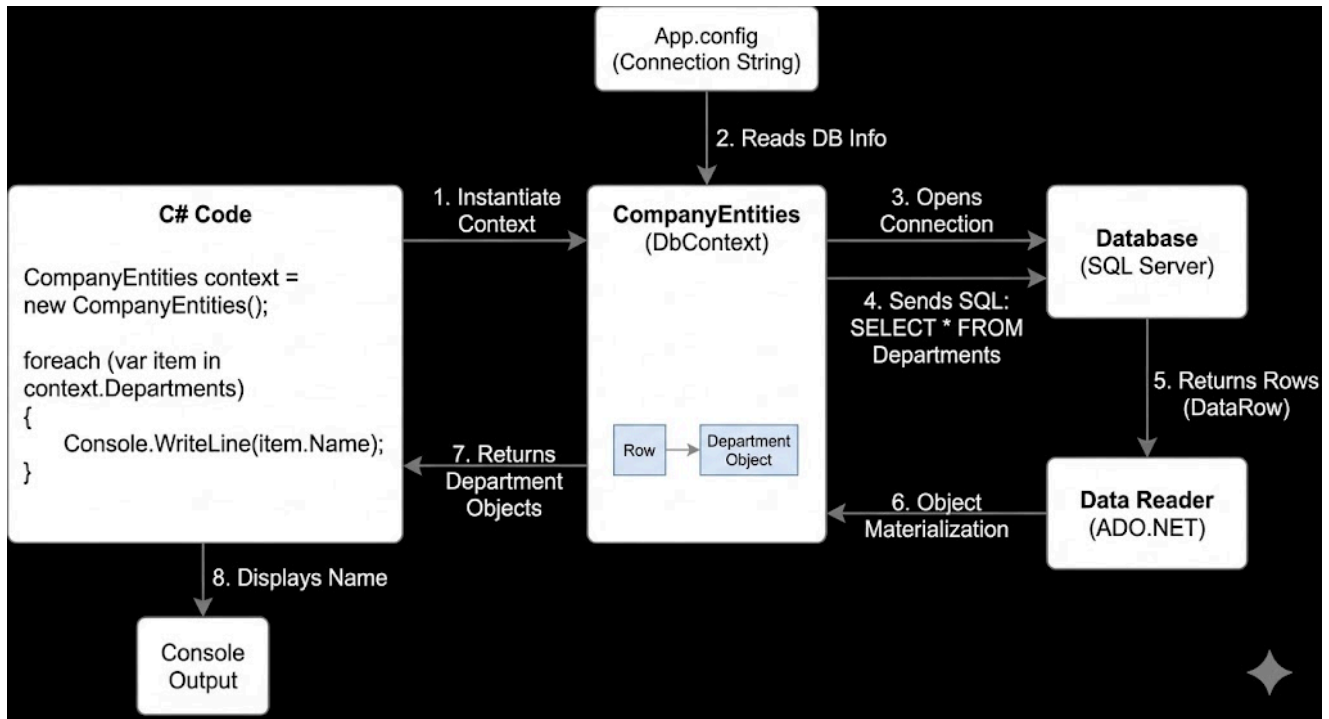
الكود اللي كتبناه:

```

1 CompanyEntities context = new CompanyEntities();
2
3 foreach (var item in context.Departments)
4 {
5     Console.WriteLine(item.Name);
6 }

```

ايه اللي حصل في الخلفية؟ (Detailed Execution Flow):



1. عملنا Instance من CompanyEntities .
2. الـ Context بص في App.config وعرف هيروح لأي داتابيز.
3. لما بدأنا الـ foreach ، الـ Context فتح Connection مع الداتابيز.
4. بعثت جملة SELECT * FROM Departments للداتابيز.
5. الداتا رجعت في شكل DataRow (زي زمان).
6. الجديد هنا: الـ EF حول الداتا دي لـ Objects من نوع Department ورجعها للكود.
7. الكود اشتغل وعرض الأسماء.

سؤال في المحاضرة: هو ده Connected ولا Disconnected Mode؟

الإجابة: ده يعتبر **Connected Mode**، لأن طول ما إحنا بنعمل Loop، الداتا بتيجي من الـ (Data Reader) واحدة بواحدة.

****نهاية الجزء الأول** إحنا جهزنا البيئة، فهمنا يعني إيه Context و DbSet، وشوفنا أول جملة Select بسيطة.

الجزء الثاني (وده الأهم والأخطر):

- إزاي الـ EF بيترجم كود الـ C# لـ SQL؟ (قصة الـ IQueryable والـ Expression Trees).

- الفرق القاتل بين IEnumerable و IQueryable .
- إزاي نشوف شكل الـ SQL اللي رايح للداتا بيز بعيننا (Profiling).
- مشاكل الأداء (Performance Pitfalls) وإزاي نتجنب "الكوارث" اللي ممكن تقفل شركات.

EF behavior with IEnumerable & IQueryable

1. لغز الـ Where وكيفية عمل الـ Query

لما بنكتب كود زي ده:

```
1 var result = context.Departments.Where(d => d.ID > 10);
```

السؤال المنطقي هنا: هل الـ EF بيروح يجيب كل الداتا من الداتابيز ويحطها في الميموري وبعدين يفلتر؟. الإجابة: طبعاً لا، وده بفضل حاجة اسمها **IQueryable**.

الفرق بين IEnumerable VS IQueryable

- الـ **IEnumerable** : دي بناعة الـ LINQ to Objects. لو استخدمتها، الفلتر بتحصل في الميموري (Client-side filtering).
- الـ **IQueryable** : دي المصممة للـ Data Providers زي الـ EF. هي بتورث من **IEnumerable** بس بتزود عليها خاصية إنها بتشيل "وصف الكويري" مش الداتا نفسها، والفلتر بتحصل في الـ Data Base.

2. شجرة التعبيرات Expression Trees

دي "الخطة السرية" للـ Entity Framework.

- لما بنكتب `Where(d => d.ID > 10)`، الـ EF مش بيعتبر الكلام ده كود هيتنفذ حالياً. هو بياخد اللامدا اكسبريشن `(d => d.ID > 10)` ويحولها لـ **Expression Tree**.
- الـ **Expression Tree** عبارة عن Object بيوصف العملية الحسابية (عندي معامل "أكبر من"، وعندي بروبتي اسمها "ID"، وعندي قيمة "10").
- الـ **Provider** (زي SQL Provider) بياخد الشجرة دي وبتترجمها لجملة SQL يفهمها السيرفر: `SELECT ... WHERE ID > 10`.

3. الـ Deferred Execution (التنفيذ المؤجل)

الكويري مش بيروح للداتابيز بمجرد ما تكتبه. الكويري بيفضل "نايم" لحد ما تطلب الداتا فعلياً عن طريق:

- عمل `foreach` على النتيجة.
- أو استخدام دالة بتحول لـ `List` زي `.ToList()` أو `.ToArray()`.

خلي بالك لو انت عملت **ToList** مثلاً في اللحظة دي مش هتبقى **IQueryable** وهتتحول **IEnumerable**

```
var query = context.Departments.ToList().Where(d => d.ID > 10);
```

```
foreach (var dept in query)
{
    Console.WriteLine(dept.Name);
}
```

زى ما انت شايف Where اتحولت لبتاعة IEnumerable

```
public static IEnumerable<TSource> Where<TSource>(this IEnumerable<TSource> source, Func<TSource, bool> predicate)
```

4. إزاي نشوف الـ SQL اللي بيتنفذ؟ (Logging)

عشان نضمن إن الكود بتاعنا بيتحول لـ SQL صح، بنستخدم خاصية الـ Log جوه الـ Context.

كود الـ Logging:

C#

```
1 context.Database.Log = log => Console.WriteLine(log);
2 // أو نطلعه في الـ Debug window
3 context.Database.Log = log => System.Diagnostics.Debug.WriteLine(log);
```

تجربة عملية من المحاضرة:

لما عملنا Where على IQueryable وشفنا الـ Log، لقينا جملة الـ SQL طالعة بـ WHERE clause واضحة وصريحة. ده معناه إن الفلترة حصلت عند السيرفر (Server-side)، وده التمام.

5. محاذير وأخطاء "تضربك بالنار"

المهندس شدد جداً على نقطتين ممكن يدمروا الـ Performance:

أ. استخدام دالات #C غير مدعومة في الـ SQL

لو حاولت تعمل int.Parse أو أي دالة خاصة بالـ #C جوه الكويري، الـ EF هيرمي Exception.

- ليه؟ لأن الـ SQL Provider مش هيعرف يترجم int.Parse لكود SQL مقابل ليها.
- الحل: اعمل الـ Parsing بره الكويري وخرنه في Variable، وبعدين استخدم الـ Variable جوه الكويري.

مثال غلط:

```
1 string idStr = "10";
2 var res = context.Deps.Where(d => d.ID > int.Parse(idStr)); // Runtime Error!
```

ب. الكارثة الكبرى: ()ToList في المكان الغلط

لو عملت ()ToList قبل الـ Where أو الـ Select :

```
1 var res = context.Departments.ToList().Where(d => d.ID > 10);
```

- إيه اللي حصل هنا؟ أنت قولت للـ EF: "روح هات كل الأقسام اللي في الداتا بيز وحطها في الميموري الأول".
- بعد ما الداتا جت الميموري (وبقت IEnumerable)، بدأت تفلتر.
- لو عندك مليون موظف، أنت كده سحببت مليون سجل عبر الشبكة عشان محتاج 5 بس! ده "جريمة" في حق البرفورمانس.

6. طرق كتابة الكويري (Syntax)

عندنا طريقتين لكتابة الكود والاثنتين بيتحولوا لنفس الـ SQL:

1. الـ Method Syntax (Fluent API):

```
1 context.Departments.Where(d => d.ID > 10).Select(d => d);
```

2. الـ Query Expression (SQL-like):

```
1 var query = from d in context.Departments
2             where d.ID > 10
3             select d;
```

ملحوظة مهمة: إحنا بنعمل كويري على الـ **Models** (الـ **Classes**) مش على جداول الداتا بيز مباشرة. لو غيرت اسم الـ Property في الكلاس، هتغيرها في الكويري والـ EF هيتصرف في المابينج.

ملخص الأسئلة اللي اتسألت في الجزء ده:

- س: لو الميموري واحدة والجهاز واحد، هيفرق البرفورمانس؟
- ج: حالياً ممكن متفرقتش، بس العادي إن السيرفر في مكان والكلابنت في مكان، بإرسال داتا مالهاش لازمة بيستهلك Memory و Network بدون داعي.
- س: نستخدم ()ToList إمتى؟
- ج: لما تتأكد إن دي آخر مرحلة وأنت فعلاً محتاج الداتا دي كلها في الميموري عشان تعرضها أو تبعثها لليوزر.

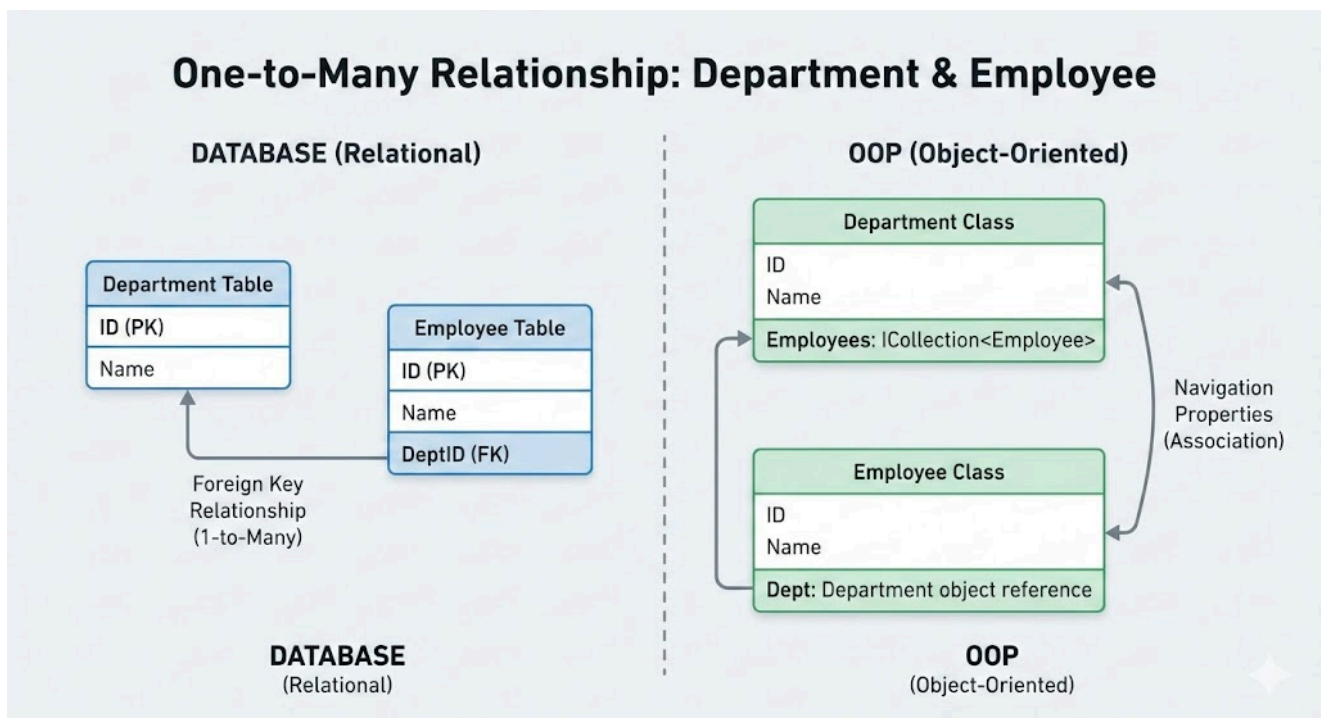
Relationships & Navigation Properties

1. مقدمة عن الكويريز (Queries)

إحنا خلاص مش محتاجين نقول حاجة زيادة في الـ Queries الأساسية. كل اللي عملناه قبل كده في الـ LINQ to Objects (زي GroupBy , Join , OrderBy) هتقدر تعمله هو هنا مع الـ Entity Framework بنفس الطريقة. دلوقتي هنركز على الحاجات الجديدة اللي الـ EF بيقدّمها عشان يسهل علينا التعامل مع العلاقات (Relationships).

2. تمثيل العلاقات (Modeling Relationships)

إزاي بنعبر عن العلاقة بين الجداول في الكلاسات؟



لو عندنا جدولين: Department و Employee ، وفيه علاقة (One-to-Many) بينهم.

- في الـ Database: بنربط بـ Foreign Key.
- في الـ OOP :
- جوه كلاس Department : نخط Collection من الموظفين (عشان القسم بيحتوي على موظفين).
- جوه كلاس Employee : نخط Object من القسم (عشان الموظف بيشتغل في قسم معين).

```
1 using System.Collections.Generic; // عشان نقدر نستخدم ICollection
2
3 // 1. كلاس القسم (Department)
4 public class Department
5 {
6     public int Id { get; set; } // Primary Key
7     public string Name { get; set; }
8 }
```

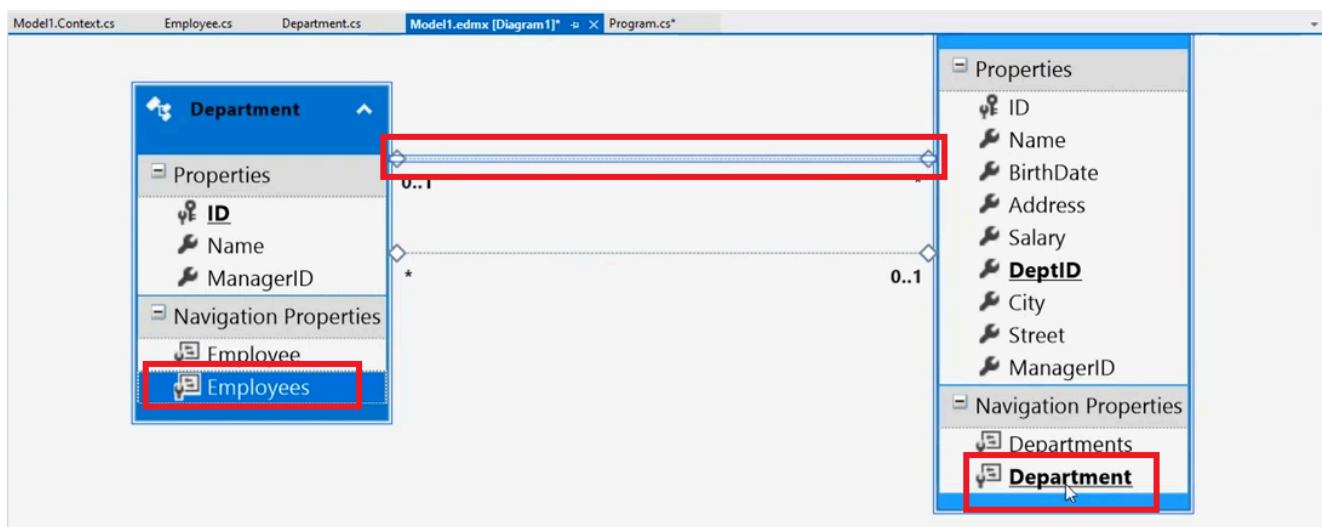
```

9      // =====
10     // Navigation Property (One-to-Many)
11     // =====
12     // عشان نعبّر إن القسم ده "بيحتوي على" مجموعة موظفين Collection حطينا هنا
13     // زي ما اتشرح Lazy Loading عشان نفعل الـ virtual كلمة
14     public virtual ICollection<Employee> Employees { get; set; }
15 }
16
17 // 2. كلاس الموظف (Employee)
18 public class Employee
19 {
20     public int Id { get; set; } // Primary Key
21     public string Name { get; set; }
22     public double Salary { get; set; }
23
24     // Foreign Key (موجود عشان الداتا بيز، والمهندس بيفضل وجوده)
25     public int DepartmentId { get; set; }
26
27     // =====
28     // Navigation Property (Many-to-One)
29     // =====
30     // واحد بس عشان نعبّر إن الموظف "بيشتغل في" قسم واحد Object حطينا هنا
31     // Lazy Loading عشان الـ virtual و بروضه
32     public virtual Department Department { get; set; }
33 }

```

دور الـ Entity Framework (Navigation Properties)

الـ EF بيعملنا الكلام ده أوتوماتيك عن طريق حاجة اسمها **Navigation Properties**. دي عبارة عن Properties بتربط الكلاسات ببعضها.



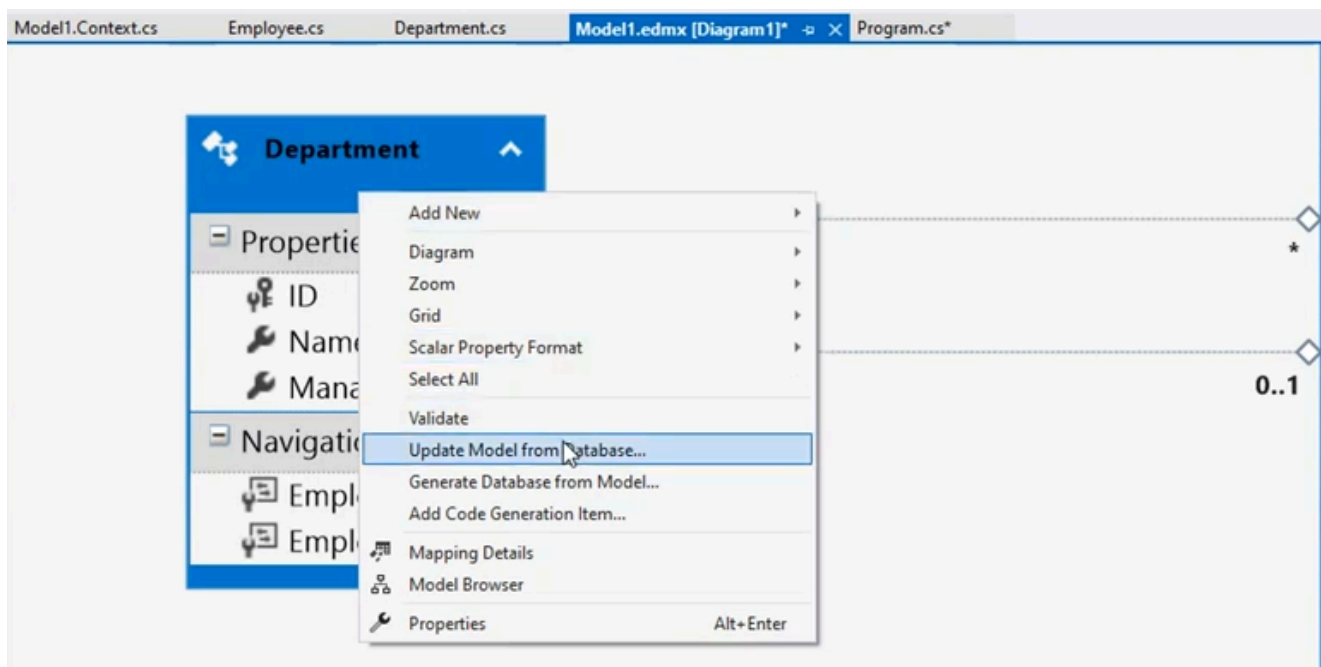
1. داخل **Department** : هيحط **<ICollection<Employee>** وسماها **Employees** (جمع) عشان ده اللي بيشتغلوا فيه.

2. داخل **Employee** : هيحط **Department object** (مفرد) عشان يشاور على القسم اللي هو فيه.

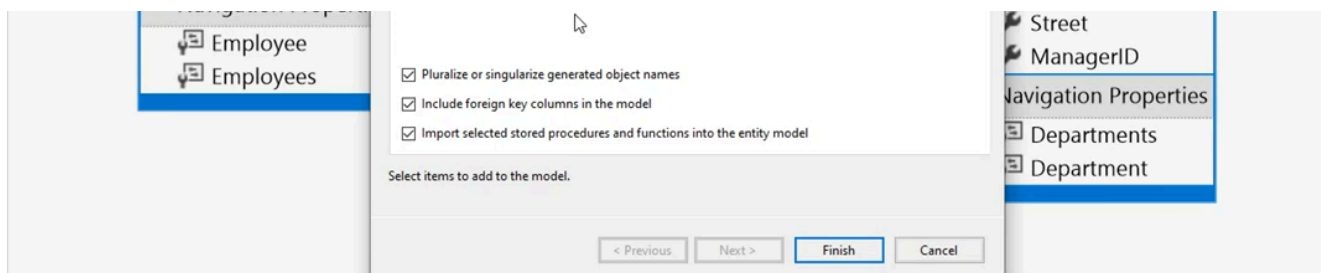
ملاحظة في الـ **Designer**: لما بتقف على الرابط (Association) في الـ **Designer**، بيعلمك على الـ **Navigation** Property في الناحيتين، وده بيفيدنا إن لو معاينا Object موظف أقدر أوصل للقسم بتاعه، ولو معاينا قسم أقدر أوصل للموظفين اللي فيه.

3. إعدادات الـ Model (Foreign Keys vs Nav Props)

واحنا بنعمل Update Model from Database بيظهرلنا شاشة فيها خيارات مهمة:



أ. Pluralize or singularize generated object names



- الخيار ده لو معمول عليه Check، الـ EF بيّفهم الجمع والمفرد.
- عشان كده لما حط Collection من الموظفين جوه القسم سماها **Employees** (جمع) مش **Employee**.

ب. Include foreign key columns in the model

- لو شلنا الـ **Check**: الـ EF هيمسح الـ Foreign Key (زي DeptID) من كلاس **Employee** وهيعتمد كلياً على الـ **Navigation Property** (Department object).
- في الحالة دي، عشان توصل للـ ID لازم تروح للـ Object الأول: **emp.Department.ID**.
- لو سبنا الـ **Check** (وده الأفضل): هيبقى عندك الاتنين، الـ DeptID كـ Property عادية، والـ Department كـ **Navigation Property**.
- المهندس بيفضل وجود الـ FK وهنعرف ليه قدام.

القاعدة: الأصل في الربط عندنا في الـ OOP هو الـ **Navigation Properties**، ويمكن نستغنى عن الـ Foreign Keys تماماً في الكلاسات، لكن الـ FK موجود في الداتابيز عشان الـ Structure.

4. الاستعلام باستخدام (Navigation Properties (Implicit Joins

أجمل حاجة في الـ Navigation Properties إنها بتغنيانا عن كتابة Join صريحة في أغلب الأوقات.

مثال 1: الفلترة باستخدام الـ ID

عايزين نجيب الموظفين اللي في قسم معين (عن طريق الـ Nav Prop):

```
1 var result = context.Employees
2     .Where(e => e.Department.DeptID == 3) // Accessing Dept via Nav Prop
3     .ToList();
4
5 // you can use also e.DepartmentID from employee directly
6 // using Foreign key
7 var result = context.Employees
8     .Where(e => e.DepartmentID == 3)
9     .ToList();
```

الـ **SQL** اللي هيتنفذ: الـ EF ذكي، لو الفلترة على الـ ID وهو موجود بالفعل في جدول الموظفين (FK)، مش هيتحتاج يعمل Join في الحالة دي لو الـ Mapping مضبوط، لكن لو احتاج بيانات من الجدول الثاني هيعمل Join.

مثال 2: الفلترة باستخدام اسم القسم (Cross-Table filtering)

عايزين كل الناس اللي شغالين في القسم اللي اسمه "SD".

```
1 var result = context.Employees
2     .Where(e => e.Department.Name == "SD") // Filtering on a related
3     .ToList();
```

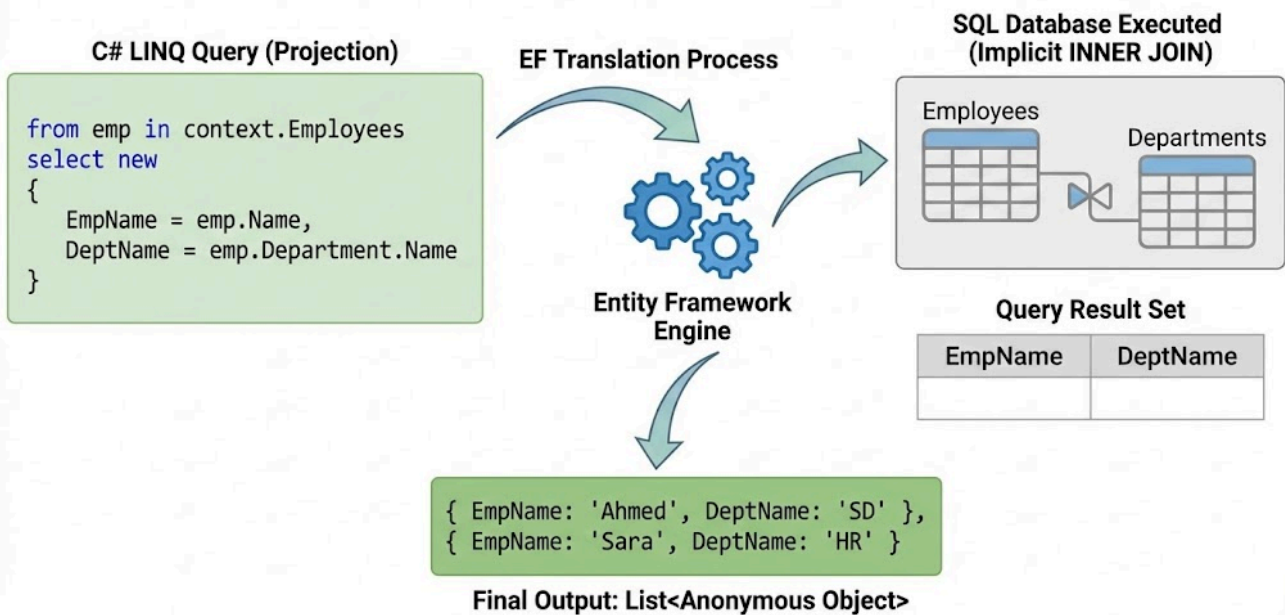
الـ **SQL** اللي هيتنفذ: هنا الـ EF هيعمل Inner Join أوتوماتيك بين Employees و Department ويرجع الداتا المطلوبة.

مثال 3: الـ Projections (Select w/ Join)

عايزين نعرض اسم الموظف واسم القسم بتاعه من غير ما نكتب Join بإيدنا.

```
1 var q = (from emp in context.Employees
2         select new
3         {
4             EmpName = emp.Name,
5             DeptName = emp.Department.Name
```

Implicit Join Projection in Entity Framework



الميزة هنا: إحنا بنعمل Join وبنطلع داتا من الجدولين من غير ما نستخدم كلمة Join خالص، بفضل الـ Navigation Properties، خلى بالك الكويرى الى هتروح الـ Database هيكون معمول فيها JOIN.

5. الـ Lazy Loading (التحميل الكسول)

دي نقطة جوهرية في EF 6 (ومختلفة في Core).

السيناريو

لو عملت كويرى عشان تجيب Department معين:

```
1 var dept = context.Departments.First();
```

السؤال هنا: هل الـ Context هيروح يجيب بيانات القسم وكمنا يجيب لسته الموظفين (Employees) اللي جواه في نفس الوقت؟

- لو جابهم كلهم: ممكن تبقى كارثة في الـ Performance لو أنا مش محتاج الموظفين دلوقتي (بجيب داتا زيادة).
- لو ماجابهمش: الـ Collection هتبقى فاضية، ولو جيت أستخدمها بعدين مش هلاقي داتا.

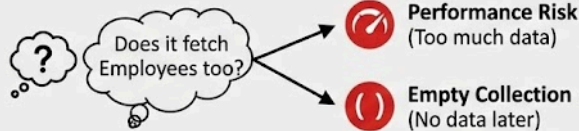
حل الـ Lazy Loading EF 6:

الـ EF مسك العصاية من النص:

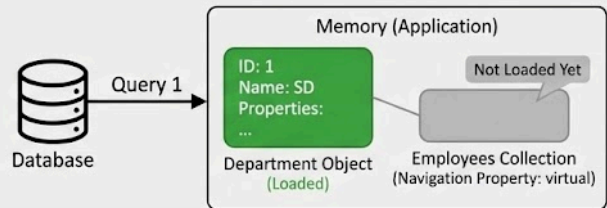
EF 6 Lazy Loading Explained

The Scenario: Initial Query

```
var dept = context.Departments.First();
```



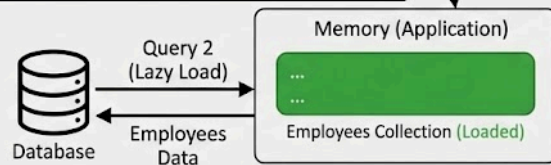
EF 6 Solution: Lazy Loading (Initial State)



The Trigger: Accessing Property

```
foreach (var emp in dept.Employees) {  
    ...  
}
```

Trigger!



Conditions & Warnings



Must be
`public virtual ICollection<Employee>
Employees { get; set; }`



Performance Impact:
Can cause N+1 Query Problem
(Many small queries)

ⓘ Different in EF Core

1. لما بتعمل Load لل Object (القسم)، الـ Navigation Property بتاعته (Employees) بتيجي فاضية في الأول.
2. أول ما كودك بيدأ يستخدم أو يعمل Loop على الـ Employees ، الـ EF في اللحظة دي هيروح بيعت كويري ثاني للداتا بيز يجيب الموظفين ويملا الـ Collection.

```
1 var dept = context.Departments.First();  
2  
3 //here it will generate sql and send it to the data base  
4 foreach (var emp in dept.Employees)  
5 {  
6  
7 }
```

تعريف Lazy Loading: هو إن لما الـ Object يحصله Load من الداتا بيز، مش بنعمل Load للـ Navigation Properties معاه. الـ Loading بيحصل لما نبدأ نستخدم الـ Property دي.

شروط تفعيل Lazy Loading

عشان الـ Lazy Loading يشتغل، لازم الـ Navigation Property تكون متعرفة بكلمة **virtual**.

```
1 public virtual ICollection<Employee> Employees { get; set; }
```

لو شلت **virtual** ، الـ Lazy Loading مش هيشغل والـ Collection هتيجي null أو فاضية ومش هتتملي لوحدها.

تحذير: الكلام ده ليه تأثير كبير على الـ Performance (ممكن يسبب مشاكل زي N+1 query problem)، وعشان كده في **EF Core** الموضوع اتغير وبقي فيه طرق ثانية (Eager, Explicit, Lazy) عشان نتفادى العيوب دي، بس ده موضوع لمحاضرات قادمة.

6. أسئلة ومفاهيم هامة

س: إيه الفرق بين Lazy Loading و Deferred Execution؟

- الـ **Deferred Execution**: معناه إن الكويري نفسه (التعريف بتاعه) مش بيروح الداتابيز بتنفذ غير لما تعمل عليه Enumeration (زي `ForEach` , `ToList`). سواء الكويري ده هيجيب الداتا كلها مرة واحدة أو حطة حطة.
- الـ **Lazy Loading**: ده خاص بالـ **Related Data** (البيانات المرتبطة). يعني أنا جيت القسم خلاص (الكويري اتنفذ)، بس بيانات الموظفين لسه ماجاتش. هتيجي لما أطلبها. ده مالوش علاقة الكويري الأصلي بتنفذ إمتى.

س: إمتى الـ Collection تتملي؟

تتملي فقط لما تعمل Access عليها (تكتب كود بيناديهها). لو طول عمر البرنامج معملتش Loop على `dept.Employees` ، الـ EF مش هيروح يجيبهم أبداً.

7. البحث باستخدام Find().

بدل ما نكتب `FirstOrDefault(x => x.ID == 3).Where()` ، عندنا ميثود سريعة ومختصرة اسمها `Find`.

```
1 var dept = context.Departments.Find(3);
```

- بتعمل إيه؟ بتاخذ الـ Primary Key (الـ ID).
- الميزة: الـ Context عارف مين هو الـ PK، فيروح يدور عليه علطول.
- الأهم: الـ `Find` بتدور الأول في الميموري (Local Cache)، لو لقت الـ Object مش بتروح الداتابيز (عكس Single أو First اللي غالباً بيروحوا)، بس النقطة دي المهندس مذكرهاش بالتفصيل هنا بس أشار إنها بتاخذ object وبتقارن بالـ ID.

تنويه: الفرق الدقيق بين `Find` و `SingleOrDefault` هيتشرح بالتفصيل لاحقاً، بس مبدئياً `Find` شغالة على الـ PK.

CRUD & Change Tracking

1. عملية التعديل (Update) والـ Change Tracker

في البداية، تعالوا نشوف إزاي نعمل **Update** لأوبجكت عندنا.

مثال عملي:

لو أنا عايز أحيب أول قسم (Department) وأغير اسمه:

C#

```
1 // 1. Load object from DB
2 var dept = context.Departments.First();
3
4 // 2. Modify property in memory
5 dept.Name = "Intake 42";
6
7 // 3. Save Changes
8 context.SaveChanges();
```

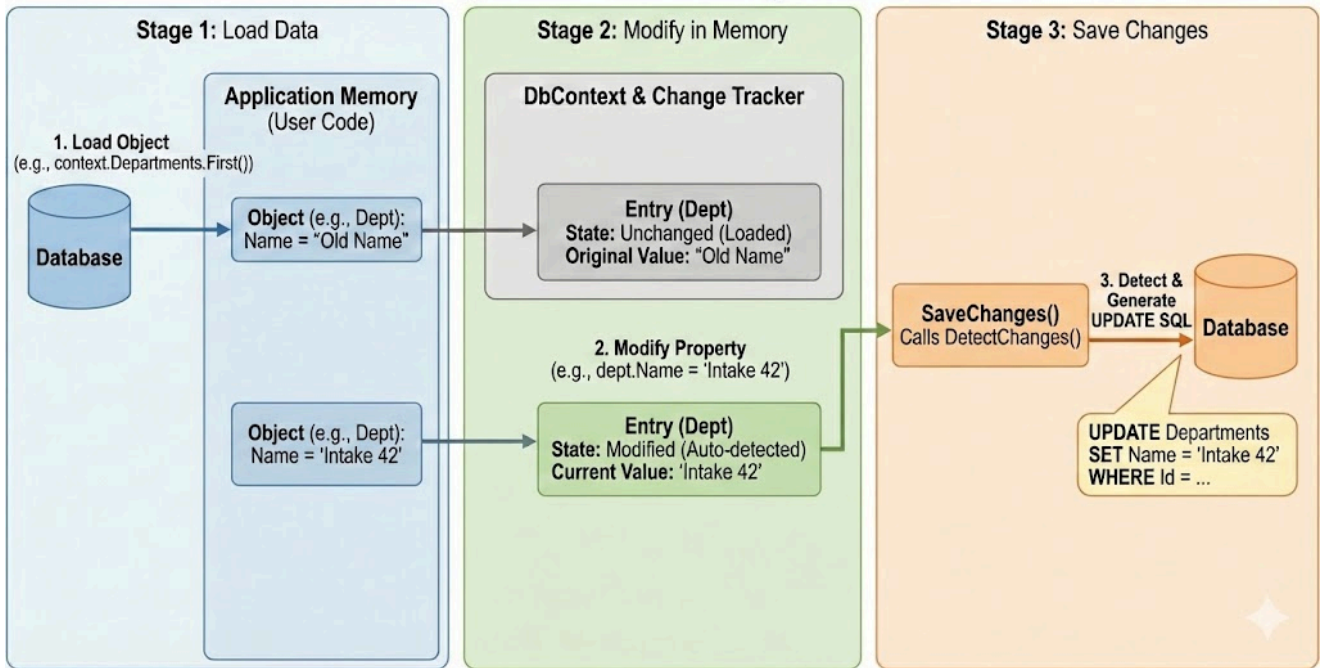
- **الملاحظة:** بمجرد ما غيرت الاسم في الكود، التغيير ده حصل في الـ **Memory** بس. الداتا بيز لسه ماتعرفش حاجة.
- لما ندهنا `context.SaveChanges()` ، التغيير سمع في الداتا بيز.

إزاي **SaveChanges** عرفت إن فيه تغيير حصل؟ (The Change Tracker)

عشان نفهم ده، لازم نعرف حاجة مهمة جداً اسمها **Change Tracker**.

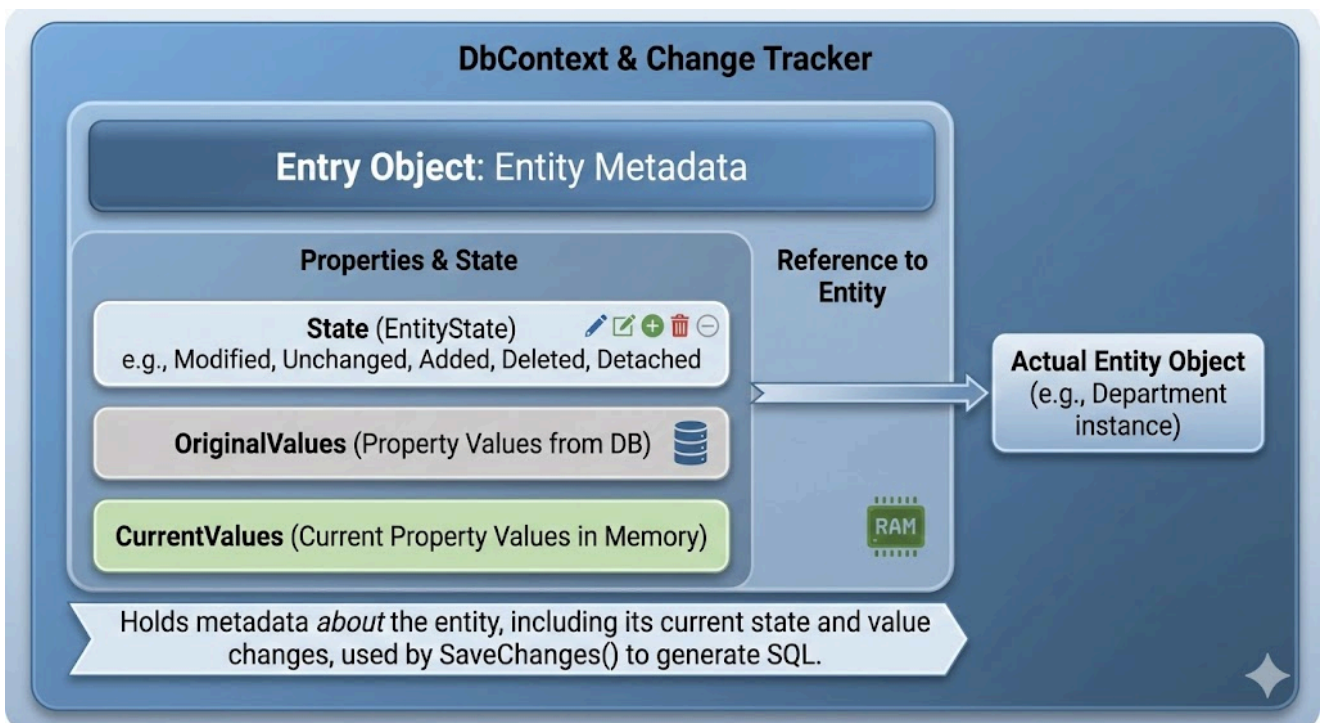
- **وظيفته:** بيقرب (Tracks) أي تغيير بيحصل على الأوبجكتس اللي الكونتكتس جابها من الداتا بيز.
- **إزاي بيشتغل؟** أي أوبجكت بيجي عن طريق الكونتكتس، الكونتكتس بيحتفظ بـ **Reference** له وبيقربه.
- الكونتكتس عنده كلاس مخصوص اسمه **Entry**. لكل أوبجكت (زي Department)، بيتعمل قصاده أوبجكت من نوع **Entry** بيشيل معلومات عنه، زي الـ **State** بتاعته.

Entity Framework Change Tracker



حالة الأوبجكت (Entity State):

ال Entry ده جواه Property مهمة اسمها State . دي اللي بتحدد وضع الأوبجكت حالياً:



- لما بتغير قيمة في الأوبجكت (زي `dept.Name = ...`)، ال State بتتحول أوتوماتيك لـ **Modified**.
- لما بتدعه **SaveChanges**، هي بتعمل **DetectChanges** وتلف على كل ال Entries اللي حالتها **Modified** وتروح تعمل جملة **Update** في الداتا بيز.

❓ سؤال طالب: ليه ما ينفعش أعدل في الداتا بيز علطول؟ ليه لازم أعدل في الأپليكشن الأول؟ رد المهندس: إنت كـ **End User** (مستخدم) من حقك تعدل بياناتك (زي پوست الفيسبوك). إحنا هنا بنتكلم عن تعديل **Data** مش **Structure**. الأپليكشن هو اللي بيدي الصلاحية دي، والـ **Context** هو الوسيط اللي بينقل التعديلات دي للداتا بيز.

2. التحكم في الـ Tracking و Performance Tuning

بما إن عملية الـ Tracking تستهلك Resources (لأن كل أوبجكت بيتعمل له Entry قصاده)، ساعات بنحتاج نوقفها أو نتحكم فيها.

2.1 إلغاء الـ Tracking (القراءة فقط)

لو أنا بجيب داتا كتير عشان أعرضها بس (Read-only)، ومش ناوي أعدل عليها، استخدم `AsNoTracking`.

```
1 var depts = context.Departments.AsNoTracking().Where(d => d.Id > 1);
```

وفيه طريقة ثانية عن طريق الـ **State** (بتستخدم أكثر لو معاك **1 object**)

```
1
2 var dept = context.Departments.First();
3 context.Entry(dept).State = System.Data.Entity.EntityState.Detached;
4 //Detached معناها انا بقوله متعملش تراك ليها ملكش دعوة بيها يعني
5
6 // if you want to make reattach as it was
7 context.Departments.Attach(dept);
8 // and the state will be (unmodified)
```

- **الفائدة:** الكونتكست مش هيعمل Track للأوبجكتس دي، وده بيحسن الـ Performance جداً.
- **النتيجة:** لو عدلت في الداتا دي بعد كده وعملت `SaveChanges`، مش هيحصل أي حاجة.

2.2 التحكم اليدوي في الـ State

تقدر تغير حالة الأوبجكت بإيدك (مع إن ده مش مفضل إلا في حالات خاصة):

```
1 context.Entry(dept).State = System.Data.Entity.EntityState.Unchanged;
```

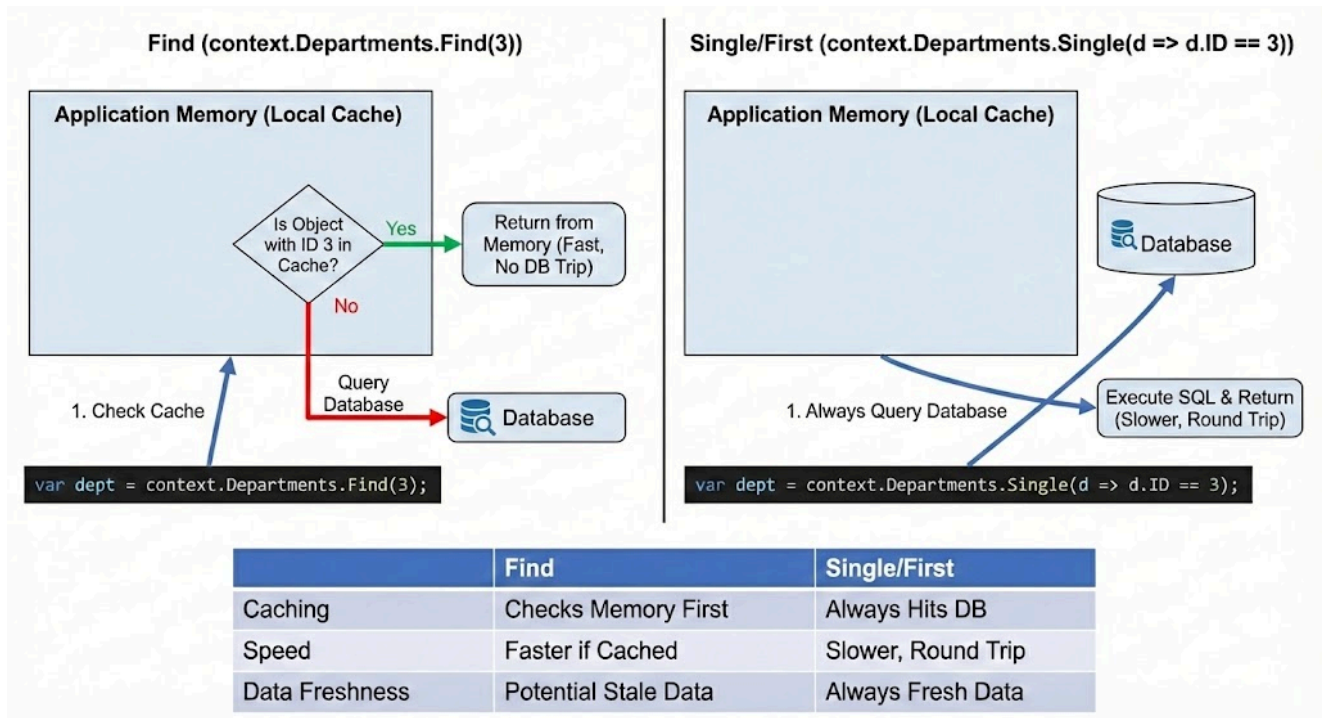
- لو خليتها `Unchanged` بعد ما عدلت البيانات، `SaveChanges` مش هتعمل حاجة لأنها شايفة إن مفيش تغيير.
- لو خليتها `Detached`: كأنك بتقول للكونتكست "شيل إيدك، مالكش دعوة بالأوبجكت ده خالص".
- لو حبيت ترجعه تاني للكونتكست، بتستخدم `Attach`. وساعتها حالته بتبقى `Unchanged` (يعني لازم تعدل الحالة بنفسك لو عايز تعمل أبيت).

3. الفرق بين Find و Single/First

سؤال مهم: إيه الفرق لما أجيب داتا بـ `Find` ولما أجيبها بـ `Single` أو `First` ؟

```
1 var dept = context.Departments.Find(3);
2 var dept = context.Departments.Single(d => d.ID == 3);
```

الميزة	Find	Single / Where
الكاش (Caching)	بتدور الأول في الميموري (Local Cache). لو الأوبجكت موجود، بترجعه من الميموري وما تروحش الداتا بيز.	بتروح تنفذ Query على الداتا بيز علطول حتى لو الأوبجكت موجود معاك.
السرعة	أسرع لو الأوبجكت كان متحمل قبل كده (Cached).	أبطأ لو الداتا معاك لأنها بتعمل Round trip للداتا بيز.
تحديث الداتا	ممكن ترجع داتا قديمة لو حد غير في الداتا بيز وأنت لسه معاك النسخة القديمة في الميموري.	بتجيب دائماً أحدث نسخة من الداتا بيز.



4. عملية الإضافة (Insert)

عشان نضيف ريكورد جديد:

```
1 var dept = new Department { Name = "Int 42" };
2 context.Departments.Add(dept);
3 context.SaveChanges();
4
5 // u can use AddRange to add more than one object
```

1. بنعمل أوبجكت جديد `new Department ()`.

2. بنضيفه للكولكشن `context.Departments.Add(newDept)`.

3. بنعمل `SaveChanges ()`.

في اللحظة دي الـ State بتبقى Added ، فالكونتكتست بيعرف إنه محتاج يعمل INSERT .

إضافة بيانات مرتبطة (Relational Data)

ال Entity Framework ذكي جداً في التعامل مع العلاقات (Primary Key & Foreign Key).

مثال: عايز أضيف قسم جديد (Department) وجواه موظفين (Employees) جدد في نفس الوقت.

```
1 // 1. Create new Department
2 Department newDept = new Department { Name = "HR" };
3
4 // 2. Add Employees to this Department directly
5 newDept.Employees.Add(new Employee { Name = "Ali" });
6 newDept.Employees.Add(new Employee { Name = "Mona" });
7
8 // 3. Add the Department to Context
9 context.Departments.Add(newDept);
10
11 // 4. Save
12 context.SaveChanges();
```

• إيه اللي بيحصل؟

- إنت عملت Add للـ Department بس، لكن هو جواه Employees.
- الكونتكست بيّفهم إن الـ Employees دول كمان جدد (Added).
- بترتيب ذكي: بيروح يعمل Insert للـ Department الأول -> الداتا بيز بتدرد بالـ ID الجديد (Identity) -> ياخذ الـ ID ده ويحطه للـ Employees -> ويعمل Insert للـ Employees.

① ملحوظة: الطريقة العكسية تنفع برضه (تعمل Employee وتديله أوبجكت Department جديد)، هيجصل نفس السيناريو.

```
1 var dept = new Department { Name = "Int 4222222" };
2
3 var emp = new Employee
4 {
5     Name = "A1111",
6     BirthDate = DateTime.Now,
7     Department = dept,
8 };
9
10 context.Employees.Add(emp);
11 context.SaveChanges();
```

إيه اللي هيجصل لما تعمل Run؟

ال Entity Framework هنا "بيطلع محترم بزيادة شويتين" وبيتصرف بذكاء.

1. لما ندهت context.Employees.Add(emp) ، حالة الـ Employee اتحولت لـ Added .

2. لما تعمل `SaveChanges` ، الكونتكست يبص على الـ `Employee` ده، بيلقي جواه خاصية `Department` بتشاور على أوبجكت `dept` جديد (لسه مش موجود في الداتا بيز).
3. الكونتكست بيفهم إن عشان يضيف الـ `Employee` ده، لازم ولايد يضيف الـ `Department` بتاعه الأول عشان ياخد الـ ID بتاعه (FK).
4. ترتيب التنفيذ في الداتا بيز:
 - الكونتكست هيروح الأول يعمل `INSERT` للـ `Department`.
 - السيكيوال سيرفر يرد عليه بالـ ID الجديد للقسم.
 - يرجع الكونتكست ياخد الـ ID ده، ويعمل `INSERT` للـ `Employee` وهو مربوط بالقسم صح.

3. الفرق بين الطريقة دي والطريقة اللي قبلها

- الطريقة اللي قبلها: كنا بنضيف الـ `Department` ، وكان جواه ليستة موظفين، فالكلي بيتضاف.
- الطريقة دي (في الصورة): إحنا ضفنا الـ `Employee` ، وهو صاحب معاه الـ `Department` بتاعه.

ملحوظة مهمة من المحاضرة لو كنت عكست الكود وكتبت `context.Departments.Add(dept)` والـ `dept` ده مكنش جواه ليستة موظفين (الكولكشن فاضية)، كان هضيف القسم بس ومش هضيف الموظف. لكن عشان إنت هنا ضفت الموظف وربطته بالقسم، فالقسم "غصب عنه" اتضاف عشان الموظف يعيش.

الخلاصة

السطر `context.Employees.Add(emp)` في الحالة دي كافي جداً إنه يضيف الموظف وكماني يضيف القسم الجديد المرتبط بيه في نفس الـ `Transaction` بشكل أوتوماتيك.

5. عملية الحذف (Delete) والـ Bulk Operations

عشان تمسح:

1. لازم تجيب الأوبجكت الأول (Load it).
2. تستخدم `Remove`.

```
1 var dept = context.Departments.Find(72);
2 context.Departments.Remove(dept);
3 context.SaveChanges();
```

مشكلة الـ Bulk Operations (تعديل أو حذف كميات كبيرة)

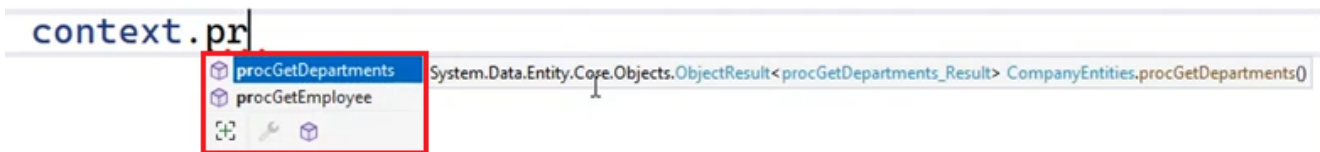
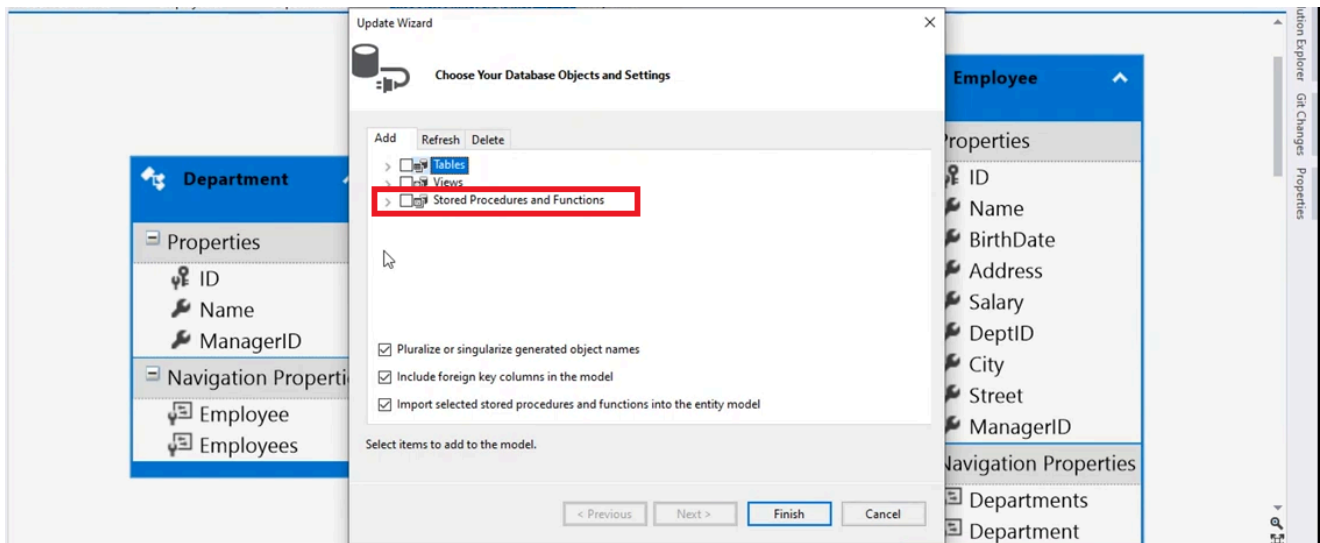
لو عايز تزود مرتبات كل الموظفين 500 جنيه، أو تمسح 1000 موظف، أو عاوز تعمل `Delete` لـ `Object` من غير ما تعمل `Load` فيه وتعمل (`Delete` زي طريقة الـ `ADO` عن طريق `SQL` مباشر).

- في `EF`: لازم تحملهم كلهم في الميموري الأول -> تعدلهم -> تعمل `Save`. وده سيء جداً للـ `Performance`.
- الحل:
- استخدم `Stored Procedures`.

- أو نفذ جملة SQL مباشرة (ExecuteSqlRaw مثلاً) لو بتستخدم EF Core (أو ExecuteStoreCommand في النسخ القديمة).

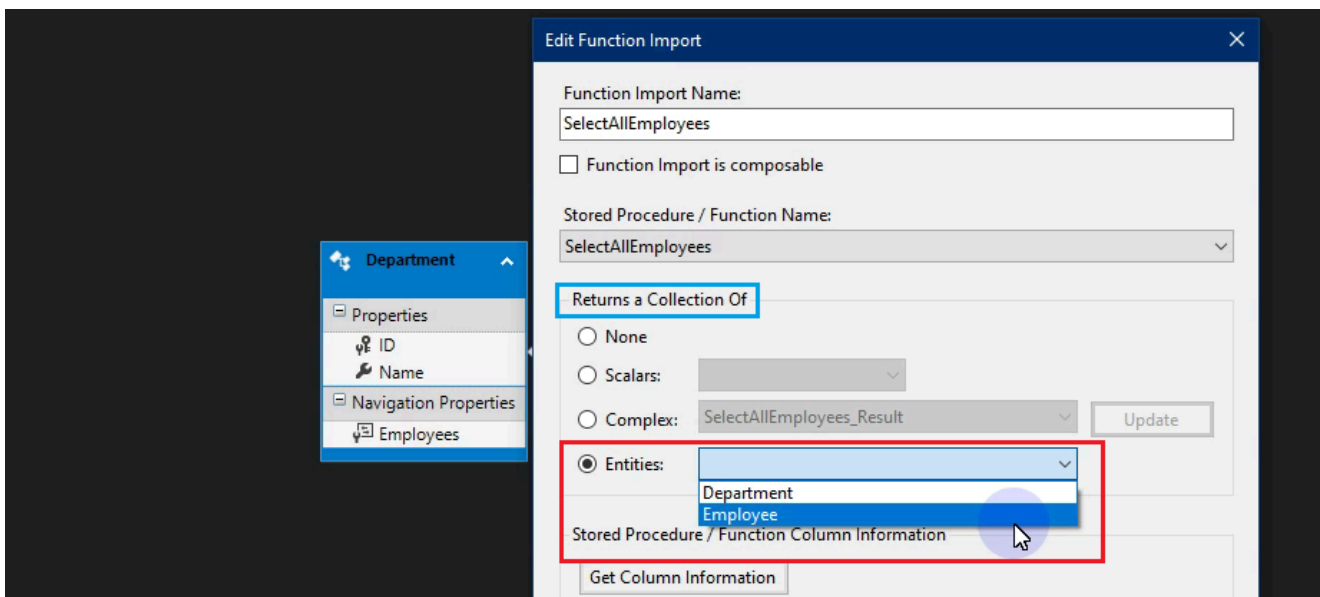
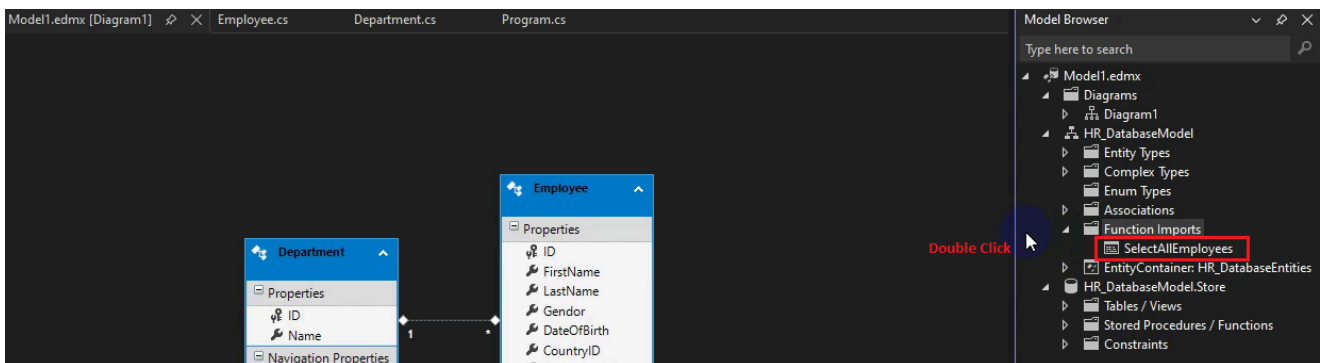
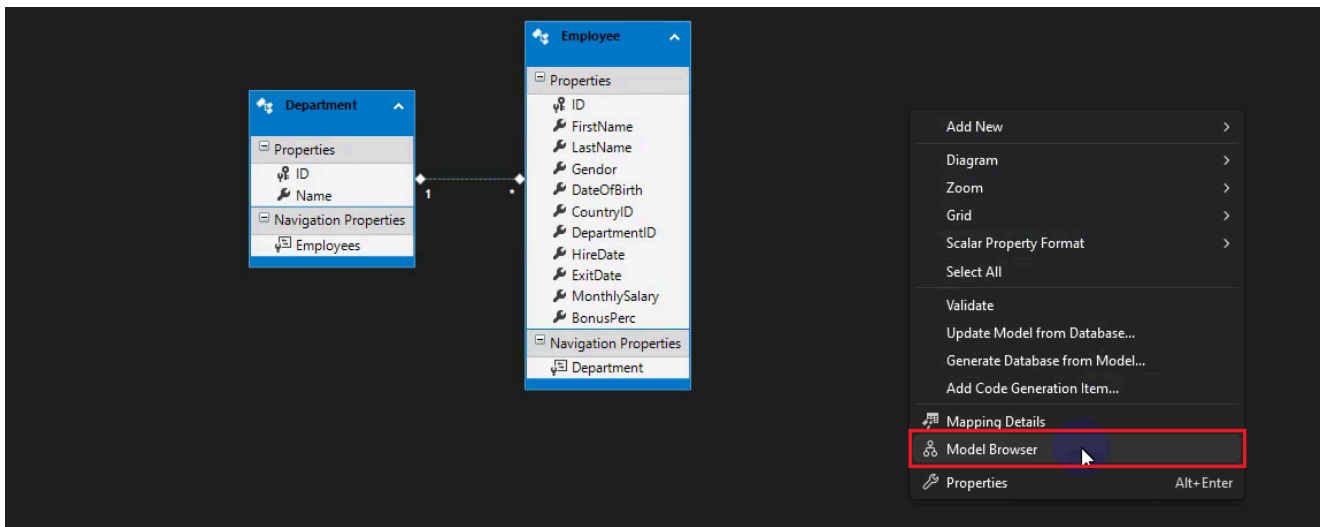
6. استدعاء Stored Procedures

ال Entity Framework بتسهل استدعاء ال Procedures.



- لما بتعمل Import لل Stored Procedure من الداتا بيز، ال EF بيعمل لها Function جوه الكونتكست.
- ال **Return Type**: ال EF بتعمل كلاس جديد (Complex Type) يطابق شكل النتيجة اللي راجعة من ال Procedure (مثلاً GetDept_Result).
- تقدر تعدل ال Mapping عشان ترجع Entity حقيقي (زي Department) لو النتيجة مطابقة.

الطريقة ال تقدر تغير بيها ال **Mapping** الى البشمةهندس قال دور عليها وتخلي البروسيدجر يرجع نوع معين من ال **Entities** بدل ما يعمل كلاس جديد (مثال على بروسيديجر بيرجع كل الموظفين وعاوز نوع الارجاع يكون من Employee :Class)



حفظ وتحديث: اضغط OK، وبعد ذلك احفظ ملف الـ .edmx (Save).

- ملاحظة: يفضل تعمل "Run Custom Tool" أو Rebuild للمشروع عشان الكود يتحدث ويشيل الكلاس القديم.

7. مشكلة الـ Concurrency (تضارب التعديلات)

دي أخطر وأهم جزئية في المحاضرة.

السيناريو (The Lost Update Problem):

تخيل عندنا "شباكين حجز تذاكر" أو "شخصين ببسحبوا فلوس من نفس الحساب" في نفس الوقت.

1. المستخدم (أ): استعلم عن الرصيد، لقاءه 5000.
2. المستخدم (ب): استعلم عن الرصيد (في نفس اللحظة)، لقاءه 5000.
3. (أ) سحب 1000: حسب في جهازه (5000 - 1000 = 4000) وراح عمل Update للرصيد بـ 4000.
4. (ب) سحب 2000: هو لسه فاكّر الرصيد 5000، فحسب (5000 - 2000 = 3000) وراح عمل Update للرصيد بـ 3000.



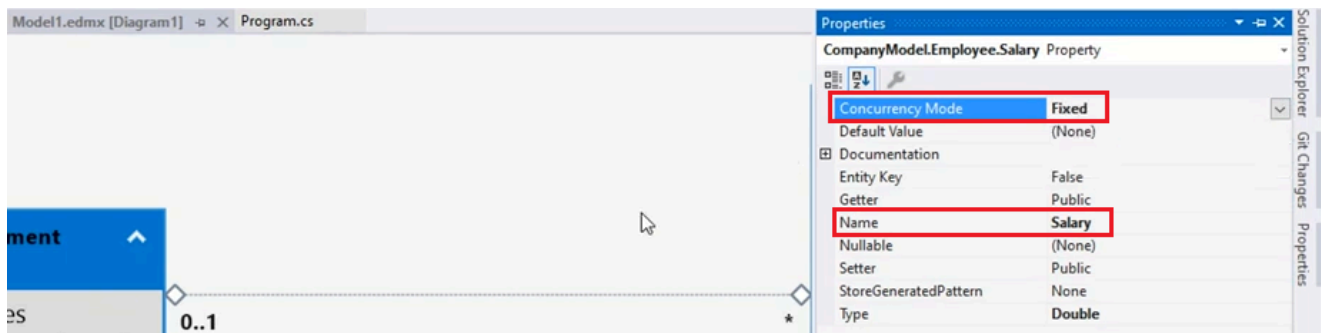
النتيجة: المستخدم (أ) عملته ضاعت، والرصيد بقي 3000 بدل المفروض (2000). ده اسمه **Lost Update**.

الحل: Optimistic Concurrency

الـ Default في EF إن "آخر واحد ببعدل هو اللي كلامه بيمشي" (Last Write Wins). عشان نغير السلوك ده ونحمي الداتا، بنستخدم **Concurrency Check**.

1. تفعيل الـ Concurrency Token

بنيجي على الـ Property المهمة (زي Salary) في الـ Model ونخلي الـ **ConcurrencyMode** بتاعها **Fixed**.



- معنى ده: "يا كونتكست، وإن بتعمل Update، خذ بالك من القيمة القديمة للعمود ده".

2. اللي بيحصل في الخلفية (SQL)

الكونتكست لما يروح يعمل Update، مش هيقول:

```
1 UPDATE Emp SET Salary = 3000 WHERE Id = 1
```

لأ، هيقول:

```
1 UPDATE Emp SET Salary = 3000 WHERE Id = 1 AND Salary = 5000
```

(القيمة القديمة اللي كانت معايا).

- لو حد غير الرصيد في النص، الشرط `Salary = 5000` مش هيتحقق، فالـ Update مش هيعدل أي صف (0 rows affected).
- الـ EF هيفهم إن حصل تضارب ويرمي **Exception** اسمه `DbUpdateConcurrencyException`.

❶ خلى بالك موضوع الـ **Concurrency** مكلف لان كل **object** هيعمل منه نسخه قديمة ونسخة جديدة علشان يقدر يعمل **check** على القيمة القديمة، علشان كده الـ EF الـ **Default** بتاعها ان ميعملش **Concurrency** انت الى بتحدد الـ **Property** الى تستاهل وحساسية ان لازم يكون عليها **Concurrency Fixed**.

3. التعامل مع الـ (Code) Exception

لازم نعمل `try...catch` ونتعامل مع الموقف:

```
1 try
2 {
3     context.SaveChanges();
4 }
5 catch (DbUpdateConcurrencyException ex)
6 {
7     // 1. Get the entry that caused the error
8     var entry = ex.Entries.First().Entity as Employee;
9
10    // 2. Reload the object from Database (Get the new fresh value)
11    context2.Entry(emp).Reload();
12    // Now the object has the value 4000 (from User A's update)
13
14    // 3. Re-apply business logic
15    var emp = (Employee)entry.Entity;
16    emp.Salary = emp.Salary - 2000; // 4000 - 2000 = 2000
17
18    // 4. Try Saving again
19    context2.SaveChanges();
20 }
```

⚠ تحذير هام: حتى الحل ده ممكن يفشل لو فيه مستخدم تالت دخل في النص بين خطوة الـ **Reload** وخطوة الـ **Save** التانية. الحل النهائي: لازم تحط الكلام ده جوه **Loop**، يفضل يحاول لحد ما العملية تنجح (تنجح الـ **Save** بدون **Exception**).

```
1 bool saved = false;
2 while (!saved)
3 {
4     try
5     {
6         context.SaveChanges();
7         saved = true;
8     }
9     catch (DbUpdateConcurrencyException ex)
10    {
11        foreach (var entry in ex.Entries)
12        {
13            entry.Reload(); // Update values from DB
14            // Re-apply logic here if needed
15        }
16    }
17 }
```

علاقة الـ Business Logic بقرار استخدام الـ Loop

المهندس قال إن مش دائماً بنحتاج `while loop`، القرار ده بيعتمد كلياً على الـ **business logic** بتاعك .

السيناريو الأول: حجز التذاكر (Ticket Reservation) - "مش محتاج Loop"

- الـ **business logic**: لو الكرسي رقم 42 اتحجز، ماينفعش أحجزه تاني غصب عن السيستم. الكرسي راح خلاص.
- التصرف في الكود:
- هتعمل `try-catch` عادية.
- لو حصل `Concurrency Exception` (معناه حد سبقك وحجز الكرسي)، مش هتحاول تاني.
- في الـ `catch`، هتقول للمستخدم رسالة زي: "معلى التذكرة دي اتحجزت خلاص، شوف لك واحدة غيرها".
- النتيجة: مفيش `Loop` هنا، المحاولة بتفشل وخلص.

السيناريو الثاني: العمليات المالية (Salary/Bank) - "لازم Loop"

- الـ **business logic**: أنا عايز أخصم 2000 جنيه من المرتب. لو المرتب كان 5000 وحد خلاص 7000 في نفس اللحظة، أنا لسه عايز أخصم الـ 2000 بتوعى من القيمة الجديدة (الـ 7000). العملية لازم تتم وتأخذ أحدث قيمة في الاعتبار.
- التصرف في الكود:
- هنا ماينفعش أقول للمستخدم "معلى مش هخصم"، لازم أجيب القيمة الجديدة وأحاول أخصم تاني .
- ليه بنحتاج **Loop**؟ لأن حتى بعد ما عملت `Reload` وجبت القيمة الجديدة (مثلاً 7000)، ممكن "حظك الوحش" إن قبل ما تعمل `SaveChanges` في المحاولة التانية، يجي "شباك تالت" ويغير القيمة تاني .
- الحل: مالهش حل غير إنك تحط الكود جوه `while loop` وتفضل تلف لحد ما العملية تنجح (تعدي من غير `Exception`).

8. ملاحظات ختامية

- الـ **Transaction**: الـ `SaveChanges` بتنفيذ كل العمليات التي قبلها في **Transaction** واحد. يا كله ينجح، يا كله يفشل (Rollback).
- المستقبل هو **Entity Framework Core**: معظم الشركات شغالة بيه دلوقتي، وهيكون فيه اختلافات بسيطة بس المفاهيم (Concepts) واحدة.

Code First

مقدمة: ليه Code First؟ ولية بنعكس الآلية؟

امبارح اتكلمنا عن **DB First** (يعني كان عندنا داتابيز جاهزة وعملنا Generate للكود منها). النهاردة هنتكلم في العكس، وهو **Code First**.

- الفكرة: إحنا كـ Developers، تفكيرنا الطبيعي بيكون **Object Oriented** (Classes, Objects, Relationships)، بينما الداتابيز تفكيرها **Relational** (Tables, FKs).
- ليه **Code First** هو المستقبل؟
- بدل ما تبدأ بتصميم الداتابيز وتخلي الأبلكيشن يعتمد عليها، إحنا هنبنّي الـ Domain Classes بتاعتنا الأول بغض النظر عن الـ Storage.
- ده بيخليك مرّن؛ النهاردة بتخزن في SQL Server، بكرة ممكن تغير لـ XML أو NoSQL أو Cloud Storage من غير ما تهد الأبلكيشن كله.
- في **EF Core** (الإصدارات الحديثة)، الـ Code First هو الاستاندرّد، وحتى الشركات اللي عندها مشاريع قديمة بتعمل خطط عشان تحول لـ Code First.

الخلاصة: النهاردة هنبنّي الـ Classes والموديلز بايدينا، وهنخلي الـ Entity Framework هو اللي يروح "يخلق" الداتابيز والجداول.

1. تجهيز المشروع (Setup)

هنشتغل على **Console Application (.NET Framework)** زي المرة اللي فاتت عشان إحنا بنشرح EF6.

الخطوة الأولى: بناء الـ Domain Classes

هنعمل 2 Classes بسطاء جداً: **Employee** و **Department**.

C#

```
1 // Employee.cs
2 public class Employee
3 {
4     public int Id { get; set; }
5     public string Name { get; set; }
6     public double Salary { get; set; }
7 }
8
9 // Department.cs
10 public class Department
11 {
12     public int Id { get; set; }
13     public string Name { get; set; }
14 }
```

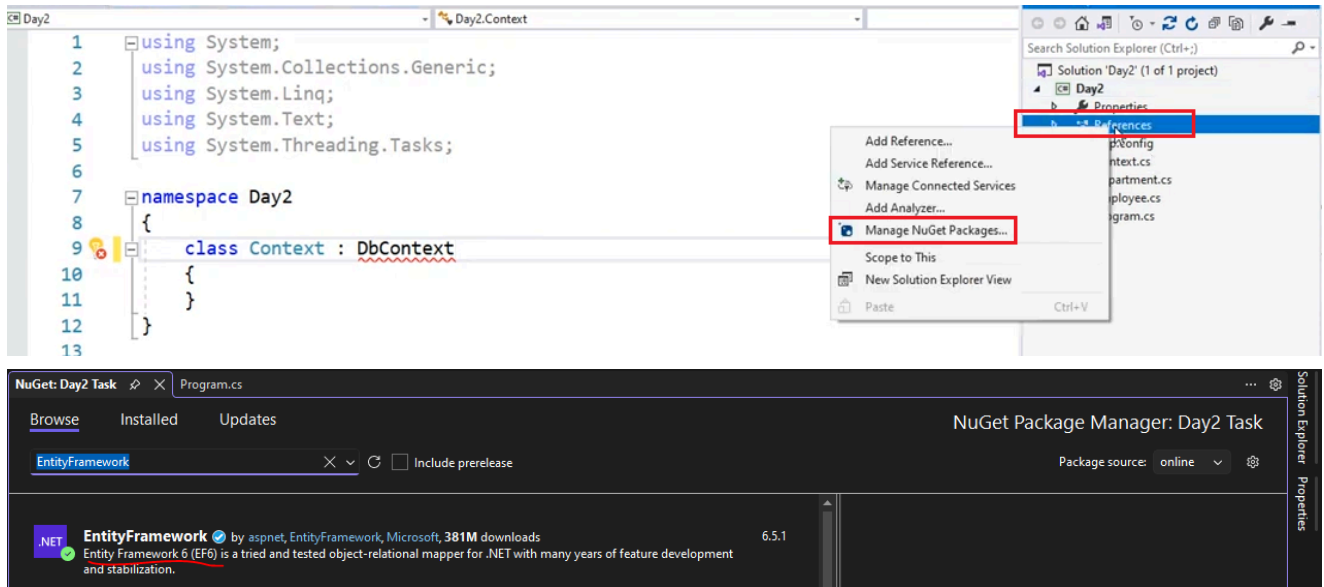
ملاحظة: لسه معملناش أي علاقات (Relationships) بينهم دلوقتي، هناخد الموضوع واحدة واحدة.

الخطوة الثانية: إنشاء الـ Context

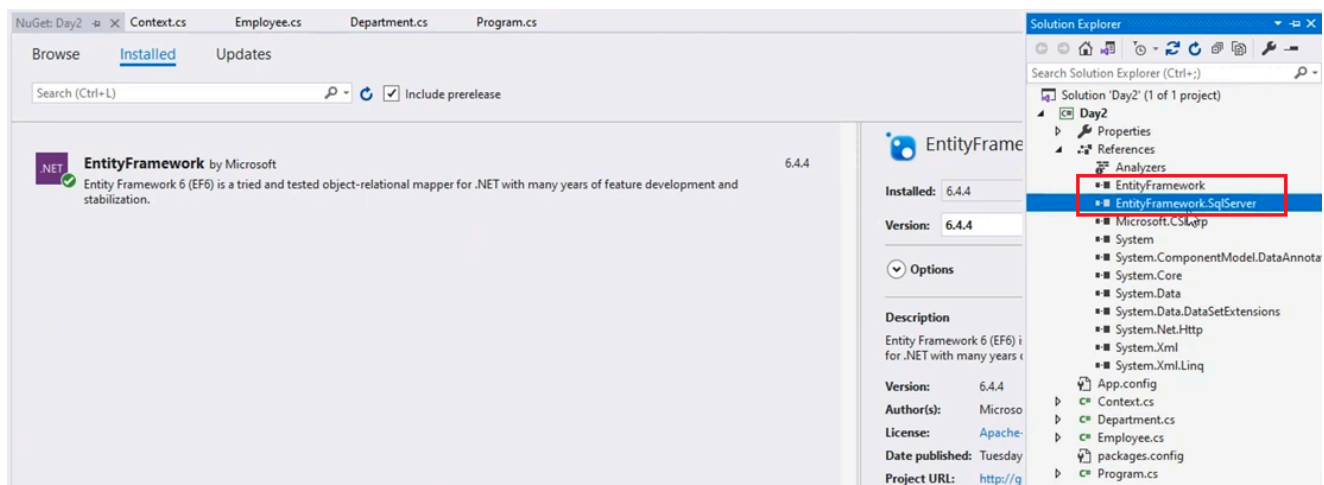
محتاجين الكلاس اللي بيمثل "البوابة" بيني وبين الداتابيز، وهنسميه Context ولازم يورث من DbContext .

- عشان DbContext يظهر، لازم نسطب مكتبة Entity Framework الأول.

تسطيب الـ Entity Framework:



1. نفتح الـ NuGet Package Manager (كليك يمين على References -> Manage NuGet Packages).
2. نعمل Browse ونبحث عن EntityFramework (نختار اللي تبع Microsoft ونعمل Install).



كتابة كود الـ Context: بعد التسطيب، نقدر نستخدم System.Data.Entity . هحدد حاجتين جوه الـ Context:

1. الجداول (DbSets): عايز جدول للـ Employees وجدول للـ Departments.
2. الداتابيز (Connection String): هنبعتها في الـ Constructor مؤقتاً هنكتبها Hardcoded هنا للتسهيل).

```
1 using System.Data.Entity;
2
3 public class Context : DbContext
4 {
```

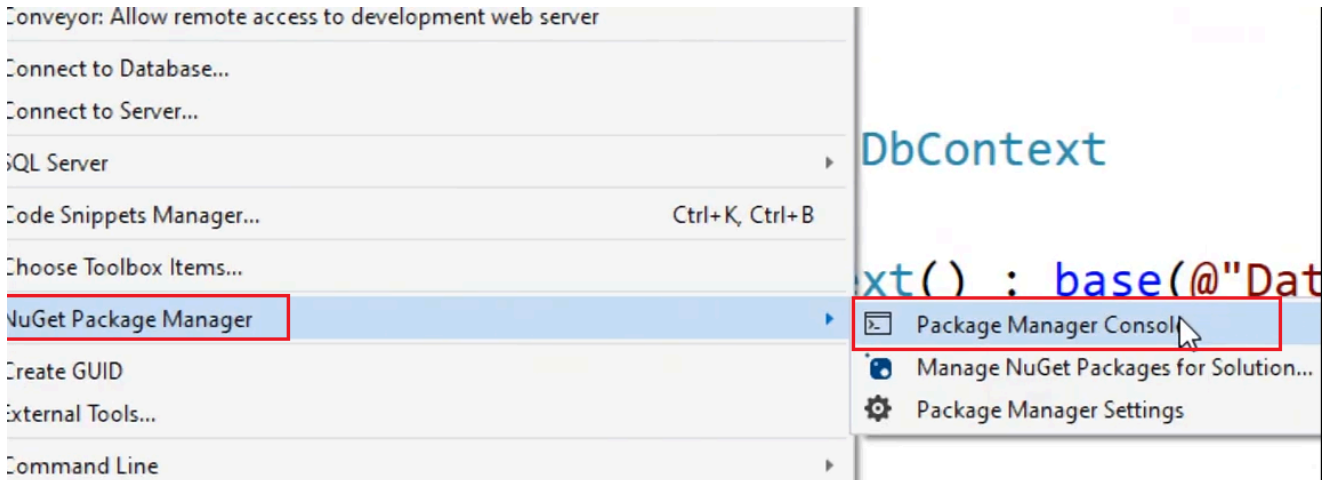
```
5     public Context() : base("Data Source=.; Initial Catalog=EF_Day2_DB;  
Integrated Security=true")  
6     {  
7         // حددنا اسم داتابيز جديدة لسه مش موجودة اسمها EF_Day2_DB  
8     }  
9  
10    public virtual DbSet<Employee> Employees { get; set; }  
11    public virtual DbSet<Department> Departments { get; set; }  
12 }
```

Migrations

2. الـ Migrations (عصب الـ Code First)

دلوقتى إحنا عملنا الكلاسات، بس مفيش داتايز لسه. وعشان كل ما نغير في الكود (نضيف Property مثلاً) التغيير ده يسمع في الداتايز، هنستخدم حاجة اسمها **Migrations**.

كل شغلنا هيكون عن طريق الـ **Package Manager Console**. لو مش ظاهر عندك هاته من:

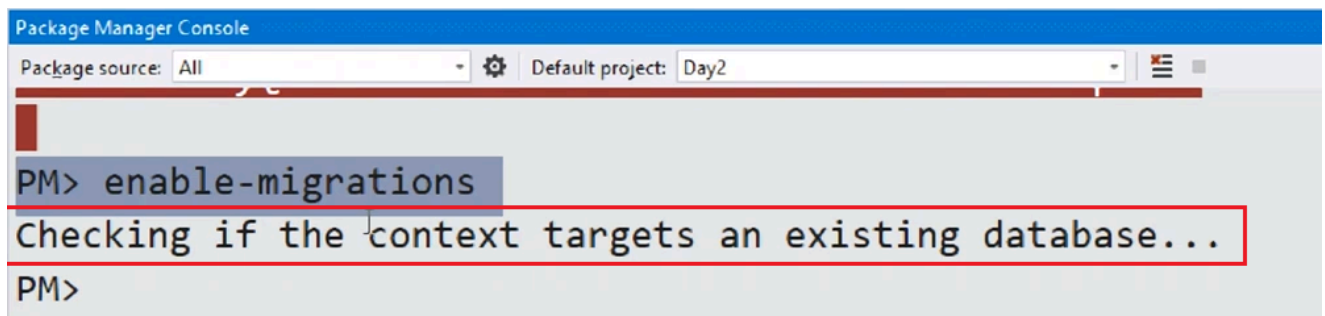


الـ Package Manager Console -> NuGet Package Manager -> Tools .

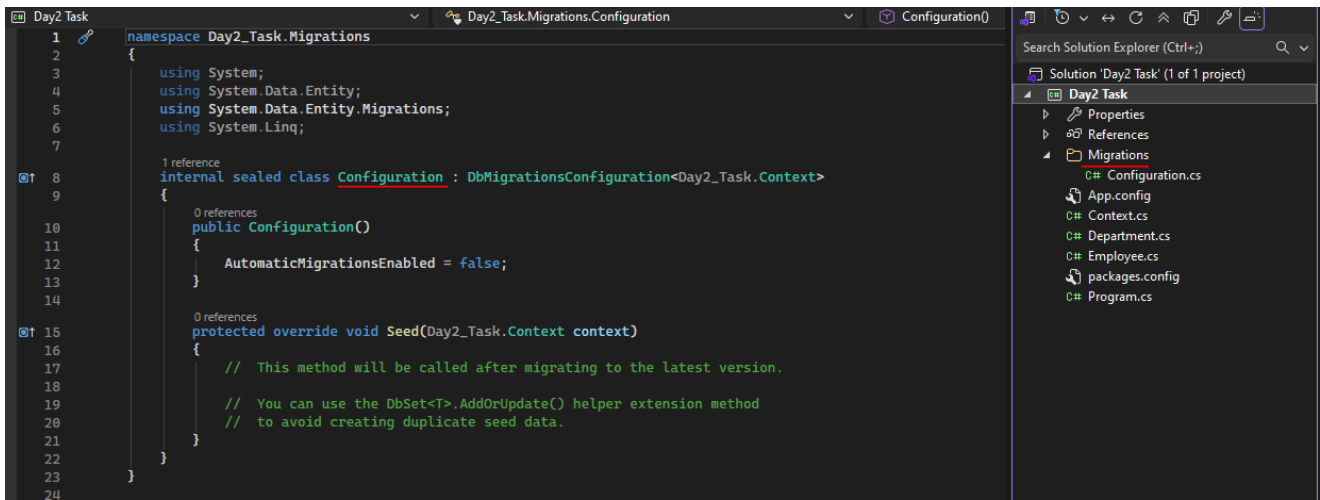
عندنا 3 أوامر (Commands) أساسية لازم تحفظهم زي اسمك:

1. أمر التفعيل: Enable-Migrations

```
1 enable-migrations
```



- **وظيفته:** بيعرف المشروع إننا هنستخدم Migrations.
- **بيكتب كام مرة؟** مرة واحدة بس في عمر المشروع.
- **النتيجة:** بيعمل فولدر جديد اسمه Migrations وجواه فايل اسمه Configuration.cs (هنعرف فايلته بعدين).



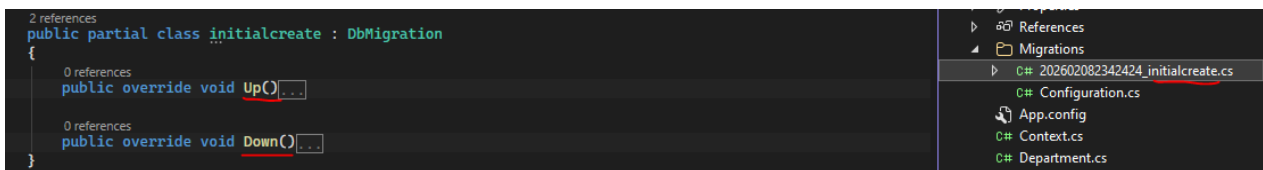
2. أمر إنشاء التغيير: Add-Migration [Name]

- **وظيفته:** يباخذ "لقطة" من التغييرات التي حصلت في الكود ويحولها لخطوات (Code) جاهزة للتنفيذ على الداتابيز.

1 Add-Migration InitialCreate

- **مثال:** Add-Migration InitialCreate

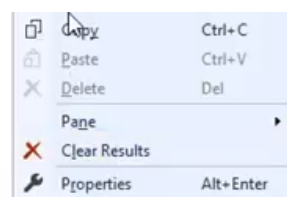
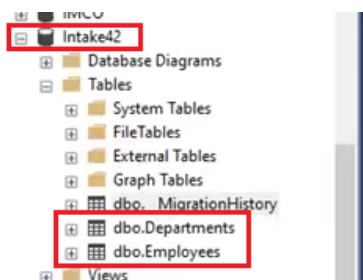
- **النتيجة:** يعمل فايل جديد في فولدر Migrations باسم Timestamp_Name . الفايل ده جواه دالتين:



- **الـ Up() :** الكود اللي هيتنفذ عشان يطبق التغيير (مثلاً CreateTable).
- **الـ Down() :** الكود اللي يلغي التغيير لو حيينا نرجع في كلامنا (مثلاً DropTable).
- **ملاحظة خطيرة:** الأمر ده ميعملش حاجة في الداتابيز، هو بس بيجيز كود التغيير.

3. أمر التنفيذ: Update-Database

- **وظيفته:** يباخذ الكود اللي في Up() ويروح ينفذه فعلياً على الـ SQL Server.
- **النتيجة:** الداتابيز بتتكرت (لو مش موجودة) والجداول بتتعمل.
- **كمان خطوة الـ snapshot** بتتاخد في اللحظة دي (اقصد الى بتتخفظ في الـ MigrationHistory)



3. قواعد الـ Code First الافتراضية (Conventions)

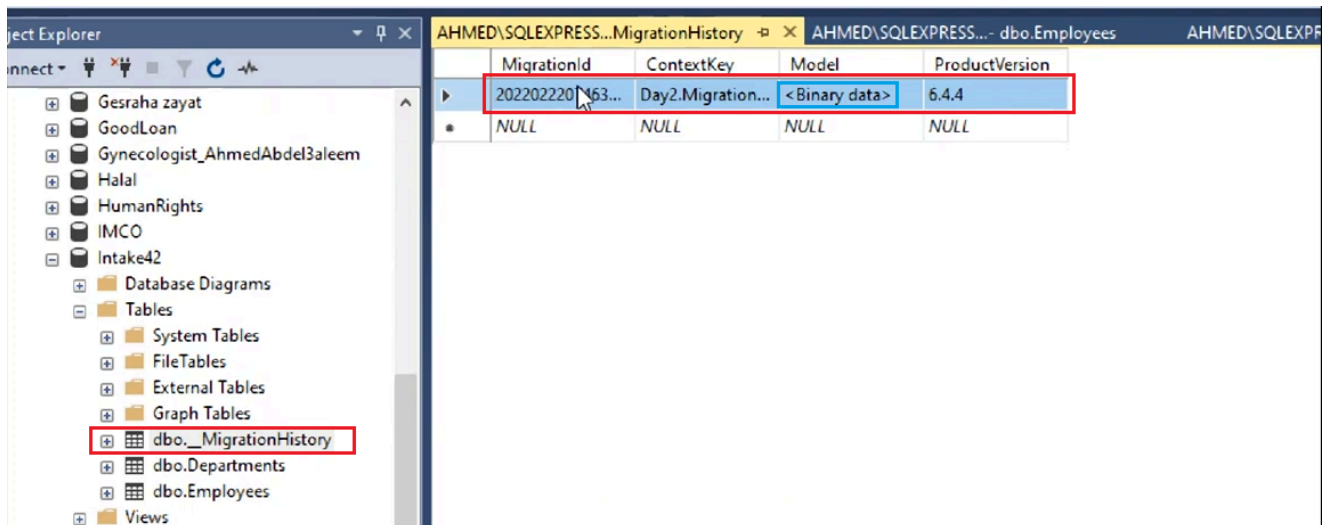
لما عملنا Add-Migration ، ال EF قرر حاجات "باي ديفولت" (By Default) من غير ما نسألّه. القواعد دي مهمة جداً:

1. اختيار الجداول: ال EF بص على ال DbSet اللي في ال Context وعمل لكل واحدة جدول.
2. اسم الجدول: خلاه جمع (Plural) لاسم الكلاس (Employee -> Employees).
3. الأعمدة (Columns): كل Property بقت عمود في الجدول.
 - ال int -> int (Not Null لأنها Value Type).
 - ال string -> nvarchar(max) (لأننا محددناش طول) وهي عملها allow null لأنها reference type.
4. المفتاح الأساسي (PK): دي أهم قاعدة.
 - لو لقي Property اسمها Id (أو ID) أو ClassNameId ، بيخليها هي ال Primary Key وتلقائياً بيخليها Identity (Auto Increment) لو نوعها رقم.

سؤال: ينفع أعمل جدول من غير Primary Key؟ الإجابة: في ال EF Code First ، لأ. هيعترض ويقولك ال Entity دي ملهاش Key.

4. فهم ال MigrationHistory__ (المنطقة المحظورة)

لما عملنا Update-Database ، ال SQL Server اتعمل فيه جدول إضافي اسمه MigrationHistory__ . الجدول ده "لعين" ومهم جداً.



MigrationId	ContextKey	Model	ProductVersion
2022022201563...	Day2.Migration...	<Binary data>	6.4.4
NULL	NULL	NULL	NULL

بيعمل إيه الجدول ده؟

الجدول ده بيخزن Snapshot (لقطة) من شكل الموديل (Classes) بتاعك في آخر مرة عملت فيها Update , خلى بالك بياخد Snapshot من شكل الموديل (Classes) كلها وليس التعديلات الجديدة فقط يعني لو ضيفت salary لكلاس Employee وهو بياخد الاسناب هياخد كل حاجه بما فيهم ال salary .

ال EF بيعرف التغييرات إزاي؟

لما نتيجي تعمل Add-Migration جديد، ال EF مش بيروح يبص على الداتا بيز الحقيقية.

- هو بيقارن الـ **Classes** الحالية بتاعتك مع الـ **Snapshot** القديمة المتخزنة في `MigrationHistory__`.
- لو لقي اختلاف (مثلاً زودت Property)، بيعمل Migration بالفرق ده.

```

6  public partial class updates : DbMigration
7
8  public override void Up()
9
10     AddColumn("dbo.Employees", "Address", c => c.Sti
11
12
13  public override void Down()
14

```

سيناريو الكارثة (وحلّه)

تخيل إنك حذفك الـ `MigrationHistory__` ده بالغلط، أو لعبت في الداتابيز بإيدك (زودت عمود في الـ SQL مباشرة).

1. جيت تعمل `Add-Migration` ، الـ EF هيبص ملقاش `Snapshot` (أو لقاها مش مطابقة).
 2. هيفترض إن دي "أول مرة" وتعمل داتابيز، فهيجنريرت كود `CreateTable`.
 3. لما تيجي تعمل `Update-Database` ، هيضرب `Error` لأن الجداول دي أصلاً موجودة في الداتابيز!.
- الحل (خدعة الـ **Empty Migration**): لو حصل والـ `History` طار أو الداتابيز والـ `Code` مش متزامنين:

1. نعمل `Add-Migration FixSync`.
 2. نروح للفايل اللي طلع، ونفسي الكود اللي جوه `Up()` و `Down()` ونخليهم فاضيين تماماً.
 3. نعمل `Update-Database`.
- كده هو مش هيغير حاجة في الداتابيز (لأن الدوال فاضية).
 - لكن: هياخد **Snapshot** جديدة من الكلاسات الحالية ويحطها في الـ `MigrationHistory__` ، وبكده رجعنا التزامن تاني.

التراجع عن الـ Migrations (Rollback)

لو عملت Migration واكتشفت إنه غلط، أو عايز ترجع لنسخة قديمة من الداتابيز، بنستخدم الأوامر التالية:

1. التراجع على مستوى الداتابيز (Update-Database)

لو أنت فعلاً نفذت أمر `Update-Database` والتغيير سمّع في الـ SQL، وعايز ترجع لـ Migration قديم:

الأمر العام:

1 "اسم_الميجريشن_القديم" Update-Database -TargetMigration

ببمعمل إيه؟ الـ Entity Framework هيروح يشوف إيه الـ Migrations اللي اتنفذت بعد "الميجريشن القديم" ده، وينفذ دالة الـ Down () بتاعتهم بالترتيب (من الأحدث للأقدم) عشان يسمح التعديلات ويرجع الداتابيز للحالة اللي أنت حددتها.

مثال عملي: لو عندك InitialCreate وبعده عملت AddAddress ، وأنت عايز تلغي العنوان وترجع للـ Initial بس:

```
1 Update-Database -TargetMigration "InitialCreate"
```

ملحوظة (تصفير الداتابيز): لو عايز تمسح كل الجداول وترجع الداتابيز "فلات" (فاضية تماماً) زي ما كانت في الأول خالص قبل أي جداول:

```
1 Update-Database -TargetMigration:0
```

ملخص الخطوات العملية (Workflow)

1. ابني/عدل في الـ Classes (مثلاً زودت Address في Employee).
 2. الـ Add-Migration AddAddressColumn -> يتكون فايل فيه AddColumn (...).
 3. الـ Update-Database -> يتنفذ على الـ SQL.
- نصيحة أخيرة في الجزء ده:** متلعبش في الـ MigrationHistory__ ولا تعدل في الداتابيز بايديك (SQL) وأنت شغال Code First، دايماً خلي التعديل يبدأ من الكود.

Configuration & Data Annotations

مقدمة: إنا بنعمل إيه؟

في المحاضرة اللي فاتت، شوفنا إن الـ EF بيشتغل بـ **Conventions** (افتراضات)؛ يعني لو سميت `Id` هو يفهم إنه **Primary Key**، ولو فيه `List` يفهم إنها علاقة، وهكذا. لكن إنا محتاجين نغير الـ **Defaults** دي (مثلاً نغير اسم الجدول، نغير نوع الـ `Column`، نحدد **Constraints**).

عندنا طريقتين عشان نعمل الـ **Configuration** ده:

1. الـ **Data Annotations**: دي عبارة عن **Attributes** بنحطها فوق الـ `Class` أو الـ `Property`. ميزتها سهلة جداً.
2. الـ **Fluent API**: دي طريقة تانية بنتكتب كود `C#`، إمكانياتها أكبر بكثير ومخصصة للـ **Code First**، بس أصعب شوية للمبتدئين عشان بتعتمد على **Lambda Expressions**.

ملاحظة: في المحاضرة دي التركيز كله كان على **Data Annotations** "حقناً للدماء".

1. استخدام **Data Annotations** لتغيير الـ **Schema**

الـ **Data Annotations** موجودة في **Namespace** اسمه:

```
1 System.ComponentModel.DataAnnotations.Schema;
```

أ. تغيير اسم الجدول والـ **Schema**

لو عايز نغير اسم الجدول من `Departments` (الجمع اللي الـ EF بيعمله) لـ `Department` (المفرد)، أو نغير الـ **Schema** من `dbo` لـ `HR`.

الكود:

C#

```
1 [Table("Department", Schema = "HR")]
2 public class Department
3 {
4     // ...
5 }
```

- الـ **[Table]**: بياخد اسم الجدول كـ **Parameter** أول، ويمكن تحدد الـ **Schema**.
- **الوضع الافتراضي (Default)**: أي جدول بيتعمل في الداتابيز بياخد اسكينا افتراضية اسمها `dbo` (اختصار لـ **DataBase Owner**). فاسم الجدول الكامل بيكون `dbo.Departments`.
- **اللي بيحصل في الكود:** لما بنكتب `Schema = "HR"`، إنا بنقول للـ **Entity Framework**: "لما تروح تكريت الجدول ده في الداتابيز، متخلهوش في الـ `dbo` العادية، لا حطه تحت تصنيف أو مجموعة اسمها `HR`".

- **التنظيم:** لو الداتا بيز كبيرة جداً، بنقسم الجداول لمجموعات (مثلاً جداول الـ HR لوحدها، جداول الـ Sales لوحدها، جداول الـ Inventory لوحدها) عشان الدنيا تكون منظمة.
- **الأمان (Security):** ممكن تدي صلاحيات لموظف معين يشوف بس الجداول اللي تحت الـ Schema اللي اسمها HR وما يشوفش باقي الجداول.

الخلاصة: في الكود ده، الـ Schema مش معناها هيكل الداتا بيز كله، لكن معناها "اسم المجموعة" اللي الجدول ده هينتمي ليها جوه الداتا بيز (زي dbo أو HR).

ب. تغيير اسم الـ Column

لو عايز تغير اسم العمود في الداتا بيز ويكون مختلف عن اسم الـ Property في الكلاس (مثلاً في الكلاس اسمه Name بس في الداتا بيز عايزه FullName).

الكود:

C#

```
1 public class Employee
2 {
3     [Column("FullName")]
4     public string Name { get; set; }
5 }
```

- الـ [Column]: بيغير اسم العمود في الداتا بيز.

ج. التعامل مع الـ Nulls والـ Required

- افتراضياً: الـ String والـ Objects في C# بيقبلوا null ، ف EF بيترجمهم لـ Nullable في الداتا بيز.
- لو عايز تخليه Not Null : بنستخدم [Required].
- العكس: لو عندك int (هو Value Type مبيقبلش null) وعايزه يقبل null في الداتا بيز، بتخليه nullable<int أو int? .

الكود:

C#

```
1 [Required] // في الداتا بيز not null هيتحول لـ
2 public string Name { get; set; }
3
4 public double? Salary { get; set; } // في الداتا بيز nullable هيتحول لـ
```

د. تحديد طول الـ String (Max Length)

بدل ما الـ String يتحول لـ nvarchar(max) ، ممكن نحدد طول معين.

```
1 [MaxLength(100)]
2 public string Name { get; set; }
```

- يمكن ندمج أكثر من Attribute ورا بعض أو فوق بعض.

هـ. تغيير الـ (Data Type Mapping)

لو عندك DateTime في #C وعائزه يتخزن في الداتابيز Date بس (من غير وقت)، بنستخدم [Column] ونحدد الـ TypeName .

الكود:

```
1 [Column(TypeName = "date")]
2 public DateTime BirthDate { get; set; }
```

2. نقاش هام: ليه أكتب Constraints هنا مش في الداتابيز؟

سؤال اتسأل: "مش الأسهل أعمل الكلام ده في الداتابيز وخلاص؟".

الرد والفوائد:

1. الـ **Code First Philosophy**: الفكرة إنك بتتحكم في كل حاجة من الكود ومش شرط تكون خبير داتابيز.
2. الـ **Validation**: دي أهم نقطة. الـ Attributes دي مش بس بتسمع في الداتابيز، دي كمان بتعمل **Validation** على مستوى الـ **Object** في الميموري قبل ما تروح للداتابيز. يعني لو حاولت تدخل اسم أطول من 100 حرف، الـ EF هيرميك Exception وأنت لسه في الكود.
3. الـ **Limitations**: فيه حاجات لازم تتعمل في الداتابيز (زي الـ Unique Constraint)، دي مش هينفع تتعمل بـ Data Annotations بسيطة ولازم تتهندل ناحية الداتابيز أو بـ Fluent API.

3. التعامل مع الـ Primary Keys

لو الـ Property مش اسمها ID أو ClassNameID ، الـ EF مش هيعرف إنها PK.

أ. تحديد الـ Primary Key يدويًا

بنستخدم [Key] .

الكود:

```
1 [Key]
2 public int EmpCode { get; set; }
```

ب. الـ Identity (Auto Increment)

لو عرفت PK، هو أوتوماتيك بيكون Identity. لو لسبب ما عايز تأكد عليه أو تغيره، بنستخدم [DatabaseGenerated].

الكود:

```
1 [Key]
2 [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
3 public int EmpCode { get; set; }
```

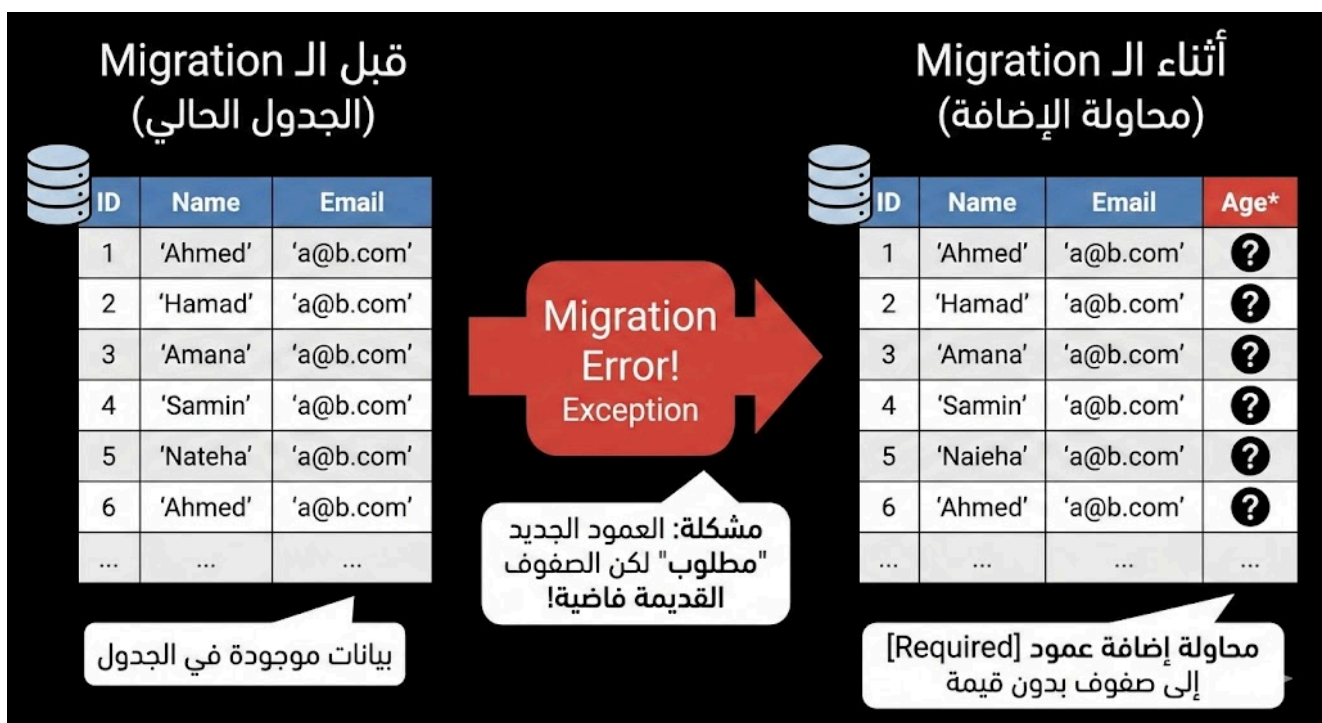
طبعا لو انت عندك اكثر من Data Annotations وعاوز تكتبهم لـ Attribute معينة هتكتب بالشكل ده

```
1 [Required, MaxLength(100), Column("FullName")]
2 public string Name { get; set; }
```

4. مشكلة الـ Migrations مع البيانات الموجودة

دي نقطة "خبيثة" ومهمة جداً في الشغل العملي. لو عندك جدول فيه داتا، وقررت تضيف عمود جديد [Required] (يعني (Not Null).

- المشكلة: الداتا بيز هتعارض؛ لأنك بتقول العمود ده ممنوع يبقى فاضي، وفي نفس الوقت فيه صفوف قديمة (Records) الخانة دي فيها فاضية!.
- الحل: الـ Migration هتضرب Exception.



طرق الحل اللي اتقالت في المحاضرة:

1. الطريقة اليدوية: خلي العمود Nullable الأول، اعمل Update، املأ البيانات للبيانات القديمة، وبعدين رجعه Required ثاني.

2. تعديل ملف الـ Migration (للمحترفين):

- لما تعمل Add-Migration ، الـ EF بيعمل ملف #C فيه الخطوات.
- ممكن تدخل جوه الملف ده، وفي دالة AddColumn ، هتلاقي باراميتر اسمه defaultValue أو defaultValueSql .
- ممكن تحط قيمة افتراضية (مثلاً string فاضي "" أو 0) عشان الداتا بيز تقبل العمود الجديد.

مثال لتعديل ملف المايجریشن:

```
1 AddColumn("dbo.Departments", "Location", c => c.String(nullable: false, defaultValue: ""));
```

المشكلة: إضافة عمود [Required] لجدول به بيانات

قبل الـ Migration

ID	Name	Email	NewCol*
1	Ariin	jezismith@gmail.com	?
2	Name	mratelæ@gmail.com	?
3	Bhör	reirairoh@gmail.com	?
4	King	mrtaruk@gmail.com	?

Migration Exception!

الصفوف القديمة فارغة، والعمود الجديد مطلوب!

الحل: استخدام قيمة افتراضية (defaultValue)

بعد الـ Migration الناجحة

ID	Name	Email	NewCol*
1	Ariin	jezismith@gmail.com	"" ✓
2	Name	mratelæ@gmail.com	"" ✓
3	Bhör	rekairoh@gmail.com	"" ✓
4	King	mrtaruk@gmail.com	"" ✓

```
AddColumn(..., "NewCol", c => c.String(nullable: false, defaultValue: ""));
```

يتم ملء الصفوف القديمة بالقيمة الافتراضية تلقائياً

تنبيه: متعملش حركة إنك تخلي nullable: true في المايجریشن وهو Required في الكلاس، كده أنت بتعمل عدم تطابق (Mismatch) بين الكود والداتا بيز.

Advanced Relationships & Teamwork

1. علاقات One-to-Many

مثال: القسم (Department) والموظفين (Employees).

الطريقة الأساسية (Conventions)

بنستخدم **Navigation Properties**.

1. في كلاس Employee :

```
1 public virtual Department Department { get; set; }
```

2. في كلاس Department

```
1 public virtual ICollection<Employee> Employees { get; set; }
```

ال EF هيفهم العلاقة لوحده و هيعمل عمود Foreign Key في جدول الموظفين يسميه Department_Id في الداتا بيز.

ليه Virtual؟ عشان نفعل خاصية ال Lazy Loading.

التحكم في ال Foreign Key (مهم جداً)

أحياناً بنحتاج يكون معنا ال ID بتاع القسم جوه كلاس الموظف ك Property صريحة، مش بس ال Object. ليه؟ عشان لو عايز أعرف هو في قسم رقم كام، مش منطقي أروح أجيب ال Object بتاع القسم كله من الداتا بيز (Lazy Load) عشان بس أعرف ال ID بتاعه. ده بيوفر في الأداء.

إزاي نربط ال ID بالـ **Navigation Property**؟ بنستخدم [ForeignKey].

الكود:

```
1 public class Employee
2 {
3     public int Id { get; set; }
4
5     // 1. ال Foreign Key Property
6     public int DeptId { get; set; }
7
8     // 2. ال Navigation Property
9     // هو المفتاح بتاع العلاقة دي DeptId إن EF بنقول للـ
10    [ForeignKey("DeptId")]
11    public virtual Department Department { get; set; }
12 }
```

- ممكن نخط ال Attribute فوق ال Navigation Property (زي المثال) ونكتب اسم ال FK Property.
- أو نخطه فوق ال FK Property ونكتب اسم ال Navigation Property.

- الأفضل: كتابة اسم الـ Navigation Property مش اسم الكلاس، عشان لو فيه أكثر من علاقة مع نفس الكلاس الـ EF ميتلخبطش.

مثال آخر الـ Entity Framework عمل جدول مع إننا ما ضفناهوش جوه الـ DbContext بشكل صريح.

السيناريو اللي حصل

في المثال، إحنا ضفنا كلاس جديد اسمه `Project` وعملنا علاقة **One-to-Many** مع `Department`.

```
1 public class Project
2 {
3     public int Id { get; set; }
4     public string Name { get; set; }
5
6     // للربط مع القسم
7     // (Optionally add Foreign Key if needed, but navigation is enough
8     for discovery)
9     public virtual Department Department { get; set; }
10 }
```

```
1 public class Department
2 {
3     public int Id { get; set; }
4     public string Name { get; set; }
5     public virtual ICollection<Employee> Employees { get; set; }
6
7
8     // يشوف كلاس البروجكت EF دي بقى "الوصلة" اللي خلت الـ
9     public virtual ICollection<Project> Projects { get; set; }
10 }
```

- جوه `Department` ضفنا `Collection` من الـ `Projects`.
- المفاجأة كانت إننا **معلمناش** `DbSet<Project>` جوه الـ `Context`.
- ومع ذلك، لما عملنا `Migration`، الـ EF راح عمل كريت للتبيل اللي اسمه `Project` وربطه بالـ `Department` والدنيا اشتغلت زي الفل.

```

10 CreateTable(
11     "dbo.Projects",
12     c => new
13     {
14         ID = c.Int(nullable: false, identity: true),
15         Name = c.String(),
16         Department_ID = c.Int(),
17     })
18     .PrimaryKey(t => t.ID)
19     .ForeignKey("HR.Department", t => t.Department_ID)
20     .Index(t => t.Department_ID);
21
22 AddColumn("dbo.Employees", "DepartmentID", c => c.Int(nullable: f
23 CreateIndex("dbo.Employees", "DepartmentID");
24 AddForeignKey("dbo.Employees", "DepartmentID", "HR.Department", "

```

التفسير: هو عرف منين؟

السؤال هنا: مش إحنا قلنا إن الـ EF بيكرت جداول للكلاسات اللي معموله DbSet بس؟ ليه عمل ده؟

الإجابة:

1. الكونتيكست بتاعنا جواه DbSet للكلاس اللي اسمه Department .
2. الـ EF لما بييجي يشتغل، بيص على الـ Structure (الهيكل) بتاع كلاس Department ده.
3. وهو بيفحص Department ، لقي جواه Collection من نوع Project .
4. فهم إن فيه Relationship (علاقة) بتربط بين Department وبين الكلاس ده.
5. استنتج إن المفروض الكلاس Project ده يتحط في الداتابيز ويتعمل له Table عشان العلاقة دي تتم.

القاعدة العامة لإنشاء الجداول في الـ DB

بناءً على الكلام ده، الجداول بيحصلها Create في الداتابيز في حالتين اتنين:

1. الحالة الصريحة: إن اسم الكلاس يكون محطوط جوه DbSet في الـ DbContext .

```

class Context : DbContext
{
    public Context() : base(@"Data source=.\sqlExpress;Initial catalog= I
    {
        ...
    }

    public DbSet<Department> Departments { get; set; }
    public DbSet<Employee> Employees { get; set; }
}

```

2. حالة الـ Reachability: إن الكلاس يكون موجود جوه كلاس ثاني (كمتغير أو List) والكلاس الثاني ده أصلاً ليه علاقة بالكونتيكست (موجود في DbSet) .

العيب: مفيش Query مباشر (Direct Query)

بما إنك معملتش `<DbSet<Project>` جوه الـ `Context`، ده معناه إن الكلاس `DbContext` بتاعك مفيهوش خاصية اسمها `. Projects`.

النتيجة:

1. مش هتقدر تكتب: `context.Projects.ToList()` أو `context.Projects.Where(...)`. الكود ده هيدريك Error لأن `Projects` مش موجودة.

2. الوسيلة الوحيدة للوصول: عشان تجيب مشروع معين، لازم "تلف" وتجيله من طريق الـ `Department`.

مثال بالكود يوضح المشكلة

1. الطريقة اللي كنا بنحلم بيها (ومش هتنتفع دلوقتي): لو كنت عايز تجيب كل المشاريع اللي اسمها "AI Project"، كنت هتكتب كده:

```
1 // ❌ (Compilation Error) الكود ده مش هيشغل
2 // context جوه الـ Projects لأن مفيش
3 var projects = context.Projects.Where(p => p.Name == "AI
  Project").ToList();
```

2. الطريقة الوحيدة المتاحة (اللفة الطويلة): عشان توصل للمشروع، لازم تجيب القسم الأول، ومن جواه تدخل على الليسته بتاعته:

```
1 // ✅ دي الطريقة الوحيدة دلوقتي
2 // عشان توصل للمشاريع SelectMany الأول وتعمل Departments لازم تروح للـ
3 var projects = context.Departments
4     .SelectMany(d => d.Projects)
5     .Where(p => p.Name == "AI Project")
6     .ToList();
```

2. علاقات Many-to-Many

الموظف (Employee) والمشروع (Project). الموظف بيشغل في أكثر من مشروع، والمشروع فيه أكثر من موظف.

أ. الطريقة السهلة (Convention)

لو العلاقة Many-to-Many صريحة ومفيش أي بيانات زيادة محتاج تخزينها في الجدول الوسيط (غير الـ IDs بس)، الـ EF بيعمل كل حاجة لوحده.

الكود:

1. في كلاس `Employee`

```
1 public virtual ICollection<Project> Projects { get; set; }
```

3. في كلاس `Project`

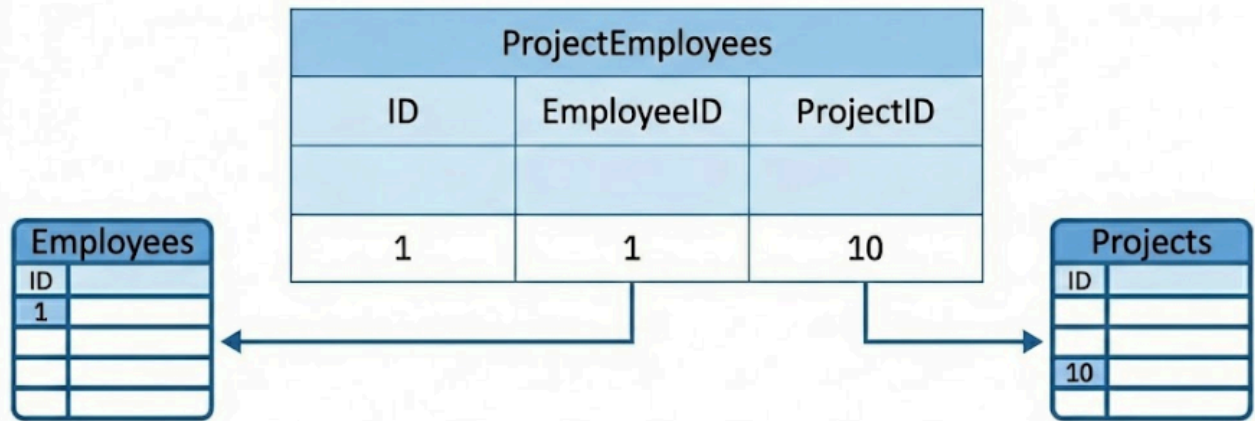
```
1 public virtual ICollection<Employee> Employees { get; set; }
```

- النتيجة: الـ EF يعمل جدول ثالث في الداتابيز اسمه ProjectEmployees (أو اسم مركب)، ويحيط فيه EmployeeID و ProjectID ويخليهم هما الاثنين Composite Primary Key.

```
public override void Up()
{
    CreateTable(
        "dbo.ProjectEmployees",
        c => new
        {
            Project_ID = c.Int(nullable: false),
            Employee_ID = c.Int(nullable: false),
        })
        .PrimaryKey(t => new { t.Project_ID, t.Employee_ID })
        .ForeignKey("dbo.Projects", t => t.Project_ID, cascadeDelete:
        .ForeignKey("dbo.Employees", t => t.Employee_ID, cascadeDelete:
        .Index(t => t.Project_ID)
        .Index(t => t.Employee_ID);
```

ولو حببت تضيف في الجدول الوسيط ده (الحالة دي مش بتتعامل مع الجدول الوسيط مباشرة) لأنه أصلاً مش موجود عندك كـ Class في الكود). بدل ما تعمل Insert في جدول، أنت بتتعامل مع الـ Collections اللي جوه الكلاسات)

```
1 // 1. هجيب الموظف رقم 1 من الداتابيز
2 var employee = context.Employees.Find(1);
3
4 // 2. هجيب المشروع رقم 10 من الداتابيز
5 var project = context.Projects.Find(10);
6
7 // 3. هضيف المشروع ده لليسته المشاريع اللي عند الموظف
8 // (الموظف ده بيشغل في المشروع ده: EF كده أنا بقول للـ)
9 employee.Projects.Add(project);
10
11 // 4. حفظ التغييرات
12 // Insert Statement هيترجم السطر اللي فوق لـ EF هنا الـ
13 context.SaveChanges();
```



ب. الطريقة اليدوية (Custom Join Table)

أما لو محتاج تخزين بيانات زيادة في العلاقة (مثلاً: عدد الساعات اللي الموظف اشتغلها في المشروع ده، أو تاريخ الانضمام)، هنا الـ Convention مش هينفع. لازم تتدخل وتعمل **Class** يمثل الجدول الوسيط بنفسك.

الخطوات:

1. نعمل كلاس جديد وليكن اسمه `WorksFor` (أو `ProjectEmployee`).
2. الكلاس ده هيتحتوي على:
 - الـ `ProjectId (FK)` و `EmployeeId (FK)`.
 - البيانات الزيادة: `int Hours`.
 - الـ `Navigation Properties` للطرفين: `Project` و `Employee`.
3. نعدل الكلاسات الأصلية:
 - بدل ما `Employee` عنده ليستة من `Project`، هيبقى عنده ليستة من `WorksFor`.
 - وبالمثل في `Project`.

الكود:

```

1  public class WorksFor
2  {
3      [Key, Column(Order = 1)]
4      public int EmployeeId { get; set; }
5
6      [Key, Column(Order = 2)]
7      public int ProjectId { get; set; }
8
9      public int Hours { get; set; } // المعلومة الزيادة
10
11     // اوبجيكت مش كوليكشن لان الصف عندي هيبقى عبارته عن التالي
12     // ID | EmployeeId(one object) | ProjectId(one object)
13     public virtual Employee Employee { get; set; }
14     public virtual Project Project { get; set; }
15 }
16

```

```

17 // في كلاس Employee
18 public virtual ICollection<WorksFor> WorksForItems { get; set; }
19 // في كلاس Project
20 public virtual ICollection<WorksFor> WorksForItems { get; set; }

```

3. مشكلة العلاقات المتعددة (InverseProperty)

دي مشكلة بتحصل لما يكون فيه أكثر من علاقة بين نفس الجدولين.

السيناريو: عندنا Department و Employee .

1. العلاقة الطبيعية: القسم فيه موظفين كثير (Employees).
2. علاقة تانية: القسم ليه مدير (Supervisor)، والمدير ده أصلاً موظف (Employee).

المشكلة: لما نتيجي تعمل List<Employee> Employees و List<Employee> Supervisors جوه كلاس Department ، الـ EF مش هيعرف يربط مين بمين.

- هل ليستة Employees دي بتاعت الناس اللي شغالة في القسم؟ ولا بتاعت المديرين؟
- الـ EF هيتلخبط لأنه شايف اتنين Foreign Keys في جدول الموظفين ومش عارف يربطهم بأنهي Navigation Property في القسم.

الحل: [InverseProperty] لازم نقول للـ EF صراحة: "يا EF، الـ Employee دي مربوطة بالـ Department دي هناك".

الكود:

```

1 public class Department
2 {
3     // 1. علاقة الموظفين العادية
4     // اللي عند الموظف Department بنقول له: الـ Employee دي هي "عكس" الـ
5     [InverseProperty("Department")]
6     public virtual ICollection<Employee> Employees { get; set; }
7
8     // 2. علاقة المديرين
9     // اللي عند الموظف ManagedDepartment بنقول له: الـ Employee دي هي "عكس" الـ
10    [InverseProperty("ManagedDepartment")]
11    public virtual ICollection<Employee> Supervisors { get; set; }
12 }
13
14 public class Employee
15 {
16     // ده الموظف شغال في انهي قسم
17     public virtual Department Department { get; set; }
18
19     // ده الموظف بيدير انهي قسم (لو كان مدير)
20     public virtual Department ManagedDepartment { get; set; }

```

- الشرح: InverseProperty ينتحط فوق الـ Collection ويتأخذ اسم الـ Navigation Property اللي في الطرف الثاني (الكلاس الثاني).

4. مواضيع للمذاكرة الذاتية (Self-Study)

المحاضر طلب القراءة عن المواضيع دي عشان هتتغير في EF Core:

1. الـ **Complex Types**: إزاي تعمل كلاس ملهوش جدول بس هو جزء من جدول ثاني (زي Address جوه User).
2. الـ **Inheritance Strategies**: وبالأخص Table Per Hierarchy (TPH)، وإزاي EF بيمثل الوراثة في الداتابيز.

5. مشاكل الـ Migrations والـ Teamwork (هام جداً)

دي أهم نصيحة عملية في المحاضرة.

السيناريو الكارثي: أنت شغال في تيم، وكل واحد عنده داتابيز Local على جهازه، أو كلكم شغالين على داتابيز مركزية (Centralized).

1. أنت عملت تعديل في الموديل، وعملت Add-Migration، وعملت Update-Database.
2. الداتابيز اتعدلت، والـ Snapshot اللي عندك اتعدلت.
3. قمت ماحطتش ملف الـ **Migration** على الـ Source Control (Git)، أو مسحته بعد ما خلصت.
4. زميلك سحب الكود (Pull)، بس معندوش ملف المايجریشن اللي أنت عملته.
5. الـ Model اللي عنده بقى جديد، بس الـ Snapshot القديمة اللي عنده مش شايعة التعديلات دي، والداتابيز (لو مشتركة) سابقة الكود بتاعه.

النتيجة: لما يجي يعمل Add-Migration شغله هيضرب، أو هيطلع مايجریشن بتعمل حاجات موجودة أصلاً، وهتحصل Conflicts لا حصر لها.

القاعدة الذهبية:

- متمسحش ملفات المايجریشن: طالما شغال في تيم، ملفات المايجریشن جزء من الكود لازم تترفع (Push) لزميلك.
- نظام الشغل: لازم التيم يتفق على نظام للمايجریشن (مين بيعملها وإمتى)، والأفضل التدريب على ده بدري قبل مشاريع التخرج والشغل الحقيقي.

ملحوظة للمستقبل: في EF Core الموضوع اختالف شوية وبقت الـ Snapshot بتعتمد على الكود أكثر، بس المبدأ واحد: لازم تنسيق.

EF Core 5 (Migrations)

1. حكاية NET Core. ولية ظهرت؟

قبل ما ندخل في الكود، لازم نفهم أصل الحكاية. زمان كان عندنا **NET Framework**، ودي كانت الـ Environment اللي لازم تكون موجودة عند الديفلوبر وعند الكلاينت عشان يرن الأبليكيشن.

- **المشكلة القديمة:** لو الكلاينت شغال Windows، لازم ينزل NET Framework للويندوز. لو شغال Linux، محتاج نسخة ثانية للينكس. يعني مع كل تغيير في الـ Platform أو الـ OS، كنا محتاجين Environment مختلفة.
- **الحل (NET Core):** مايكروسوفت عملت **NET Core** عشان تخلينا نعمل **Cross-Platform Applications**. يعني بتكتب الكود مرة واحدة، ويرن على أي حاجة (Windows, Linux, Mac, Mobile, etc).

تطور الإصدارات (Versions History)

- بدأنا بـ Core 1, 2, 3, 3.1.
- بعد 3.1، مايكروسوفت عملت "قفزة" وسمت الإصدار اللي بعده **NET 5** (مسموش Core 5 ولا عملوا 4 عشان مايحصلش لخبطة مع 4.8 NET Framework القديم).
- حالياً إحنا بنجري وراهم في NET 6 وما بعدها.
- الـ **EF Core Versions**: ماشية مع أرقام الدوت نت، يعني EF Core 5 تحت مظلة NET 5، وهكذا.
- **الأداء (Performance):** في فرق شاسع في السرعة والأداء لصالح NET 5 و 6 مقارنة بـ EF6 القديمة.

2. تجهيز المشروع (Installation & Packages)

المرة دي مش هنسطب باكجج واحدة زي زمان (6 EntityFramework وخلص). الـ EF Core مبنية بشكل Modular، يعني بتنزل اللي انت محتاجه بس. وعشان نشغل صح محتاجين نركز في النقط دي:

تعدد الـ Providers

في EF Core، إحنا مش مربوطين بـ SQL Server بس. إحنا ممكن نكونكت على Oracle, PostgreSQL، أو حتى **Non-Relational DB** زي CosmosDB بتاعة Azure. عشان كده لازم نحدد إحنا هنشتغل مع مين.

الـ Packages المطلوبة

عشان نبدأ مشروع Console Application بيستخدم EF Core مع SQL Server، لازم نفتح الـ NuGet Package Manager ونسطب الـ 3 دول:

1. الـ **Microsoft.EntityFrameworkCore**: دي الباكجج الأساسية للكور.
2. الـ **Microsoft.EntityFrameworkCore.SqlServer**: دي عشان إحنا هنكلم SQL Server (خلي بالك فيه واحدة ثانية للـ SQLite، فمتلخبطش).
3. الـ **Microsoft.EntityFrameworkCore.Tools**: ودي مهمة جداً عشان نقدر نستخدم أوامر الـ Migrations (لأن مفيش **Enable-Migrations** زي زمان، الأوامر دي بتيجي مع الباكجج دي).

3. الـ Configuration والاتصال بالـ داتا بيز

بعد ما عملنا الكلاسات بتاعتنا (Department , Employee , Branch) وعملنا كلاس الـ Context وورثنا من DbContext ، نيجي لأول اختلاف كبير.

تحديد الـ Connection String

زمان كنا بنبعت الـ Connection String للـ Base Constructor. دلوقتي لأ، الـ EF Core بتستخدم طريقة تانية عشان هي بتدعم كذا Provider.

بنعمل **Override** لميثود اسمها OnConfiguring . الميثود دي بيحصلها Call وإحنا بنعمل Configuration للـ Context.

الكود:

```
1 // context داخل كلاس
2 protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
3 {
4     // عشان نحدد البروفايذر والكونكشن سترينج optionsBuilder بنستخدم الـ
5     // مش هتظهر غير لما تكون مسطب الباكج بتاعتها UseSqlServer الميثود
6     optionsBuilder.UseSqlServer("Data Source=.;Initial
Catalog=EFCoreDB;Integrated Security=True");
7 }
```

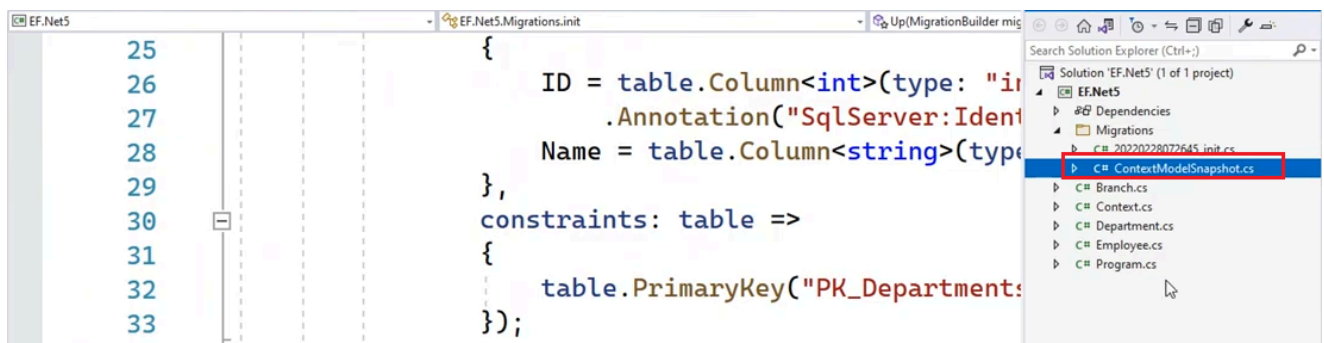
- الـ DbContextOptionsBuilder : ده كلاس بيشيل كل الإعدادات اللي الـ EF محتاجها عشان تكلم الداتا بيز (تطبيق الـ Builder Design Pattern).
- سؤال طالب: ينفع أكونكت على أكثر من داتا بيز في نفس الوقت؟
- الرد: "At a time" أكيد لأ، هو Provider واحد شغالين عليه في الـ Instance الواحدة.

4. الـ Migrations (الاختلافات والمشاكل)

دي أكثر حته فيها تغييرات جوهرية عن زمان، ولازم نتفهم كويس عشان المشاكل اللي بتقابلنا في الشغل.

1. الـ Snapshot (التغيير الكبير)

لما عملنا Add-Migration InitialCreate ، لاحظنا إنه عمل فايل الميجريشن العادي، بس كمان عمل فايل ثاني اسمه ContextModelSnapshot .



- **زمان (EF6):** كانت الـ Snapshot (صورة الداتا بيز الحالية) بتتخزن جوه الداتا بيز نفسها (في جدول MigrationHistory_ __ مشفرة. وكان بيقرن الكود بتاعك باللي موجود في الداتا بيز.
- **دلوقتي (EF Core):** الـ Snapshot بقت **File جوه البروجكت** (ناحية الأبليكيشن).
- **ليه؟** عشان مشاكل الـ Conflicts اللي كانت بتحصل لما تيم كامل شغال على Source Control (Git)، وكل واحد معاه داتا بيز Local، فكانت الدنيا بتضرب لما نيجي نجمع الشغل. دلوقتي المقارنة بتحصل بين الكلاسات (Models) وبين فايل الـ Snapshot اللي معاك في الكود.

2. التعامل مع الـ (Migrations (Remove & Update)

خد بالك من السيناريوهات دي عشان بتلبنسنا في الحيط:

سيناريو أ: عملت Migration ولسه ماعملتش Update Database

- لو اكتشفت إن في حاجة غلط في الميجريشن (مثلاً ضفت Salary بالغلط).
- **الحل:** عادي جداً تعمل

1 Remove-Migration

- **اللي بيحصل:** بيمسح فايل الميجريشن، وبيرجع الـ Snapshot خطوة لورا كأنك ماعملتش حاجة.

سيناريو ب: عملت Migration وعملت Update Database خلاص

- هنا مينفعش تعمل Remove-Migration علطول، لأنه هيقولك إن الميجريشن دي اتطبقت على الداتا بيز.
- **الحل:** لازم ترجع الداتا بيز خطوة لورا الأول (Update للي قبلها) وبعدين تعمل Remove.

اللي بيحصل هنا: الـ EF Core هتروح تنفذ ميثود Down () الموجودة في ميجريشن AddSalary، فتشيل العمود من الداتا بيز، وترجع الداتا بيز لحالة InitialCreate

1 Update-Database InitialCreate

دلوقتي الداتا بيز رجعت سليمة، بس لسه عندك ملف الميجريشن (AddSalary.cs) والـ Snapshot لسه فاكدة إن العمود موجود. دلوقتي تقدر بأمان تعمل:

1 Remove-Migration

سيناريو ج (المصيبة): مسحت فايل الميجريشن بإيدك بعد ما عملت Update

- انت عملت Add-Migration (فأل Snapshot اتعدلت)، وعملت Update-Database (فالداتا بيز اتعدلت).
- جيت انت بذكاء مسحت فايل الميجريشن من الـ Solution Explorer بإيدك.
- النتيجة:
- 1. الـ Snapshot لسه شايعة التعديل.
- 2. الداتا بيز شايعة التعديل (وفيها ريكورد في جدول الهيستوري).
- 3. انت ضيعة الفايل اللي بيربطهم ببعض.
- لو جيت تعمل Add-Migration جديد، هيقارن الموديل بالـ Snapshot، فممك مايحسش بفرق، أو لو مسحت الـ Snapshot خالص، هيفتكر إن دي أول مرة تنشئ الداتا بيز.

3. حل مشكلة "Table Already Exists"

لو حصل لخبطة (مسحت الـ Snapshot أو الميجريشنز)، وجيت تعمل ميجريشن جديد، الـ EF Core هتعمل Script فيه Create Table. لما تيجي ترن Update-Database، هيضرب في وشك ويقولك "Table Already Exists" لأن الجداول فعلاً موجودة في الداتا بيز.

الحل السحري (Hack):

1. اعمل Add-Migration عادي عشان يظبط الـ Snapshot على الوضع الحالي للموديل.
2. ادخل جوه فايل الميجريشن اللي لسه طالع.
3. فضي محتوى الـ Up() والـ Down(): امسح الأكواد اللي جواهم.

```
1 protected override void Up(MigrationBuilder migrationBuilder)
2 {
3     // سيب دي فاضية عشان لما يرن مايحاولش يكرت جداول موجودة أصلاً
4 }
```

4. اعمل Update-Database.

5. كده هو هيضيف ريكورد في الداتا بيز إن الميجريشن دي تمت، والـ Snapshot بتاعتك بقت متحدثه، والدنيا رجعت Sync من غير ما يضر.

سؤال وجواب من المحاضرة:

- سؤال: لو مسحت فايل السناپ شوت وجيت أعمل Add-Migration ؟
- الإجابة: الـ EF هتفتكر إن الداتا بيز فاضية وهتحاول تكرت كل الجداول من جديد في فايل الميجريشن، ولو رنيت هيضرب لأن الجداول موجودة أصلاً. الحل إنك تفضي ميثود الـ Up زي ما قلنا في الجزء الأول.

EF Core 5 Model Configuration (Fluent API)

في الجزء ده هنتكلم عن إزاي بنتحكم في شكل الجداول وإزاي بنجيب الداتا المرتبطة ببعضها (Navigation Properties).

1. طرق التحكم في الـ Model (Configuration)

إحنا عندنا 3 طرق عشان نخلي الـ EF تفهم إحنا عايزين الداتا بيز شكلها إيه:

1. الـ **By Convention**: دي الحاجات اللي هي بتفهمها لوحدها من غير ما نتكلم.
2. الـ **Data Annotations**: الأتريبيوتس اللي بنحطها فوق البروبرتي (زي [Key], [Required]).
3. الـ **Fluent API**: وده الكود اللي بنكتبه جوه الـ `OnModelCreating` وده أقوى طريق فيهم.

أ- الـ Conventions (الذكاء الاصطناعي بتاع EF)

- الـ **Primary Key**: أي بروبرتي اسمها `Id` أو `EntityId` (زي `EmployeeId`) الـ EF بتعتبرها أوتوماتيك البرايمري كي وبتخليها `Identity` لو هي `int`.
- الـ **Foreign Key**: لو عندك `DepartmentId` في كلاس الـ `Employee` وعندك نافيجيشن بروبرتي اسمها `Department` ، هي لوحدها بتفهم إن ده الفورين كي من غير ما تشاور عليه.

ب- الـ Fluent API (التحكم الكامل)

دي بنستخدمها جوه ميثود بنعمل لها `override` اسمها `OnModelCreating`.

- ليه نستخدمها؟ في حاجات الـ `Data Annotations` مابتعرفش تعملها ، زي الـ **Composite Primary Key**.
- مثال الـ **Composite Key**: لو عندنا جدول `Attendance` والكي بتاعه عبارة عن `EmployeeId` مع `Date`

```
1 internal class Attendance
2 {
3     public int EmployeeID { get; set; }
4     public DateTime Date { get; set; }
5 }
6
7 internal class Context : DbContext
8 {
9     protected override void OnModelCreating(ModelBuilder modelBuilder)
10    {
11        modelBuilder.Entity<Attendance>()
12            .HasKey(a => new { a.EmployeeId, a.Date });
13        // كده حددنا إن الاتنين مع بعض هما الكي
14    }
15 }
```

- تغيير اسم الجدول أو البروبرتي:

```
1 protected override void OnModelCreating(ModelBuilder modelBuilder)
2 {
3     modelBuilder.Entity<Department>()
```

```
4      .ToTable("MyDepartments") // تغيير اسم الجدول [cite: 55]
5      .Property(d => d.Name)
6      .IsRequired() // جعل الحقل Required [cite: 56]
7      .HasColumnName("DeptName"); // تغيير اسم الكولوم
8  }
9
```

ملحوظة: المهندس بيفضل الـ Data Annotations في الحاجات البسيطة زي `[Required]` عشان بتفيد كمان في الـ Validation، لكن الـ Fluent API هي الأساس في الشغل الثقيل.

EF Core 5 Data Loading (Lazy, Explicit, Eager)

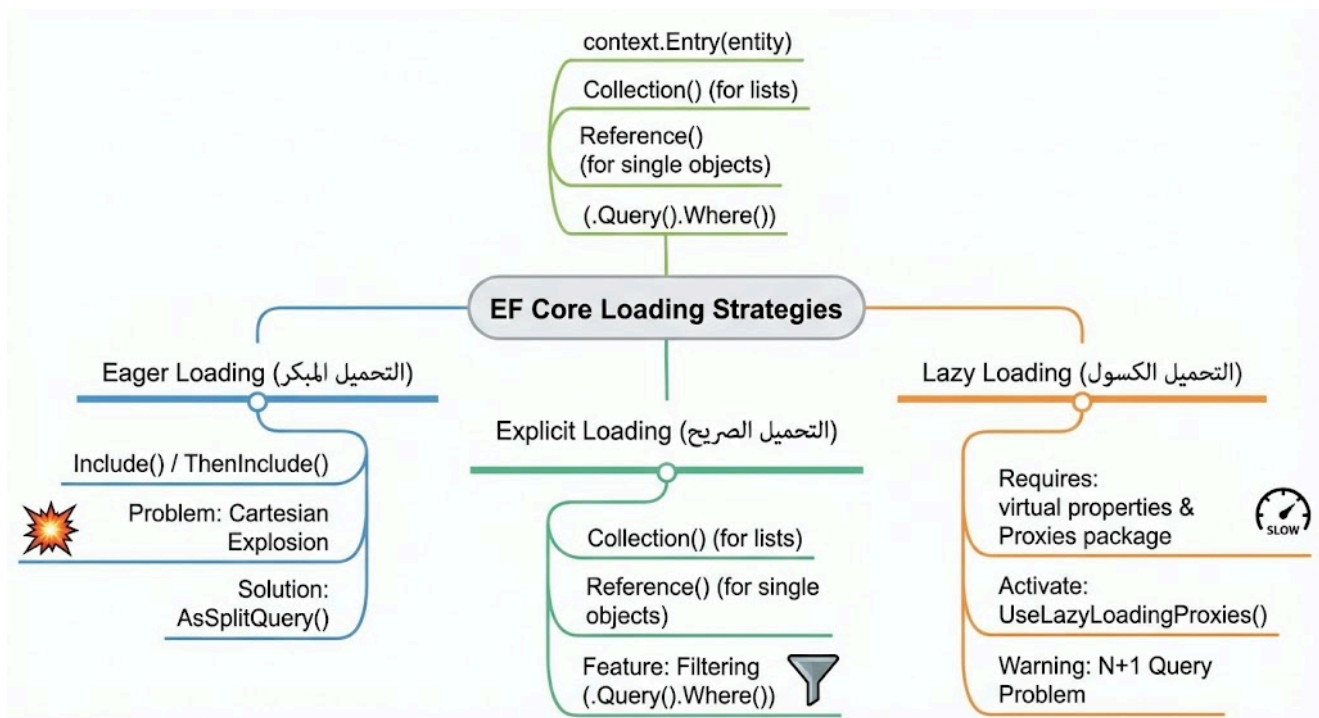
1. قصة الـ Lazy Loading (ليه اتقف؟)

دي أهم معلومة في المحاضرة: الـ Lazy Loading في EF Core مقفول Default. حتى لو عملت البروبرتي virtual مش هيشغل لوحده.

ليه يا مايكروسوفت؟

بسبب الـ Performance. فيه "Control" مشهورة زي الـ DataGridView في الـ Desktop Applications. لما بتديها List من الـ Departments، هي من نفسها (متطوعة) بتروح تحاول تفتح الـ Employees اللي جوه كل Department عشان تعرضهم. لو عندك 100 قسم، الـ DataGridView هتروح تعمل 100 كويري للداتا بيز عشان تجيب الموظفين، وده بيعمل حمل رهيب (N+1 Problem) ويوقع الأداء.

2. بدائل الـ Lazy Loading (إزاي نجيب الداتا؟)



عشان نجيب الداتا المرتبطة (زي الموظفين اللي جوه القسم)، عندنا كذا طريقة، أشهرهم:

أ- الـ Eager Loading (هات كله معاك)

باستخدام ميثود اسمها `.Include()`. وأنا بجيب الأقسام، بقوله "بالمرة هات معاك الموظفين".

مثال كود:

```
1 var depts = context.Departments
2   .Include(d => d.Employees) // هات الموظفين
3   .Include(d => d.Projects)   // وهات كمان المشاريع بتاعة القسم
4   .ToList();
```

ب- الـ (Nested Include ThenInclude)

لو عايز أجيب (القسم) وجواه (الموظفين) وجوه كل موظف أجيب (سجلات الحضور بتاعته):

```
1 var result = context.Departments
2     .Include(d => d.Employees)
3     .ThenInclude(e => e.Attendances) // ThenInclude قبلها بتجيب للي قبلها
4     .ToList();
```

- الـ .Include(): بتشتغل على الكلاس الأساسي (Department).
- الـ .ThenInclude(): بتدخل جوه الكولكشن اللي لسه جايينها (Employees) وتجيب اللي جواها.

3. مشاكل الـ Include وحلها (Cartesian Explosion)

المشكلة: الـ Query اللي "بيجيب مصايب"

كنا وقفنا المرة اللي فاتت عند كويري فيه Include كتير. تعالوا نشوف الكويري ده لما يترجم لـ SQL بيبقى شكله عامل ازاي. لو عندنا Department وعملنا Include للـ Projects و Employees و Attendance .

```
1 // الكود اللي بيسبب المشكلة
2 var result = (context.Departments
3     .Include(d => d.Projects)
4     .Include(d => d.Employees)
5     .ThenInclude(d => d.Attendances)).ToList();
```

لما نبص على الـ SQL اللي رايح (من خلال الـ Logs أو Output Window) ، هنلاقي كارثة:

```
1 // debug جوه كلاس كونتكتست هعمل لوج يظهر في الـ
2 protected override void OnConfiguring(DbContextOptionsBuilder
3     optionsBuilder)
4     {
5         optionsBuilder.UseSqlServer(@"Data source = .\sqlexpress;...");
6         optionsBuilder.LogTo(log => Debug.WriteLine(log));
7         base.OnConfiguring(optionsBuilder);
8     }
```

```

Output
Show output from: Debug
SELECT [d].[ID], [d].[BranchID], [d].[Name], [p].[ID], [p].[DepartmentID], [p].[Name], [t].[ID], [t].[DepartmentID], [t].[
FROM [Department] AS [d]
LEFT JOIN [Project] AS [p] ON [d].[ID] = [p].[DepartmentID]
LEFT JOIN (
    SELECT [e].[ID], [e].[DepartmentID], [e].[Name], [a].[EmployeeID], [a].[Date]
    FROM [Employees] AS [e]
    LEFT JOIN [Attendance] AS [a] ON [e].[ID] = [a].[EmployeeID]
) AS [t] ON [d].[ID] = [t].[DepartmentID]
ORDER BY [d].[ID], [p].[ID], [t].[ID], [t].[EmployeeID], [t].[Date]

```

مشكلة الـ Join في EF Core (Cartesian Explosion)

الـ Entity Framework لما بتطلب منه يعمل Include لحاجات كثير (Collections) مع بعض، بيحاول ينفذ ده بذكاء "غبى" شوية في الـ SQL.

يعني إيه المشكلة؟

لما بتعمل Include لـ Department ومعه Employees ومعه كمان Projects ومعه Attendances ، الـ EF بيترجم ده لـ SQL Query واحد بس بيستخدم فيه LEFT JOIN .

الـ SQL بطبيعته بيرجع جدول "مسطح" (Flat Table)، يعني مفيش حاجة اسمها "ليستة جوه سطر". عشان يربط الموظفين بالمشاريع في نفس الجدول، بيضطر يعمل عملية ضرب (Multiplication) لعدد الصفوف.

مثال بالأرقام (عشان نتخيل الكارثة)

تعال نفترض داتا صغيرة جداً عشان نفهم اللي بيحصل:

- عندنا 1 قسم (Department = IT).
- القسم ده فيه 3 موظفين (Employee = علي، منى، أحمد).
- القسم ده ماسك 2 مشروع (Project = ERP، Mobile App).

أ) لو عملنا Join بين Department و Employees بس:

النتيجة هتكون منطقية (3 صفوف)، صف لكل موظف، واسم القسم هيتكرر.

Department	Employee
IT	علي
IT	منى
IT	أحمد

ب) الكارثة: لو زدنا Projects في نفس الـ Join:

هنا بيحصل الـ Cartesian Product. عشان الداتا بيترجع كل موظف مع كل مشروع في جدول واحد، لازم تضرب عدد الموظفين في عدد المشاريع.

الحسبة: 3 Employees × 2 Projects = 6 Rows

Department	Employee	Project	ملاحظة
IT	علي	ERP	(علي اكرر)
IT	علي	Mobile App	(علي اكرر ثاني)
IT	منى	ERP	(منى اكررت)
IT	منى	Mobile App	(منى اكررت ثاني)
IT	أحمد	ERP	(أحمد اكرر)
IT	أحمد	Mobile App	(أحمد اكرر ثاني)

لاحظت إيه؟

بيانات الموظف "علي" اكررت مرتين عشان نربطه مرة بمشروع الـ ERP ومرة بمشروع الـ App. ده اسمه "تكرار بيانات" (Data Duplication).

مثال المحاضرة (الأرقام المربعة)

المهندس كبر المثال عشان يوريك حجم "المصيبة" لو الداتا كترت شوية:

لو عندك:

- 10 أقسام (Departments).
- 10 مشاريع (Projects).
- 100 موظف (Employees).
- 1000 سجل حضور (Attendances).

لو عملت Include لكل دول في كويري واحد، الـ SQL هيعمل عملية ضرب:

$$10 \times 10 \times 100 \times 1,000 = 10,000,000$$

النتيجة: 10 مليون سطر راجعين من الداتابيز!

ليه ده "أسوأ حاجة في الدنيا"؟

1. الـ **Network Traffic**: بتنقل 10 مليون سطر على الشبكة، وفيهم بيانات مكررة (اسم القسم متكرر 10 مليون مرة!).
2. الـ **Memory & Processing**: الـ Application بتاعك بيستلم الـ 10 مليون سطر دول، ويقعد يفلترهم ويركبهم ثاني عشان يطلعلك في الآخر 20 أو 30 أوبيجكت بس في الميموري. ده بيقتل الـ رامات والـ CPU.
3. الـ **Database Performance**: الداتابيز نفسها بتعاني عشان تعمل الـ Join الضخم ده.

الحل: AsSplitQuery

عشان نحل المصيبة دي، الـ EF Core قدمت لنا حل سحري اسمه AsSplitQuery .

الفكرة: بدل ما يعمل Join ضخيم، بيقسم الكويري لعدة كويريز صغيرة:

1. كويري يجب الـ Departments.
2. كويري يجب الـ Projects الخاصة بيهم.
3. كويري يجب الـ Employees الخاصة بيهم... وهكذا.

```
1 // الحل باستخدام Split Query
2 var result = (context.Departments.AsSplitQuery() // السطر السحري
3     .Include(d => d.Projects)
4     .Include(d => d.Employees)
5     .ThenInclude(d => d.Attendances)).ToList();
```

النتيجة:

بدل ما يرجعك 10 مليون سطر مكررين، هيرجعك حوالي 1200 سطر بس، والـ EF يركبهم في الميموري من غير تكرار ومشاكل.

```
1 SELECT * FROM Departments;
2 -- النتيجة: 10 سطور (لأن عندنا 10 أقسام)
3
4 SELECT * FROM Projects WHERE DepartmentId IN (1, 2, ... 10);
5 -- النتيجة: 100 سطر (10 أقسام × 10 مشاريع)
6
7 SELECT * FROM Employees WHERE DepartmentId IN (1, 2, ... 10);
8 -- النتيجة: 1000 سطر (10 أقسام × 100 موظف)
9
10 --etc...
```

إزاي الـ EF بيركبهم في الميموري؟ Relational Fixup Stage

العملية دي اسمها **Relational Fix-up** وبتحصل جوه الميموري (RAM) عندك في الـ Application مش في الداتايز.

1. **الخطوة الأولى:** الـ EF بيستلم الـ 10 أقسام، ويعمل لهم 10 Objects في الميموري. (كل أوبجيكت له Id مميز).
2. **الخطوة الثانية:** الـ EF بيستلم الـ 100 مشروع. كل مشروع جاي معاه DepartmentId (الـ Foreign Key). الـ EF بيبيص في الميموري: "يا ترى فين القسم اللي الـ Id بتاعه كذا؟" .. أهو! يقوم واخذ المشروع وحاطه جوه الـ List اللي اسمها Projects في القسم ده.
3. **الخطوة الثالثة:** نفس الكلام مع الموظفين. كل موظف بيبيجي معاه DepartmentId ، الـ EF يدور على الأوبجيكت بتاع القسم في الميموري، ويضيف الموظف ده في الـ List اللي اسمها Employees .

النتيجة النهائية: في الآخر خالص، اللي بيرجعك في المتغير بتاعك هو الـ 10 أقسام، وكل قسم جواه الداتا بتاعته مترتبة صح، من غير ما نكون نقلنا ملايين السطور المكررة من الداتايز.

📌 ملحوظة مهمة استخدام AsSplitQuery هنا أفضل بكثير في الـ Performance مقارنة بالـ Single Query (الـ JOIN الضخم) في حالات الـ One-to-Many Collections الكبيرة.

4. أنواع الـ Loading الأخرى

احنا عرفنا الـ Eager Loading (باستخدام Include). تعالوا نشوف النوعين التانيين.

أ- Explicit Loading (التحميل الصريح)

ده معناه إنك معاك الـ Entity الأساسية (مثلاً Department)، وعاليز تحمل الـ Related Data بتاعتها (زي Employees) في وقت لاحق (مش في نفس الكويري).

ازاي بنعمله؟ عن طريق الـ Context.Entry :

- لو عاليز تحمل Collection (زي Employees):

```
1 var dept = context.Departments.First(); // جينا القسم بس
2
3 // دلوقتى عاليز أحمل الموظفين بتوعه
4 context.Entry(dept)
5     .Collection(d => d.Employees) // بحدد الكولكشن
6     .Load();
```

```
info: 2/28/2022 10:55:51.453 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (173ms) [Parameters=[@__p_0='?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT [e].[ID], [e].[DepartmentID], [e].[Name]
      FROM [Employees] AS [e]
      WHERE [e].[DepartmentID] = @__p_0
```

النتيجة: بيعمل كويري منفصل يجيب الموظفين اللي تبع القسم ده.

مثال آخر

```
1
2 //SELECT [Id], [Name], ... FROM [Departments]
3 var query = context.Departments.ToList();
4
5 foreach (var dept in query)
6 {
7     //SELECT [Id], [Name], [DepartmentId], ...
8     //FROM [Employees]
9     //WHERE [DepartmentId] = @CurrentDeptId
10    // في كل لفه
11    context.Entry(dept).Collection(d => d.Employees).Load();
12
13    foreach (var emp in dept.Employees)
14    {
15        Console.WriteLine(emp.Name);
16    }
17
18    Console.WriteLine(dept.Name);
19 }
```

⚠ : استخدام Load() داخل foreach يرسل استعلاماً منفصلاً لكل عنصر (مثال: 100 قسم = 101 استعلام)، مما يقتل الأداء.

💡 الحل: استخدم Eager Loading بـ Include(d => d.Employees). في الاستعلام الرئيسي خارج اللوب.

النتيجة: يتم جلب البيانات المرتبطة في استعلام واحد (Single Query) بدلاً من مئات الاستعلامات.

• لو عايز تحمل Reference (زي Branch جوه Department):

```
1 context.Entry(dept)
2     .Reference(d => d.Branch) // يحدد الريفرنس
3     .Load();
```

ليه استخدم .Reference() مع Branch؟

لأن العلاقة هنا **One-to-One** (أو **Many-to-One** من ناحية القسم). القسم الواحد (Department) مربوط بفرع واحد (Branch). يعني الـ Property في الكلاس عبارة عن **Object** واحد مش قائمة.

📌 الخلاصة (قاعدة عامة):

- لو الـ Property عبارة عن **List** (حاجات كتير) ← استخدم .Collection().
- لو الـ Property عبارة عن **Object** (حاجة واحدة) ← استخدم .Reference().

ميزة قوية جداً (Filtering inside Explicit Loading)

تقدر تعمل فلتريشن على الداتا اللي لسه هتيجي قبل ما تعملها Load!! مثلاً: عايز الموظفين بتوع القسم ده بس اللي الـ ID بتاعهم أقل من 10.

```
1 context.Entry(dept)
2     .Collection(d => d.Employees)
3     .Query() // حولناها لكويري
4     .Where(e => e.Id < 10)
5     .Load() // أو .ToList();
```

🔗 تريك لو استخدمت .Query() تقدر تكمل LINQ عادي خالص وكأنك بتعمل كويري منفصل.

ب- Lazy Loading (التحميل الكسول)

ده الأسلوب القديم بتاع زمان، اللي هو "لما أحتاج الحاجة أروح أجيبها". الـ EF Core مش بتشغله by default، لازم إعدادات خاصة.

1. الـ **Virtual Properties**: لازم كل الـ Navigation Properties تكون `virtual`.

```
1 public virtual ICollection<Employee> Employees { get; set; }
2 public virtual Branch Branch { get; set; }
```



2. الـ **Install Package**: نزل باكج `Microsoft.EntityFrameworkCore.Proxies`.

3. الـ **Configuration**: في الـ `OnConfiguring` فعل البروكسي:

```
1 optionsBuilder.UseLazyLoadingProxies();
```

امتي نستخدمه؟ لو أنت واثق 100% إنه مش هيعمل مشاكل الـ Performance (مشاكل الـ N+1 Query) أو مشاكل الـ Serialization مع حاجات زي الـ APIs أو WPF DataGrid.

⚠ تحذير الـ **Lazy Loading** ممكن يدمر الـ **Performance** لو استخدمته غلط (مثلاً في لوب)، أو يسبب **Infinite Loop** في الـ **JSON Serialization** لو مش واخد بالك.

Evaluation, Tracking, Global Query

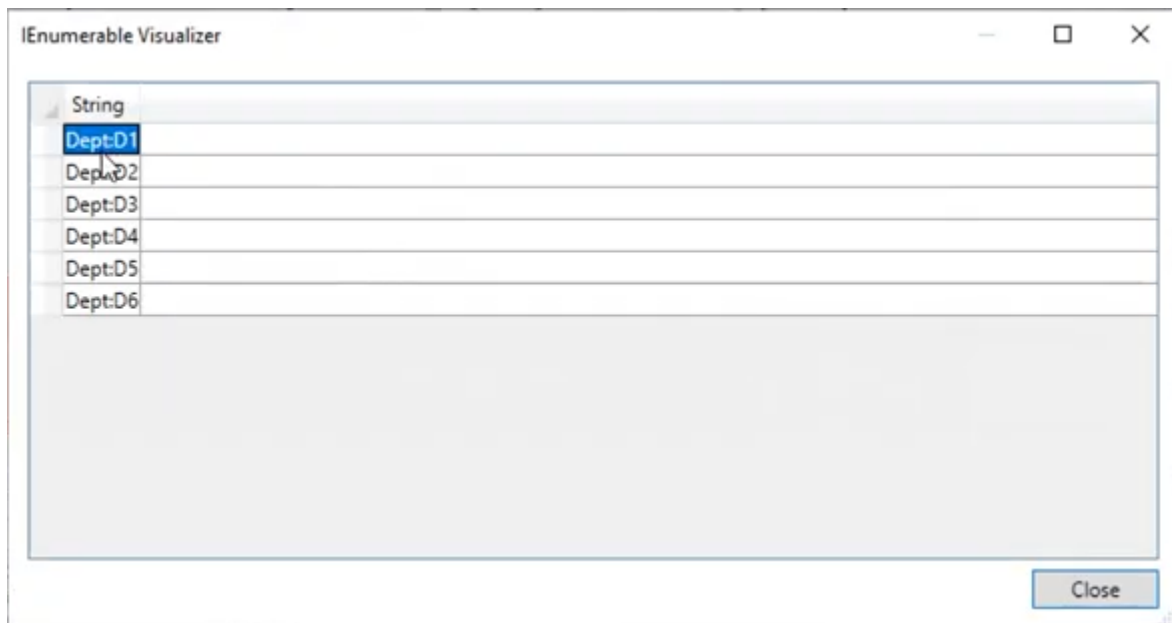
Client Evaluation vs. Server Evaluation 5. الـ

EF Core (خاصة)، الـ C# أو دوال String.Join زي (SQL ملهاش مقابل في الـ LINQ لو كتبت دالة في الـ EF Core بي تعامل بذكاء.

- في EF 6 (القديم): كان بيرمي Exception فوراً.
- في EF Core:
- لو الدالة دي موجودة في جزء (Projection) **Select**: هيعمل كويري يجيب الداتا من الـ DB، وبعدين ينفذ الدالة دي في الـ Memory (Client Side).
- لو الدالة دي في (Filtering) **Where**: هيرمي Exception لأنه لازم يفلتر في الداتا بيز ومش عارف يترجم الدالة.

مثال:

```
1 // ده يشتغل عادي (Client Evaluation)
2 Context context = new Context();
3 var query =
4     (from d in context.Departments
5      select string.Join(':', "Dept", d.Name)).ToList();
6 // يحصل في الميموري Concatenation هيجيب البيانات الخام، والـ SQL الـ
```



```
1
2 // ده يضرب Exception
3 var q = context.Departments
4     (from d in context.Departments
5      // خطأ! مش بيترجم لـ SQL
6      .where string.Join(':', "Dept", d.Name) == d.Name
```

6. EF.Functions دوال

لو محتاج تستخدم دوال SQL صريحة (زي LIKE بدل Contains)، مايكروسوفت عملت كلاس اسمه EF.Functions.

مثال (LIKE):

```
1 var result = context.Departments
2   .Where(d => EF.Functions.Like(d.Name, "%M%")) // LIKE بتترجم لـ
3   .ToList();
```

ده مفيد لو عايز تستخدم Patterns معينة في البحث مش بتوفرها Contains العادية.

7. الـ Tracking Strategies (NoTracking & Identity Resolution)

الـ AsNoTracking

لو بتعمل كويري عشان تقرأ بيانات بس (Read-Only) ومش ناوي تعدل فيها، استخدم AsNoTracking().

- الميزة: أسرع وأخف في الميموري (الـ Context مش بيعمل Snapshot ولا بيراقب التغييرات).
- إعداد عام: ممكن تخلي الـ Context كله NoTracking من الـ Constructor:

```
1 Public Context()
2 {
3     this.ChangeTracker.QueryTrackingBehavior =
4         QueryTrackingBehavior.NoTracking;
5 }
6
```

بعض مشاكل الـ AsNoTracking مشكلة تكرار الأوبجيكس (Identity Resolution)

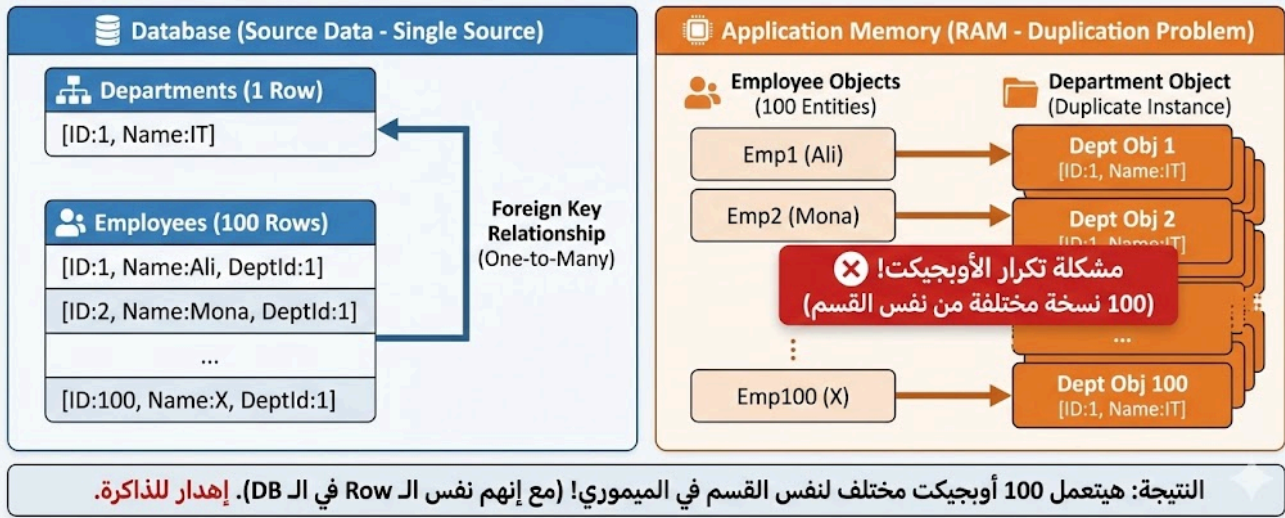
لو استخدمت AsNoTracking مع Include، بيحصل تكرار للأوبجيكس في الميموري.

- السيناريو: لو جبت الموظفين وعملت Include للقسم، وفيه 100 موظف في نفس القسم.
- النتيجة: هيتعمل 100 أوبجيكس مختلف لنفس القسم في الميموري! (مع إنهم نفس الـ Row في الـ DB).

EF Core: Identity Resolution Problem with AsNoTracking

السيناريو: لو جيت الموظفين وعملت Include للقسم، وفيه 100 موظف في نفس القسم.

```
context.Employees.AsNoTracking().Include(e => e.Department).ToList()
```



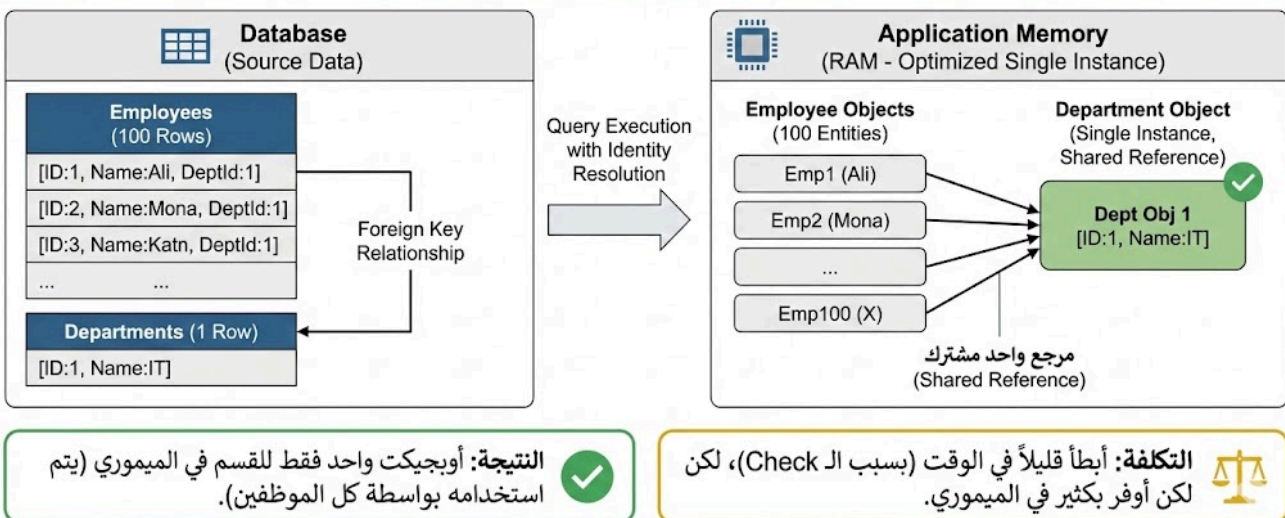
الحل: AsNoTrackingWithIdentityResolution

عشان تحل المشكلة دي وتوفر ميموري، استخدم:

```
1 var result = context.Employees
2   .AsNoTrackingWithIdentityResolution() // الحل هنا
3   .Include(e => e.Department)
4   .ToList();
```

الحل: AsNoTrackingWithIdentityResolution (لحل مشكلة تكرار الأوبجكت وتوفير الميموري)

```
var result = context.Employees.AsNoTrackingWithIdentityResolution().Include(e => e.Department).ToList();
```



- بيعمل إيه؟ بيتشيك هل الأوبجكت ده جه قبل كده ولا لا، لو جه بيشاور عليه بدل ما يكرت واحد جديد.
- التكلفة: أبطأ شوية في الوقت (عشان ال Check)، بس أوفر بكثير في الميموري.

```

1 // 1. الكويري بتاعنا
2 var allEmployees = context.Employees
3     .AsNoTrackingWithIdentityResolution()
4     .Include(e => e.Department)
5     .ToList();
6
7 // 2. تعال نمسك أول موظف، ونأخذ منه نسخة القسم
8 var employee1 = allEmployees[0];
9 var sharedDept = employee1.Department; // (IT) ده القسم المشترك
10
11 // 3. المفاجأة: لو دخلت جوه القسم ده، هتلاقي الليستة مليانة
12 // IT هتلاقي هنا كل الموظفين اللي تم تحميلهم في الكويري ونوعهم
13 foreach (var emp in sharedDept.Employees)
14 {
15     Console.WriteLine(emp.Name);
16 }

```

حصل إزاي روح راجع الـ Relational Fixup Stage

8. الـ Global Query Filters (Soft Delete)

لو عندنا كولوم `IsDeleted` وعابزين نخفي أي حاجة ممسوحة من كل الكويريز في السيستم من غير ما نكتب `Where(!IsDeleted)` كل مرة.

الحل: `HasQueryFilter` في الـ `OnModelCreating`.

```
1 modelBuilder.Entity<Employee>().HasQueryFilter(e => !e.IsDeleted);
```

- النتيجة: أي كويري هيتعمل على الـ `Employee`، الـ `EF Core` هيزود عليه أوتوماتيك `WHERE IsDeleted = 0`.

9. الـ Shadow Properties

يعني إيه **Shadow Property**؟ يعني خاصية (Column) موجودة في الداتابيز بس مش موجودة في الـ `C# Class` بتاعك. زي مثلاً: `IsDeleted` , `LastUpdatedBy` , `CreatedDate` (لو مش عابزها تظهر في الموديل).

تعريفها

```

1 modelBuilder.Entity<Department>()
2     .Property<bool>("IsDeleted"); // IsDeleted بعرف بروبرتى اسمها

```

الوصول إليها (Access & Update)

بما إنها مش في الكلاس، مش هتعرف تعمل dept.IsDeleted . بتوصلها كده:

- عشان تعدل قيمتها:

```
1 context.Entry(dept).Property("IsDeleted").CurrentValue = true;
```

- عشان تستخدمها في LINQ Query:

```
1 // استخدام EF.Property
2 var deletedDepts = context.Departments
3     .Where(d => EF.Property<bool>(d, "IsDeleted") == true)
4     .ToList();
```

10. الـ Bulk Configuration (تطبيق على كل الجداول)

لو عايز تطبق الـ Shadow Property (زي IsDeleted) أو الـ Query Filter على كل الجداول مرة واحدة بدل ما تكتبهم واحد واحد.

بنستخدم لوب على modelBuilder.Model.GetEntityTypes():

```
1 foreach (var entityType in modelBuilder.Model.GetEntityTypes())
2 {
3     // 1. إضافة Shadow Property لكل الجداول
4     modelBuilder.Entity(entityType.Name).Property<bool>("IsDeleted");
5
6     // 2. ديناميكياً Global Query Filter ممكن هنا برضه تطبق
7     // Expressions متقدم شوية أو Reflection محتاجة كود
8 }
```

11. واجب ومواضيع للقراءة (Homework)

- الـ Relational Fix-up: اقرأ في الكتاب (Chapter 6 أو 7) عن موضوع الـ Relational Fix-up وازاي الـ EF يربط العلاقات ببعضها في الميموري، وتأثير الـ NoTracking عليها.
- الـ Recursion: لو عندك موظف ومدير (المدير هو موظف برضه)، ازاى تحمل الشجرة كاملة (الموظف ومديره ومدير مديره...)? دور على الـ Recursive queries أو الـ approach المناسب.
- الـ Database Mapping: محتاجين ندور على كيفية عمل الـ Map لـ Views, Stored Procedures, و Scalar Functions.

نصيحة أخيرة: "محدث بي فهم حاجه غير لما يقرأ... ومهما بلغت من العلم، كل ما تقرا بتكتشف حاجات زياده". شغلانتنا محتاجة مذاكرة وتجربة مستمرة.

Handling Stored Procedures

الـ EF Core شاطرة جداً في إنها تحول الـ Entity Classes لجداول وعلاقات، لكن لما نيجي نتكلم عن حاجات Database-Specific زي الـ **Stored Procedures** أو الـ **Views**، هنا الـ EF Core بتترفع إيدها وتقولك "أنا معرفش الكلام ده، اتصرف انت".

عشان نعمل الكلام ده، الكتاب طرح طريقتين أساسيتين:

1. **التعديل اليدوي لملف الـ Migration** (باستخدام `MigrationBuilder.Sql`).
2. **استخدام SQL Scripts خارجية** (Handcoded SQL).

تعال نمسكهم واحدة واحدة.

1 الطريقة الأولى: التعديل اليدوي (Editing Migration Class)

دي الطريقة اللي بنستخدم فيها أدوات الـ EF Core العادية بس بنتدخل فيها يدوياً. الفكرة ببساطة إننا بنعمل Migration فاضية (أو فيها تعديلات تانية)، وبعدين نفتح ملف الـ `cs` اللي اتكون ونكتب فيه كود SQL صريح.

✂ إزاي بنعملها؟ (Steps)

1. إنشاء الـ **Migration**: بنعمل `Add-Migration` عادي جداً.
2. **فتح الملف**: بتفتح ملف الـ Migration اللي اتعمل (اللي بيتنهي بـ `cs`).
3. **كتابة الـ SQL**: بنستخدم ميثود اسمها `migrationBuilder.Sql` عشان نمرر كود الـ SQL بتاع الـ **Stored Procedure**.

💡 مثال عملي (Code Example)

تخيل إننا عايزين نعمل **Stored Procedure** بتجيب اليوزرز. الكود في ملف الـ Migration هيكون بالشكل ده:

```
1 public partial class AddGetUsersProcedure : Migration
2 {
3     protected override void Up(MigrationBuilder migrationBuilder)
4     {
5         // عشان نكريت الـ SQL هنا بنكتب كود الـ
6         // migrationBuilder.Sql لاحظ إننا بنستخدم
7         migrationBuilder.Sql(@"
8             CREATE PROCEDURE GetUsers
9             AS
10             BEGIN
11                 SELECT * FROM Users
12             END
13         ");
14     }
15 }
```

```

16     protected override void Down(MigrationBuilder migrationBuilder)
17     {
18         // Migration ومهم جداً مننساها نمسحها لو حيننا نرجع في الـ
19         migrationBuilder.Sql("DROP PROCEDURE GetUsers");
20     }
21 }

```

المبدأ ده تم شرحه في الكتاب بالتطبيق على SQL Commands بشكل عام في.

2 الطريقة الثانية: استخدام Extension Methods (التنظيم)

الكتاب بيقولك لو إنت راجل "حريف" وبتستخدم الـ Custom SQL ده كتير (سواء Views أو Stored Procedures)، مش منطقي تقعد تكتب migrationBuilder.Sql كل مرة وتكتب الـ SQL Strings بايدك. الأفضل تعمل Extension Method تسهل عليك الدنيا.

الكتاب ضرب مثال بإنشاء View ، بس نفس المنطق بيطبق على الـ Stored Procedures.

فكرة الـ Extension Method

بدل ما تكتب SQL كل مرة، بتعمل ميثود بتاخذ منك اسم الـ Procedure والباراميترز، وهي تكون الـ SQL String وتنفذه.

مثال من الكتاب (Listing 9.6) بتصرف بسيط لتوضيح الفكرة:

```

1     public static class MigrationExtensions
2     {
3         // الـ Views دي ميثود احنا اللي عاملينها عشان نسهل إنشاء الـ
4         public static void AddViewViaSql<TView>(
5             this MigrationBuilder migrationBuilder,
6             string viewName,
7             string tableName,
8             string whereSql)
9             where TView : class
10        {
11            // مثلاً SQL Server التأكد إننا شغالين على 1.
12            if (!migrationBuilder.IsSqlServer())
13                throw new NotImplementedException("Warning: This only works
with SQL Server");
14
15            // 2. (Reflection) تجميع أسماء الأعمدة من الكلاس
16            var selectNamesString = string.Join(", ",
17                typeof(TView).GetProperties().Select(x => x.Name));
18
19            // 3. SQL تكوين جملة الـ
20            var sqlCommand = $"CREATE OR ALTER VIEW {viewName} AS " +
21                $"SELECT {selectNamesString} FROM {tableName} "

```

```

22         $"WHERE {whereSql}";
23
24         // 4. التنفيذ
25         migrationBuilder.Sql(sqlCommand);
26     }
27 }

```

ملحوظة: الكود اللي فوق ده لـ View، بس تقدر تعمله بنفس الشكل لـ CREATE PROCEDURE ، وده بيخلي كود الـ Migration بتاعك أنضف بكثير.

3 الطريقة الثالثة: استخدام SQL Change Scripts (Handcoded)

لو الموضوع معقد أوي، أو إنت بتفضل تكتب SQL بإيدك وتتحكم في كل فتقوتة، الكتاب بيقتراح طريقة **Handcoded SQL Change Scripts**.

مميزات الطريقة دي:

- **تحكم كامل:** تقدر تكتب Stored Procedures معقدة، وتستخدم مميزات قاعدة البيانات اللي EF Core متعرفش عنها حاجة.
- **فصل المسؤوليات:** المطور اللي شاطر في الـ SQL يكتب السكريبت، والـ Backend Developer يركز في الكود.

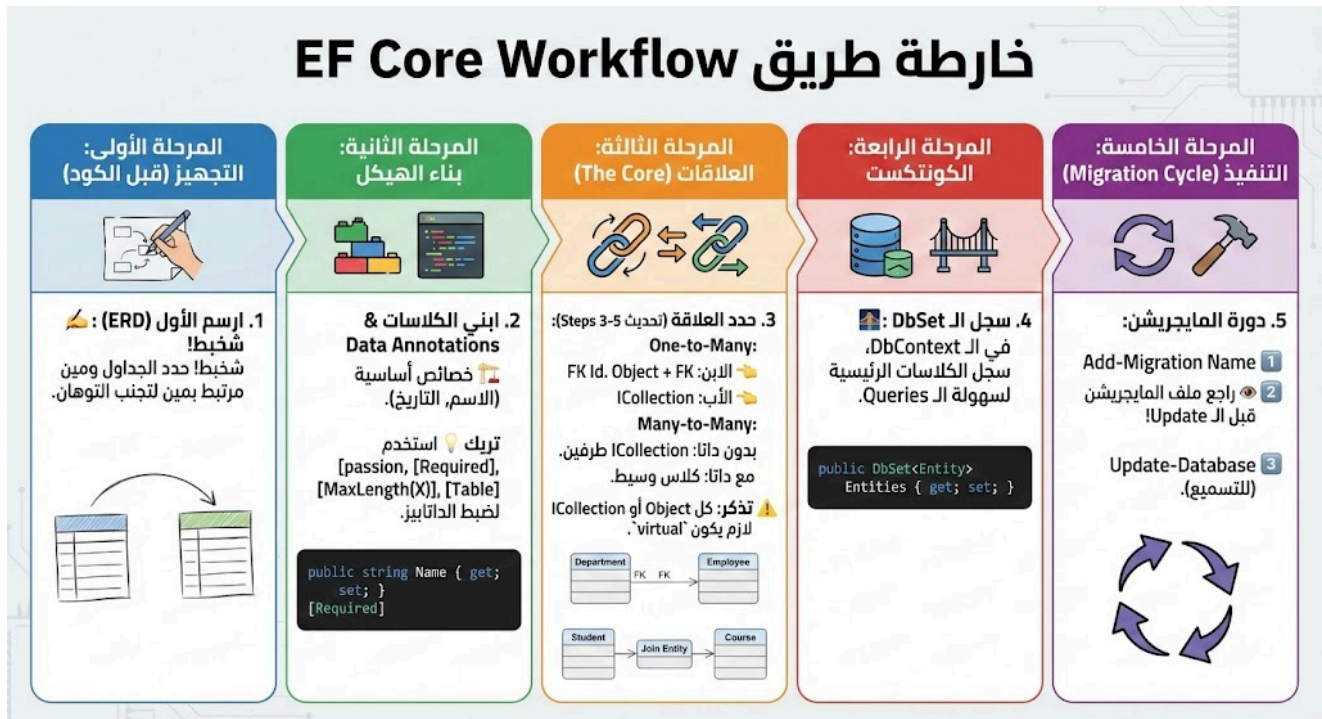
⚠ عيوبها:

- لازم تكون بتعرف تكتب SQL كويس.
- مغيش Down script أوتوماتيك، لازم تكتب سكريبت تاني عشان تمسح الـ Procedure لو حبيت ترجع.

🔍 الخلاصة والنصائح (Tips from the Author)

1. **مكان الكود:** لما تستخدم migrationBuilder.Sql ، خلي بالك من الترتيب. لو الـ Procedure بتعتمد على جدول معين، لازم كود الـ SQL يتحط **بعد** كود إنشاء الجدول في دالة الـ Up .
2. **الـ Views والـ SPs:** التعامل معاهم واحد في الـ Migrations، الاتنين بيعتبروا Custom SQL ولازم يضافوا يدويًا أو عبر سكريبتات.
3. **نصيحة شخصية من المؤلف:** الطريقة اليدوية (تعديل ملف الـ Migration) ممتازة للتعديلات الصغيرة. لكن لو بتعمل تغييرات ضخمة ومعقدة في الـ SQL، الأفضل تتجه لـ SQL Scripts منفصلة.

How to Start your Projects with Code First



المرحلة الأولى: التجهيز (قبل الكود)

1. ارسم الأول (ERD): قبل ما تكتب كلاس، شخبط على ورقة. حدد مين الجداول ومين مرتبط بمين. دي أهم خطوة عشان متوهش في العلاقات.

المرحلة الثانية: بناء الهيكل (Classes & Scalar Properties)

2. ابني الكلاسات بالـ Properties العادية: ابدأ بكتابة الـ Properties الأساسية (الاسم، التاريخ، السعر) لكل الكلاسات.

- تريك: استخدم Data Annotations فوراً.
- الـ Strings خليها [MaxLength(X)] بدل ما تبقى nvarchar(max).
- الحاجات المهمة ادبها [Required] عشان تمنع الـ Nulls.
- لو اسم الجدول مش عاجبك، استخدم [Table].

المرحلة الثالثة: العلاقات (The Core)

3. حدد العلاقة وامش بالقاعدة دي:

- لو One-to-Many:
• عند الـ Many (الابن): حط Foreign Key + Object (مثال: Department + DeptId).
- ليه نخط FK صريح؟ عشان لو عايز تعرف رقم القسم بس من غير ما تجيب الداتا كلها (Lazy Loading)، وعشان الـ Update يبقى أسهل.

- عند الـ **One** (الأب): `ICollection` .
- لو **Many-to-Many**:
- بدون *داتا زيادة*: `ICollection` عند الطرفين، والـ `EF` يعمل الجدول الوسيط لوحده.
- فيه *داتا زيادة (ساعات، تاريخ)*: لازم تعمل **كلاس ثالث (Join Entity)** بإيدك، وتخلي العلاقة **One-to-Many** مع الطرفين.

4. لا تنس كلمة **virtual**: أي `Object` أو `ICollection` لازم يكون `virtual` عشان تفعل الـ **Lazy Loading**.

5. استخدم `[ForeignKey]` **بذكاء**: لو سميت الـ `FK` اسم غريب (مش `Id` ولا `ClassId`)، لازم تستخدم `[ForeignKey("Name")]` فوق الـ `Navigation Property` عشان تربطهم ببعض.

6. الـ `[InverseProperty]`: دي ممتازة، استخدمها فوراً لو لقيت كلاسين بينهم أكثر من سهم رايع جاي يعني أكثر من علاقة (زي مثال المدرسة والتحويلات).

المرحلة الرابعة: الكونتيكست (Connecting to DB)

7. **سجل الـ DbSet**: مش لازم تسجل كل الكلاسات، بس سجل الكلاسات "الرئيسية" اللي عايز تعمل عليها `Queries` مباشرة. الباقي الـ `EF` هيشوفه بـ `"Reachability"`، بس الأفضل تسجلهم عشان سهولة الكود.

المرحلة الخامسة: التنفيذ (Migration)

8. **دورة حياة الـ Migration**:

- الـ `Add-Migration Name`: عشان تترجم الكلاسات لكود `#C`.
- **مراجعة ملف المايجريشن**: (مهمة جداً) بص عليه قبل ما تعمل `Update`، ممكن تلاقي عمود `Required` وأنت عندك داتا قديمة وتعمل مشكلة.
- الـ `Update-Database`: للتسميع في السيكوال.

Inheritance Mapping (TPH & TPT)

في العادي، كل **Entity Class** عندنا بيقابل جدول واحد في الداتابيز (1-mapping-to-1). بس ساعات بنحتاج نغير القاعدة دي، خصوصاً لما نكون بنستخدم الـ **Inheritance** (الوراثة) في الكود بتاعنا.

النهاردة هنركز على طريقتين للتعامل مع الوراثة دي:

1. الـ **Table Per Hierarchy (TPH)**: جدول واحد لكل الوراثة.

2. الـ **Table Per Type (TPT)**: جدول لكل كلاس منفصل.

8.9.2 الـ Table Per Hierarchy (TPH)

معناها **Placing inherited classes into one table**

النوع ده هو الـ Default في EF Core. الفكرة هنا إننا بنخزن كل الـ **Classes** اللي بتورث من بعض في **جدول واحد** بس في الداتابيز.

إمتى نستخدم TPH؟

- لما يكون عندك بيانات متشابهة جداً بين الـ **Classes** وبعضها، والفروقات بسيطة.
- مثال: لو عندنا نظام دفع (**Payment**). الدفع ممكن يكون كاش (**Cash**) أو فيزا (**Card**). الاثنين فيهم "المبلغ" (**Amount**)، بس الفيزا بتزيد عليها حاجة زي "رقم الإيصال" (**Receipt**).
- بدل ما نعمل جدول للكاش وجدول للفيزا، بنعمل جدول واحد اسمه **Payments** ونحط فيه كله.

1. إعداد TPH بـ (By Convention)

لو مشينا بالطريقة التقليدية، هنعمل الكلاسات بتاعتنا عادي جداً:

الكود (C# Classes):

```
1 // الكلاس الأساسي (وممكن يكون فيه الخصائص المشتركة)
2 public class PaymentCash
3 {
4     public int PaymentId { get; set; }
5     public decimal Amount { get; set; }
6 }
7
8 // الكلاس اللي بيورث منه وبيزود خاصية
9 public class PaymentCard : PaymentCash
10 {
11     public string ReceiptCode { get; set; }
12 }
```

الكود (DbContext):

في ال DbContext، هنعرف DbSet لكل نوع:

```
1 public class Chapter08DbContext : DbContext
2 {
3     // ...
4     public DbSet<PaymentCash> CashPayments { get; set; }
5     public DbSet<PaymentCard> CreditPayments { get; set; }
6
7     // ...
8 }
```

إزاي EF Core بيترجم ده لـ SQL؟

ال EF Core ذكي، هيفهم إن PaymentCard وارث من PaymentCash ، فهيعمل جدول واحد بس. بس هيضيف عمود سحري من عنده اسمه **Discriminator** (المميز).

EF Core TPH: Single Table for Inheritance Hierarchy (e.g., CashPayments)

[PK] PaymentId (int)	Amount (decimal)	[DISCRIMINATOR] Discriminator (nvarchar)	[NULLABLE] ReceiptCode (nvarchar)
1	100.00	'PaymentCash'	NULL
2	250.50	'PaymentCard'	'RCPT-12345'
3	50.00	'PaymentCash'	NULL

EF Core "Magic Column"
to distinguish types
(e.g., Cash vs. Card)

Subclass-specific property.
Must be NULLABLE for
base type rows (e.g., Cash).

الكود (SQL الناتج):

```
1 CREATE TABLE [CashPayments] (
2     [PaymentId] int NOT NULL IDENTITY,
3     [Amount] decimal(18, 2) NOT NULL,
4
5     -- ضافه عشان يفرق ده كاش ولا كارد EF Core
6     [Discriminator] nvarchar(max) NOT NULL,
7
8     -- Nullable عشان كده هو Card، العمود ده موجود بس في ال-
9     -- NULL لو الدفع كاش، العمود ده هيبقى
10    [ReceiptCode] nvarchar(max) NULL,
11
12    CONSTRAINT [PK_CashPayments] PRIMARY KEY ([PaymentId])
```

ملحوظة مهمة: أي خاصية موجودة في الـ (ReceiptCode زي) ومش موجودة في الأب، لازم تتحول في الداتابيز لـ **Nullable Column**. ليه؟ لأن لو السطر ده بيمثل "كاش"، مفيش ReceiptCode ، فلازم يقبل Null.

2. تحسين TPH باستخدام Fluent API

الطريقة اللي فاتت شغالة، بس مش أحسن حاجة لأن عندنا DbSet لكل نوع. الأفضل نعمل **Abstract Base Class** بلم الليلة دي كلها، ونستخدم DbSet واحد بس.

التصميم الجديد للكلاسات:

هنعمل كلاس أب abstract اسمه Payment ، ونخلي الأنواع تورث منه.

```

1 // Abstract Base Class
2 public abstract class Payment
3 {
4     public int PaymentId { get; set; }
5     public PTypes PType { get; set; } // Enum عشان نحدد النوع
6     public decimal Amount { get; set; }
7 }
8
9 public class PaymentCash : Payment
10 {
11     // مفيش خصائص زيادة هنا
12 }
13
14 public class PaymentCard : Payment
15 {
16     public string ReceiptCode { get; set; }
17 }
18
19 // Enum للأنواع
20 public enum PTypes : byte { Cash = 1, Card = 2 }
```

الكود (DbContext Update): دلوقتي هنستخدم DbSet واحد للأب، وهنستخدم **Fluent API** عشان نعرف EF Core إننا هنستخدم الـ Enum كـ Discriminator.

```

1 public class Chapter08DbContext : DbContext
2 {
3     // DbSet للأب
4     public DbSet<Payment> Payments { get; set; }
5
6     protected override void OnModelCreating(ModelBuilder modelBuilder)
7     {
```

```

8      modelBuilder.Entity<Payment>()
9          // بنقله إن العمود اللي هيفرق الأنواع هو
10         .HasDiscriminator(b => b.PType)
11         // يبقى اوبجكت Cash، لو القيمة
12         .HasValue<PaymentCash>(PTypes.Cash)
13         // يبقى اوبجكت Card، لو القيمة
14         .HasValue<PaymentCard>(PTypes.Card);
15     }
16 }

```

التعامل مع البيانات في TPH (CRUD)

إزاي نضيف ونقرأ البيانات في الحالة دي؟

1. الإضافة (Create):

بتعمل `new` من النوع اللي انت عايزه وتضيفه عادي.

```

1  var payment = new PaymentCash { Amount = 100 };
2  context.Add(payment);
3  context.SaveChanges();

```

الـ EF Core هتخط في الداتابيز في عمود الـ Discriminator القيمة 1 (اللي هي Cash).

2. القراءة (Read):

لو عايز تجيب كل المدفوعات (كاش وفيزا):

```

1  var allPayments = context.Payments.ToList();

```

هيرجلك لسته فيها `PaymentCash` و `PaymentCard` ، كل واحد بنوعه الصحيح.

لو عايز تجيب الفيزا بس؟ نستخدم `OfType<T>` :

```

1  var cardPaymentsOnly = context.Payments.OfType<PaymentCard>().ToList();

```

ده هيتحول لـ SQL بيعمل Filter على عمود الـ Discriminator.

8.9.3 الـ Table Per Type (TPT)

الـ Each class has its own table

دي ميزة نزلت بداية من **EF Core 5**. هنا كل كلاس (الأب والأبناء) بيبقى ليه جدول مستقل بذاته في الداتابيز.

إمتى نستخدم TPT؟

- عكس الـ TPH، بنستخدمها لما يكون كل كلاس ابن عنده بيانات كثير ومختلفة تماماً عن الثاني.
- مثال: حاويات الشحن (Containers). عندنا حاوية شحن سفن (ShippingContainer) وحاوية بلاستيك صغيرة (PlasticContainer).
- الاثنين ليهم طول وعرض (مشارك).
- بتاعة السفن ليها "سمك حديد" و "نوع باب".
- البلاستيك ليها "لون" و "شكل".
- هنا TPT أفضل عشان مبيقاش عندنا جدول واحد فيه أعمدة كثير فاضية (Nulls).

إعداد TPT

هنعمل الكلاسات بتاعتنا عادي (وراثية):

الكود (Entity Classes):

```

1  // الأب (مشارك)
2  public abstract class Container
3  {
4      public int ContainerId { get; set; }
5      public int HeightMm { get; set; }
6      public int WidthMm { get; set; }
7      public int DepthMm { get; set; }
8  }
9
10 // النوع الأول
11 public class ShippingContainer : Container
12 {
13     public int ThicknessMm { get; set; }
14     public string DoorType { get; set; }
15     public bool Refrigerated { get; set; }
16 }
17
18 // النوع الثاني
19 public class PlasticContainer : Container
20 {
21     public int CapacityMl { get; set; }
22     public string ColorARGB { get; set; }
23 }
```

الكود (DbContext Configuration): السر كله هنا في استخدام ToTable . بنقول لـ EF Core حط كل واحد في جدول باسمه.

```

1  public class Chapter08DbContext : DbContext
2  {
3      // DbSet للأب عشان نقدر نستعلم عن كله
4      public DbSet<Container> Containers { get; set; }
5
6      protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```

7      {
8          // بتحدد اسم جدول لكل نوع
9          modelBuilder.Entity<ShippingContainer>()
10             .ToTable(nameof(ShippingContainer));
11
12          modelBuilder.Entity<PlasticContainer>()
13             .ToTable(nameof(PlasticContainer));
14
15          // DbSet من الـ Containers الأب ضمناً بياخذ اسم
16          // Entity<Container>().ToTable("Containers") أو ممكن تحدد بـ
17      }
18  }

```

شكل الداتابيز (Database Schema)

هيتكون عندنا 3 جداول:

1. الـ **Containers**: فيه `ContainerId` , `Height` , `Width` , `Depth` .
2. الـ **ShippingContainer**: فيه `ContainerId` (Foreign Key للأب وكـ Primary Key) , `Thickness` , `DoorType` .
3. الـ **PlasticContainer**: فيه `ContainerId` (Foreign Key للأب وكـ Primary Key) , `Color` , `Capacity`

التعامل مع البيانات في TPT (Querying)

الموضوع هنا سحري شوية.

لو عملت:

```
1 var all = context.Containers.ToList();
```

الـ EF Core هيرجوع يعمل **Joins** بين جدول الأب وجداول الأبناء عشان يجمعك البيانات، وهيرجلك `List<Container>` بس جواها اوبجكتات من نوع `ShippingContainer` و `PlasticContainer` حسب الداتابيز.

طرق تانية للكويري:

1. الـ **OfType**:

```
1 context.Containers.OfType<ShippingContainer>().ToList();
```

ده هيرجلك بس حاويات الشحن.

2. الـ **Set**: (أكثر كفاءة شوية في الـ SQL)

```
1 context.Set<ShippingContainer>().ToList();
```

بتعمل نفس النتيجة بس ال SQL Generated يكون مباشر أكثر على الجدول المطلوب.

ملخص سريع للمقارنة (عشان المذاكرة)

وجه المقارنة	TPH (Hierarchy)	TPT (Type)
عدد الجداول	جدول واحد لكل	جدول لكل كلاس
الأداء (Performance)	أسرع في ال Query (مفیش Joins)	أبطأ شوية (بيحتاج Joins)
حجم الداتايز	ممكن يبقى فيها Nulls كتير	منظمة ومفیش Nulls زيادة
الاستخدام	الكلاسات متشابهة جداً	الكلاسات مختلفة جداً في البيانات

ده كان شرح مفصل لجزئية ال TPH وال TPT.

تحب أكملك شرح باقي الطرق البديلة زي ال Owned Types وال Table Splitting؟

Owned Types

أنت جيت في منطقة مهمة جداً. اللي كنت بتسميه زمان في الـ EF6 باسم **"Complex Types"**، بقى اسمه في الـ EF Core الـ **"Owned Types"**.

Domain-Driven Design خصوصاً لو شغال بطريقة الـ Software Design، المفهوم ده بيحل مشاكل كتير في الـ **Value Objects** ومحتاج تطبيق فكرة الـ (DDD)

8.9.1 الـ Owned Types (Complex Types)

يعني إيه Owned Types؟

باختصار، ده عبارة عن "كلاس" عادي جداً بيثيل مجموعة بيانات مرتبطة ببعض (زي العنوان: شارع، مدينة، رقم بريدي)، لكن مش Entity قائم بذاته، يعني مالوش Primary Key خاص بيه.

- هويته بتعتمد بالكامل على الـ Entity اللي "بيملكه" (Owner).
- في لغة الـ DDD، بنسمي الحاجات دي Value Objects.

🔗 الفرق بين EF6 و EF Core في EF6 كنا بنسميها **Complex Types**، وكان أي كلاس مفيهوش **Primary Key** بيعتبر **Complex Type** أوتوماتيك. في **EF Core**، لازم تعرفه صراحة إن ده **Owned Type** عشان تتجنب الـ **Bugs**، وفيه ميزة زيادة إنك تقدر تخزن الـ **Owned Type** ده في جدول منفصل (مش بس في نفس الجدول زي زمان).

إحنا عندنا طريقتين نستخدم بيهم الـ Owned Types:

1. البيانات تتخزن في نفس الجدول مع الـ Entity الأساسي.
2. البيانات تتخزن في جدول منفصل.

الحالة الأولى: التخزين في نفس الجدول (Same Table)

دي الحالة الأشهر. تخيل عندنا **OrderInfo** (طلب شحن) ومحتاجين عنوانين: عنوان للفواتير (**BillingAddress**) وعنوان للتوصيل (**DeliveryAddress**).

بدل ما نكتب خصائص العنوان (شارع، مدينة، كود) مرتين جوه الـ **Order**، بنعمل كلاس اسمه **Address** ونستخدمه مرتين.

1. شكل الكود (Entity Classes)

```
1 // Owner Entity ده الـ
2 public class OrderInfo
3 {
4     public int OrderInfoId { get; set; } // Primary Key
5     public string OrderNumber { get; set; }
6
7     // مرتين Owned Type هنا استخدمنا الـ
8     public Address BillingAddress { get; set; }
9     public Address DeliveryAddress { get; set; }
10 }
11
12 // Owned Type (لاحظ الـ Attribute)
13 [Owned]
14 public class Address
15 {
```

```

16 // خالص Primary Key مفهوش
17 public string NumberAndStreet { get; set; }
18 public string City { get; set; }
19 public string ZipPostCode { get; set; }
20
21 [Required]
22 [MaxLength(2)]
23 public string CountryCodeIso2 { get; set; }
24 }

```

2. ال Configuration (لو مش هتستخدم Attribute)

لو مش عايز تحط [Owned] فوق كلاس العنوان، تقدر تعملها بالـ Fluent API في الـ OnModelCreating :

```

1 protected override void OnModelCreating(ModelBuilder modelBuilder)
2 {
3     modelBuilder.Entity<OrderInfo>()
4         .OwnsOne(p => p.BillingAddress); // بنقله ده مملوك ليك
5
6     modelBuilder.Entity<OrderInfo>()
7         .OwnsOne(p => p.DeliveryAddress);
8 }

```

3. شكل الداتابيز (SQL Generated)

الـ EF Core هيدمج كل الخصائص دي في جدول واحد اسمه Orders . وعشان يفرق بين العنوانين، بيغير أسامي العواميد (Columns) عن طريق إضافة اسم الـ Prefix كـ Property.

Orders									
OrderInfo Id (PK)	OrderNumber	BillingAddress_City	BillingAddress_NumberAndStreet	BillingAddress_ZipPostCode	BillingAddress_CountryCodeIso2	DeliveryAddress_City	DeliveryAddress_NumberAndStreet	DeliveryAddress_ZipPostCode	DeliveryAddress_CountryCodeIso2
1	ORD-2023-001	Cairo	123 Nile St	11511	EG	Alexandria	456 Corniche Rd	21523	EG
2	ORD-2023-002	Giza	789 Pyramids Ave	12511	EG	NULL	NULL	NULL	NULL
3	ORD-2023-003	Dubai	101 Sheikh Zayed Zayed Rd	00000	AE	Dubai	101 Sheikh Zayed Rd	00000	AE
4	ORD-2023-004	London	10 Downing St	SW1A 2AA	GB	Manchester	20 Piccadilly	M1 1AG	GB
5	ORD-2023-004	Cairo	10 Downing St	SW1A 2AA	GB	Manchester	20 Piccadilly	M1 1AG	GB

Entire Owned Type is null -> All its columns are NULL

Flattened 'BillingAddress' Owned Type

Flattened 'DeliveryAddress' Owned Type

الكود اللي هيطلع (SQL Server) هيكون كده:

```

1 CREATE TABLE [Orders] (
2     [OrderInfoId] int NOT NULL IDENTITY,
3     [OrderNumber] nvarchar(max) NULL,
4
5     -- Billing Address Columns
6     [BillingAddress_City] nvarchar(max) NULL,
7     [BillingAddress_NumberAndStreet] nvarchar(max) NULL,
8     [BillingAddress_ZipPostCode] nvarchar(max) NULL,
9     [BillingAddress_CountryCodeIso2] nvarchar(2) NULL, -- ركز هنا
10
11     -- Delivery Address Columns
12     [DeliveryAddress_City] nvarchar(max) NULL,
13     [DeliveryAddress_NumberAndStreet] nvarchar(max) NULL,
14     [DeliveryAddress_ZipPostCode] nvarchar(max) NULL,
15     [DeliveryAddress_CountryCodeIso2] nvarchar(2) NULL,
16
17     CONSTRAINT [PK_Orders] PRIMARY KEY ([OrderInfoId])
18 );

```

4. ملحوظة خطيرة بخصوص الـ Nullability

لو بصيت على الـ SQL فوق، هتلاقي العمود `BillingAddress_CountryCodeIso2` معمول `NULL` ، مع إننا حاطين عليه `[Required]` في الكود! ليه؟

- السبب: لأن الـ `Object` اللي اسمه `BillingAddress` كله على بعضه ممكن يكون `null` . فـ `EF Core` بتخلي كل عواميده تقبل `Null` عشان لو مفيش عنوان خالص، تخزن `NULL` في كل الخانات.
- القاعدة: لو كل العواميد بتاعة الـ `Owned Type` قيمتها `NULL` في الداتابيز، الـ `EF Core` ولما تيجي تقرأ البيانات هترجلك الـ `Property` دي بـ `null` في الكود.

الحل في **EF Core 5**: لو عايز تجبر إن لازم يكون فيه عنوان، بتستخدم `IsRequired()` على الـ `Navigation` نفسه:

```

1 modelBuilder.Entity<OrderInfo>()
2     .Navigation(p => p.DeliveryAddress)
3     .IsRequired(); // كده لازم يكون فيه داتا، والاعمدة هتبقى NOT NULL

```

الحالة الثانية: التخزين في جدول منفصل (Separate Table)

أحياناً بتبقى عايز تفصل بيانات الـ `Owned Type` في جدول تاني عشان متزحمش الجدول الرئيسي، بس لسه عايز تعامله كـ `Owned Type` (يعني ملكية خاصة ومفيش `reuse` فيه مع حد تاني).

مثال: عندنا `User` وعنده `HomeAddress` .

1. الـ Configuration

بنستخدم `.ToTable` بعد `OwnsOne` :

```

1 modelBuilder.Entity<User>()
2     .OwnsOne(p => p.HomeAddress)
3     .ToTable("Addresses"); // اسم الجدول المنفصل

```

2. اللي بيحصل في الخلفية

- الـ EF Core هيعمل جدولين: Users و Addresses .
- الجدول Addresses هيكون الـ Primary Key بتاعه هو نفس الـ Primary Key بتاع Users (علاقة 1-to-1).
- الـ Foreign Key ده بيكون OnDelete Cascade ، يعني لو مسحت اليوزر، عنوانه يتمسح أوتوماتيك.

3. شكل الداتا بيز (SQL)

```

1 CREATE TABLE [Users] (
2     [UserId] int NOT NULL IDENTITY,
3     [Name] nvarchar(max) NULL,
4     CONSTRAINT [PK_Users] PRIMARY KEY ([UserId])
5 );
6
7 CREATE TABLE [Addresses] (
8     [UserId] int NOT NULL, -- في نفس الوقت FK و PK ده
9     [City] nvarchar(max) NULL,
10    [CountryCodeIso2] nvarchar(2) NOT NULL, -- عادي NOT NULL هنا بقی
11
12    CONSTRAINT [PK_Addresses] PRIMARY KEY ([UserId]),
13    CONSTRAINT [FK_Addresses_Users_UserId] FOREIGN KEY ([UserId])
14        REFERENCES [Users] ([UserId]) ON DELETE CASCADE
15 );

```

① ميزة هنا في الحالة دي (جدول منفصل)، الخصائص الـ Required بتتحول لأعمدة NOT NULL عادي جداً، لأن الصف كله في جدول عناوين إما يكون موجود أو مش موجود.

ليه نستخدم Owned Types؟ (الخلاصة)

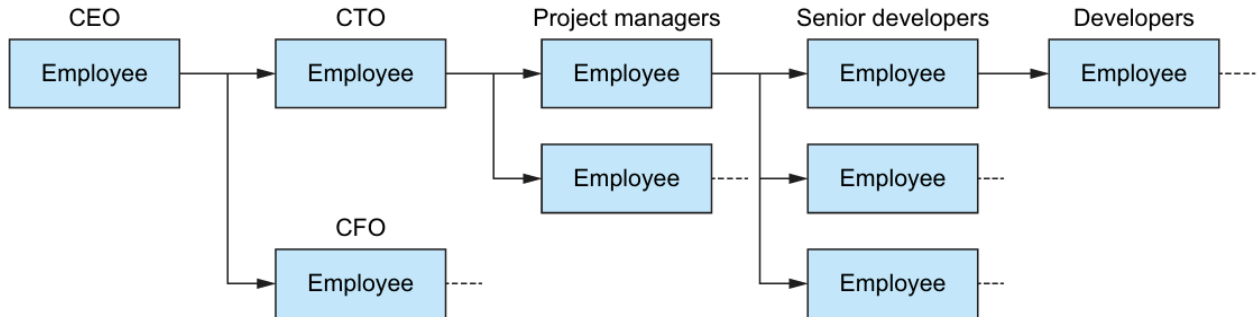
1. تنظيم الكود: بتجمع البيانات المرتبطة ببعض في كلاس واحد (Address) وبتقدر تستخدمه في أكثر من مكان (Billing و Delivery) من غير تكرار كود.
2. أداء عالي (Performance):
 - لو مخزنهم في نفس الجدول، بيبجوا مع الـ Query الأساسية من غير Join .
 - حتى لو في جدول منفصل، الـ EF Core بيعملهم Load أوتوماتيك (مش محتاج تعمل Include).
3. بديل لعلاقة One-to-One: جولي ليرمان (خبيرة الـ EF) بتصح باستخدام Owned Types بدلاً من علاقات 1-to-1 الصغيرة، لأنها أسرع وأسهل في التعامل.

4. الـ **Nested Owned Types**: تقدر تعمل Owned Type جوه Owned Type تاني (مثلاً CustomerContact جواه Address)، والـ EF Core هيفرد كل ده في الجدول الرئيسي (Flattening).

Reading in hierarchical data efficiently

قراءة البيانات الهرمية (Hierarchical Data) بذكاء

في الجزء ده، المؤلف بيحكينا عن مشكلة مشهورة جداً بتقابلنا لما يكون عندنا داتا "شجرية" أو هرمية (Hierarchical)، والمستويات فيها غير محددة (Unknown Depth).



Hierarchical data can have an unknown level of depth—that is, you don't know how many `.Include(x => x.WorksForMe)` but need to load all the data.

1. أصل المشكلة (The Scenario)

المؤلف كان شغال مع عميل عنده بيانات عبارة عن سلسلة من الـ Entities المتربطة ببعضها لعمق غير محدد (زي شجرة الموظفين في شركة: CEO تحته CTO تحته Managers تحتهم Developers وهكذا).

- المطلوب: كان محتاج يقرأ الهيكل (Hierarchy) ده كله عشان يعرضه.
- الحل التقليدي (واللي طلع بطيء):
- في البداية، استخدم الـ **Eager Loading** (يعني `Include` و `ThenInclude`) لأول مستويين.
- وبعدين للمستويات الأعمق استخدم **Explicit Loading** (يعني يروح يعمل Query لكل مستوى لوحده).
- النتيجة: الطريقة دي اشتغلت، بس الأداء (Performance) كان سيء جداً، والداتا بيز كانت بتصرخ من كتر الـ Requests الصغيرة اللي رايحة جاية.

2. الفكرة الذكية (The "Aha!" Moment)

المؤلف سأل نفسه: "طالما الـ **Relational Fixup** (اللي شرحناه قبل كده إنه بيربط الـ Entities ببعضها في الميموري) ذكي أوي كده، هل ممكن يساعدني أحسن الأداء؟". والإجابة كانت: أيوة!

تعال نضرب مثال بجدول الموظفين (Employees) عشان نفهم الفكرة: تخيل عندنا هيكل شركة زي الصورة دي:

- الـ **CEO** (رئيس مجلس الإدارة).
- تحته **CTO**.
- تحته **Project Managers**.
- تحتهم **Senior Developers**.
- تحتهم **Developers**.

المشكلة هنا إنك متعرفش الشجرة دي نازلة لحد فين، يعني متعرفش هتحتاج كام . Include(x => x.WorksForMe).

ملحوظة مهمة الحل اللي واصلنا له إن واحدة بس . Include. كفاية جداً! وباقي الشجرة هيتربط ببعضه أوتوماتيك بفضل الـ Relational Fixup.

3. الحل بالكود (The Code Solution)

بدل ما تعمل ThenInclude كتير، إحنا هنعمل Query واحدة بس تجيب كل الموظفين اللي احنا عايزينهم، و EF Core هيربطهم ببعض في الميموري.

المثال ده بيحجب كل الموظفين اللي شغالين في قسم الـ Development وعلاقاتهم ببعض:

```
1 // واحدة بس Query SQL الكود ده بيتحول لـ
2 var devDept = context.Employees
3     // مرة واحدة بس للعلاقة اللي بتربط الموظف بمديره/أو مرؤوسيه Include بنعمل
4     .Include(x => x.WorksForMe)
5     // بس Development بنفلتر الموظفين اللي شغالين في الـ
6     .Where(x => x.WhatTheyDo.HasFlag(Roles.Development))
7     .ToList(); // التنفيذ هنا بيحمل الداتا في الميموري
```

شرح اللي حصل جوه EF Core:

1. الـ Query دي راحت جابت كل الموظفين (Employees) اللي في قسم الـ Development من الداتابيز دفعة واحدة.
2. بمجرد ما البيانات دي وصلت للميموري، الـ Relational Fixup اشتغل.
3. بدأ يبص على الـ Primary Keys والـ Foreign Keys، ولقى إن الموظف (أ) هو مدير الموظف (ب)، فراح رابطهم ببعض في الـ Navigation Properties أوتوماتيك.
4. النتيجة: بقى عندك شجرة كاملة متربطة ببعضها من غير ما تكتب ThenInclude مليون مرة.

4. شروط وتنبيهات مهمة (Constraints)

عشان التريكة دي تشتغل معاك صح، لازم تاخذ بالك من نقطتين:

(أ) لازم تستخدم Tracking (ممنوع AsNoTracking)

- لو استخدمت AsNoTracking ، الطريقة دي مش هتشتغل.
- ليه؟ لأن ربط العلاقات ببعضها (Wiring up) بيعتمد كلياً على ميزة الـ Relational Fixup الموجودة جوه الـ Change Tracker بتاع EF Core. والـ AsNoTracking أصلاً بتلغي الميزة دي.
- البديل: لو عايز الداتا للقراءة فقط (Read-only) وعايز تستفيد من السرعة دي، استخدم AsNoTrackingWithIdentityResolution ، دي ممكن تشتغل لأنها بتحافظ على تعريف هوية الـ Entities، بس المؤلف ركز إن AsNoTracking العادية مش هتنفع.

ب) تختار مين في الـ Include؟

في العلاقات الهرمية (Hierarchical)، عادة يكون عندك علاقتين:

1. الـ Manager (ده الـ Parent - علاقة Single).
2. الـ WorksForMe (دول الـ Children - علاقة Collection).

المؤلف لاحظ حاجة غريبة ومهمة جداً:

- لما عمل Include للـ WorksForMe (الكوليكتشن)، الـ EF Core ملأ الطرفين: ملأ ليستة الموظفين (WorksForMe) وملأ كمان خاصية المدير (Manager).
- لكن لما عمل Include للـ Manager بس، الـ Collection (WorksForMe) جت فاضية (Null) لو مكش فيه داتا تانية تجبرها تتحمل.

⚠ تحذير لو عملت Include للـ Manager بس، الـ WorksForMe collection مش هتتملي إلا لو فيه Entities تانية مجبرة ترتبط بيها. عشان كده الأضمن تعمل Include للـ Collection (اللي هي WorksForMe).

ملخص سريع للنقطة دي للـ Obsidian

- **المشكلة:** تحميل بيانات شجرية (Hierarchical) بعمق غير معروف بيسبب مشاكل في الأداء لو استخدمنا ThenInclude كثير أو Lazy Loading.
- **الحل السحري:** استخدم Query واحدة، واعمل Include لمستوى واحد، وهات كل الداتا اللي تهملك ToList().
- **السر:** الـ Relational Fixup بتاع EF Core هيقوم بالواجب ويربط كل الـ Entities ببعضها في الميموري بناءً على الـ Foreign Keys.
- **الشرط:** لازم الـ Query تكون Tracked (يعني بلاش AsNoTracking العادية).
- **نصيحة:** اعمل Include للـ Collection Property (زي WorksForMe) مش للـ Single Property (زي Manager) عشان تضمن إن العلاقات تتربط صح من الناحيتين.

Relational Fixup Stage

يعني إيه Relational Fixup؟

ببساطة شديدة، لما بتعمل Query عشان تقرأ داتا من الداتابيز، الـ EF Core بيمر بمرحلة اسمها **Relational Fixup**. في المرحلة دي، الـ EF Core بيبدأ "يملا" الـ **Navigational Properties** (زي الليستات أو الـ **Objects** المرتبطة) بناءً على الداتا اللي موجودة عنده بالفعل في الـ **Memory (Tracked Entities)**.

الفكرة الجوهرية: الـ **Relational Fixup** مش بس بيص على الداتا اللي راجعة من الـ **Query** الحالية بتاعتك، لا.. ده كمان بيص على كل الـ **Entities** اللي معمولة لها **Tracking** حالياً في الـ **DbContext**.

يعني لو الداتا موجودة في الميموري، الـ EF Core هيقوم موصل الأسلاك ببعضها (**Links**) بناءً على الـ **Foreign Keys**.

إزاي العملية دي بتحصل؟ (مثال عملي)

عشان نفهم الموضوع صح، خلينا نبص على المثال اللي في الكتاب (Figure 6.1) اللي بيقرن بين طريقتين لتحميل كتاب (**Book**) مع المراجعات بتاعته (**Reviews**).

الطريقة الأولى: استخدام Include (الطريقة التقليدية)

هنا إحنا بنقول لـ EF Core هات الكتاب وهات معاه المراجعات في نفس الـ **Query**.

```
1 // 1. One query, using Include
2 var book = context.Books
3     .Include(x => x.Reviews) // بنحمل المراجعات مباشرة
4     .Single(x => x.BookId == bookId);
5
6 // النتيجة:
7 // book.Reviews.Count (أو حسب العدد في الداتابيز)
8 // مليانة وجاهزة Navigation Property الـ
```

الطريقة الثانية: استخدام استعلامين منفصلين (Relational Fixup Magic)

هنا بقى بيظهر ذكاء الـ EF Core. إحنا هنجيب الكتاب لوحده، وبعدين هنجيب المراجعات لوحدها في **Query** تانية خالص.

```
1 // 2. Two separate queries
2
3 // الخطوة 1: هات الكتاب بس من غير
4 var book = context.Books
5     .Single(x => x.BookId == bookId);
6
7 // book.Reviews فاضية (null أو count = 0) في اللحظة دي
8
9 // الخطوة 2: هات المراجعات الخاصة بالكتاب ده في ليستة منفصلة
10 var reviews = context.Set<Review>()
```

```

11         .Where(x => x.BookId == bookId)
12         .ToList();
13
14     // المفاجأة هنا !
15     // اللي جنباه في الخطوة 1 book روح بص ثاني على الاوبيجكت
16     // book.Reviews.Count 2 هتلاقها بقت !

```

إيه اللي حصل في الطريقة الثانية؟

1. لما جبت الـ `Reviews` في الخطوة الثانية، الـ `EF Core` بدأ يعمل `Track` للـ `Reviews` دي.
2. اشتغلت عملية `Relational Fixup`.
3. الـ `EF Core` لاحظ إن الـ `Reviews` دي عندها `Foreign Key` (اللي هو `BookId`) بيشاور على `Book` هو أصلاً عامل له `Track` (اللي جنباه في الخطوة 1).
4. قام أوتوماتيكياً رابط الـ `Reviews` دي بالـ `Book` ده، وملا الـ `Navigation Property` اللي اسمها `Reviews` جوه الـ `Book`، وكمان ملا الـ `Navigation Property` اللي اسمها `Book` جوه الـ `Review`.

🔗 ملحوظة مهمة النتيجة النهائية في الحالتين واحدة: الـ `Book` والـ `Reviews` بقوا مربوطين ببعض في الميموري.

شرط أساسي: Tracking

عشان الـ `Relational Fixup` يشتغل، لازم الـ `Entities` تكون `Tracked`.

- لو استخدمت `AsNoTracking()`، الـ `EF Core` مش هيحفظ بالـ `Entities` دي في الـ `Change Tracker`، وبالتالي مش هيقدر يعمل عملية الربط السحرية دي بين استعلام والثاني.
- الـ `Relational Fixup` بيشتغل أوتوماتيك طالما الـ `Query` بتاعتك "Read-Write" عادية (من غير `AsNoTracking`).

سيناريو متقدم: تجميع الداتا المتفرقة

الكتاب بي طرح مثال قوي جداً إنك ممكن تحمل داتا متفرقة، والـ `EF Core` يجمعها ببعضها زي البازل (Puzzle).

تخيل الكود ده:

```

1 var books = context.Books.ToList();
2 var reviews = context.Set<Review>().ToList();
3 var authorsLinks = context.Set<BookAuthor>().ToList();
4 var authors = context.Authors.ToList();

```

إيه اللي هيجصل بعد السطور دي ما تنتفذ؟

- الكتب (`books`) هتلاقي جواها الـ `Reviews` بتاعتها اتملت.

- الكتب هتلاقي جواها الـ `AuthorsLink` اتملت.
- جدول الوصل (`BookAuthor`) هتلاقي جواه الـ `Book` والـ `Author` اتربطوا صح.
- كل ده حصل لأن الـ `EF Core` عمل `Fixup` للعلاقات دي كلها بناءً على الـ `Keys` الموجودة، رغم إنك حملت كل جدول في سطر لوحده!

الخلاصة (Summary)

- **التعريف:** الـ `Relational Fixup` هي عملية ربط الـ `Navigation Properties` ببعضها أوتوماتيكياً.
- **التوقيت:** بتحصل لما الـ `Query` تخلص وتجب داتا.
- **النطاق:** بتبص على الداتا اللي لسه جاية + الداتا القديمة اللي معمولة لها `Tracking` في الـ `Context`.
- **الفائدة:** بتخليك تقدر تحمل `Data` على مراحل (أو `Queries` منفصلة) والـ `EF Core` يضمن إن الـ `Graph` بناع الاوبجيكتس سليم ومتربط ببعضه.

الـ `Understanding AsNoTracking and its variants`

لما بنعمل `Query`، إحنا غالباً بنكون عاوزين حاجة من اتنين:

1. الـ `Read-Write`: هنعدل على الداتا ونعمل `SaveChanges` . (ده الطبيعي اللي بيعمل `Tracking`).
2. الـ `Read-Only`: هنعرض الداتا بس (للقراءة)، زي تقرير أو قائمة للعرض.

عشان نحسن الأداء في حالة الـ `Read-Only`، الـ `EF Core` بتقدم لنا حلين: `AsNoTracking` و `AsNoTrackingWithIdentityResolution` . تعال نفهم الفرق بينهم وبين الاستعلام العادي.

1. الفرق بين الثلاثة أنواع (Normal vs NoTracking vs IdentityResolution)

أ) Normal Query (Tracking)

- **الوصف:** ده الطبيعي بتاعك. الـ `EF Core` بتعمل `Track` لكل `Entity` راجع.
- **الميزة:** ببيضمن إن مفيش تكرر لنفس الـ `Row` في الميموري (`Identity Resolution` شغال).
- **العيب:** أبطأ شوية وبيستهلك ميموري أكثر عشان الـ `Snapshot Tracking`.

ب) `AsNoTracking` (السرعة القصوى)

- **الوصف:** بتقول لـ `EF Core` "هات الداتا ومتعلمش لها `Tracking`".
- **الميزة:** أسرع وقت تنفيذ (`Quicker query time`).
- **العيب:** مفيهوش `Identity Resolution`. وده معناه إنه ممكن يكرر الـ `Objects` في الميموري لنفس الـ `Row` في الداتابيز.

ج) `AsNoTrackingWithIdentityResolution` (الوسط)

- **الوصف:** زي `AsNoTracking` بس بيشغل ميزة الـ `Identity Resolution`.

- **الميزة:** يضمن إن العلاقات تكون سليمة ومفيش تكرار للـ Objects (نسخة واحدة لكل Row).
- **الأداء:** أسرع من الـ Normal Query، بس أبطأ شوية من الـ AsNoTracking الصريف.

2. يعني إيه Identity Resolution؟ (مثال عملي)

عشان تفهم المشكلة اللي الـ AsNoTracking بتعملها، تخيل عندنا داتا بيز فيها 4 كتب، وأول كتابين لنفس المؤلف (مثلاً "Martin Fowler").

لو عملنا Query تجيب الكتب دي مع المؤلفين:

سيناريو الـ AsNoTracking : الـ EF Core هتعمل 4 Objects من نوع Author ، اتنين منهم بنفس البيانات (Martin Fowler). يعني في الميموري ه يكون عندك نسختين مختلفتين لنفس الشخص.

⚠ تحذير لو بتعرض الاسم بس، مش هتفرق معاك. لكن لو بتعتمد على العلاقات (Reference Equality) أو بتعمل Report يجمع كتب المؤلف الواحد، الموضوع ده هيعملك مشاكل.

سيناريو الـ AsNoTrackingWithIdentityResolution (أو العادي): الـ EF Core هتعمل 3 Objects بس من نوع Author . وأول كتابين هيشاوروا (Reference) على نفس الـ Object بتاع "Martin Fowler".

3. مقارنة الأداء (Performance Benchmark)

الكتاب عمل اختبار بسيط لتحميل 100 كتاب مع المراجعات والمؤلفين، والنتيجة كانت كالتالي :

نوع الاستعلام	الزمن (مللي ثانية)	نسبة الاختلاف	الملاحظات
Normal Query	95 ms	100%	الأساس للمقارنة (الأبطأ)
AsNoTracking	40 ms	42%	الأسرع (تقريباً الضعف)
WithIdentityResolution	85 ms	90%	أسرع قليلاً من العادي

📌 نصيحة استخدم الـ AsNoTracking دائماً في الـ Read-Only Queries طالما مش محتاج Identity Resolution، لأنه بيوفر كثير في الأداء.

4. تأثير AsNoTracking على الـ Relational Fixup

دي نقطة فنية مهمة جداً "المحترفين": في الـ AsNoTracking (بنوعيه)، عملية الـ Relational Fixup بتشتغل فقط داخل حدود الـ Query الواحدة.

- في الـ **Normal Query**: الـ **Fixup** يبيص على كل الـ **Tracked Entities** في الـ **Context** (زي ما شرحنا في الجزء اللي فات).
- في الـ **NoTracking**: لو عملت 2 **Queries** منفصلين بييجيوا نفس الداتا، الـ **EF Core** هتعمل **Objects** جديدة في كل مرة، ومش هتربطهم ببعض لأنها مش شايقة القديم أصلاً (لأنه مش **Tracked**).

5. الخلاصة: إمتى أستخدم إيه؟

1. عايز تعدل داتا؟ -> استخدم **Normal Query** (بدون أي حاجة).
2. عايز تعرض داتا بس (**Read-Only**) والسرعة هي الأهم؟ -> استخدم **AsNoTracking**.
3. عايز تعرض داتا بس، بس محتاج العلاقات تكون مطبوعة (بدون تكرار **Objects**)؟ -> استخدم **AsNoTrackingWithIdentityResolution**.

طيب ازاى انا عامل **AsNoTracking** او **AsNoTrackingWithIdentityResolution** وفي نفس الوقت حصل **Relational Fixup** مع انك قولت ان من شروط **Fixup** يكون فيه **Tracking** ؟

السر كله في كلمة واحدة: "النطاق" (**Scope**).

الـ **Fixup** بيشتغل في حالتين، وكل حالة ليها "حدود" مختلفة:

1. الـ **Fixup** مع الـ **Tracking** (المخزن الكبير)

لما بتشتغل عادي (**Tracking**)، الـ **Context** بتاعك بيبقى عامل زي "مخزن كبير".

- أي كويري بتعملها بترمي داتا في المخزن ده.
- الـ **Relational Fixup** هنا بيبص على المخزن كله.
- النتيجة: لو جيت كتاب في كويري، وبعدين جيت مؤلف في كويري تانية بعدها بساعة، الـ **EF Core** هيربطهم ببعض فوراً لأن الاثنين في نفس "المخزن" (**Change Tracker**).

2. الـ **Fixup** مع الـ **NoTracking** (الشنطة المقفولة)

لما بتستخدم **AsNoTracking**، إنت بتقول لـ **EF Core**: "أنا مش عايز أأخذ حاجة في المخزن الكبير، هاتلي الداتا في شنطة وامشي".

هنا الـ **Fixup** بيشتغل برضه، بس جوه حدود "الشنطة" دي فقط (**The Query Scope**).

يعني إيه؟

لما بتعمل كويري فيها **Include** وتستخدم **AsNoTracking**:

```
1 var books = context.Books
```

```

2      .AsNoTracking()
3      .Include(b => b.Reviews) // ركز هنا
4      .ToList();

```

1. الكويري دي راحت الداتا بيز وجابت (Books + Reviews).
2. البيانات دي كلها رجعت في "نفس الرحلة" (نفس الـ Result Set).
3. الـ EF Core وهو بيحول الداتا دي لـ Objects (Materialization)، لازم يربط الكتاب بالمراجعات بتاعته عشان يدرك النتيجة صح.
4. هنا الـ Relational Fixup بيشتغل، بس بيبص فقط على الداتا اللي راجعة في نفس الكويري دي.

🔗 الخلاصة في حالة `AsNoTracking` ، الـ `Fixup` بيشتغل "محلياً" (Locally) جوه الكويري الواحدة عشان يربط الـ `Parent` بالـ `Children` اللي جاينين معاه في نفس الطلب.

إيه الدليل إن الـ `NoTracking Fixup` محدود؟

لو جربت تعمل حركتين منفصلين بـ `AsNoTracking` ، الـ `Fixup` مش هيشغل بينهم، وده الفرق الجوهرى عن الـ `Tracking`.

مثال يوضح الفرق:

```

1 // 1. هات الكتاب (NoTracking)
2 var book = context.Books.AsNoTracking().First(b => b.Id == 1);
3
4 // 2. هات المراجعات (NoTracking)
5 var reviews = context.Reviews.AsNoTracking().Where(r => r.BookId ==
6     1).ToList();
7
8 // النتيجة هنا:
9 // book.Reviews هتفضل فاضية (Count = 0)
10 // بتاع الكويري الثانية ميعرفش حاجة عن الكويري الأولى Fixup ليه؟ لأن الـ
11 // يربطهم ببعض (Tracker) كل واحدة فيهم كانت "شنطة مقفولة" ومفيش مخزن

```

لو شلت `AsNoTracking` من الكود اللي فوق: الـ `book.Reviews` هتتملي أوتوماتيك، لأنهم اتقابلوا في "المخزن الكبير" (Change Tracker).

نقطة فنية (Identity Resolution في NoTracking)

لما بتستخدم `AsNoTrackingWithIdentityResolution` ، الـ EF Core بيعمل "مخزن مؤقت" (Temporary Tracker) عمره هو عمر الكويري بس.

1. بيبدا الكويري -> يفتح مخزن صغير.

2. يخطط فيه الـ Entities وهو يقرأها.
3. يخطط الـ Fixup ويمنع التكرار باستخدام المخزن الصغير ده.
4. أول ما الكويري تخلص -> يرمي المخزن الصغير ده ويرجعك النتيجة.

إذن:

- شرط الـ Fixup إنه يكون فيه Tracking؟ نعم، بس في حالة NoTracking التتبع بيبقى لحظي ومحدود بحدود الكويري عشان يربط الـ Includes ببعضها، ومش دائم على مستوى الـ Context.